

Biophysically Based Texturing: A Spatially Aware Autoencoder Approach to Interactive Human Skin Rendering

by

Joel Johnson

B.A.Sc., The University of British Columbia, 2022

A THESIS SUBMITTED IN PARTIAL FULFILLMENT OF
THE REQUIREMENTS FOR THE DEGREE OF

MASTER OF APPLIED SCIENCE

in

The College of Graduate Studies

(Electrical Engineering)

THE UNIVERSITY OF BRITISH COLUMBIA

(Okanagan)

August 2024

© Joel Johnson, 2024

The following individuals certify that they have read, and recommend to the College of Graduate Studies for acceptance, a thesis entitled:

BIOPHYSICALLY BASED TEXTURING: A SPATIALLY AWARE AUTOENCODER APPROACH
TO INTERACTIVE HUMAN SKIN RENDERING

submitted by JOEL JOHNSON in partial fulfilment of the requirements of the degree of
Master of Applied Science

Dr. Kenneth Chau, School of Engineering
Supervisor

Dr. Patricia Lasserre, Irving K. Barber Faculty of Science
Supervisory Committee Member

Dr. Zheng Liu, School of Engineering
Supervisory Committee Member

Dr. Gemma Rodriguez-Perez, Irving K. Barber Faculty of Science
University Examiner

Dr. Nicolas Peleato, School of Engineering
External Examiner

Abstract

This work aims to produce a biologically and physically accurate model for the relationship between skin structure and color that is practical for real-time rendering and editing of skin appearance. The motivation behind this endeavour stems from the need to enhance visual realism in digital media and to improve the inclusivity of training data for computer vision applications. As virtual and augmented realities become more prevalent, the demand for realistic skin rendering grows, necessitating a model that captures the subtle nuances of skin tones and textures and allows for dynamic modifications in real-time.

Monte Carlo light transport simulations are utilized to generate a comprehensive tabular dataset encompassing skin chromophore amounts, thicknesses, reflectance distributions, and corresponding RGB colors. This dataset serves as a Lookup Table (LUT) for converting diffuse 3D images and albedo textures into their corresponding biophysical parameter maps. Supervised autoencoders (SAEs) are used to learn the relationship between opto-structural skin parameters and color to facilitate a more efficient mapping of the data. We then evaluate the SAE in parameter map generation, image recovery, and image editing, aiming for the most biophysically accurate representation of skin color. We iterate on the simulation inputs and SAE model parameters, using these evaluations to find the ideal training data and SAE architecture.

We then create a procedural data pipeline for generating synthetic human/landmark pairs, utilizing SAEs for biophysically based skin color augmentation of diffuse albedo textures. Using this synthetic data, we train a DLM for facial landmark detection. Finally, we demonstrate a novel combination of DLMs and SAEs to facilitate spatially coherent alterations within the latent space, ensuring realistic and accurate transfer of latent biophysical properties between different images of human skin. This novel approach facilitates photo-realistic, real-time skin rendering and enables a biologically based spatial distribution of skin characteristics.

This comprehensive methodology sets a new standard for skin rendering, editing, and generation, combining biological accuracy with practical applicability to enhance realism and inclusivity in digital humans.

Lay Summary

This work models the relationship between skin structure and color, enabling dynamic mapping applicable to games, movies, extended reality, and biomedical applications. We leverage simulation-based methods to generate data, which is subsequently used to decompose images of human skin into maps for various skin properties. The use of machine learning allows this bidirectional mapping to be done in a fast and intuitive manner. We then utilize this mapping to augment data and expand image datasets of human faces. Finally, we combine this skin color augmentation model with facial landmark recognition models to better represent the spatial distribution of skin properties.

Preface

This thesis is the result of my independent research efforts, with guidance and feedback provided by my research supervisor, Dr. Kenneth Chau, and my industry supervisor, Wei Sen Loi, through weekly meetings. I was the sole contributor to the work, excluding Section 4.2, which covers aspects of a SIGGRAPH poster in which I was the principal investigator. My contributions cover the identification and design of the research, the analysis of research data, and the preparation of the manuscript.

In addition to my supervisor’s guidance, I held weekly meetings with my research team at Huawei Technologies Canada, where my work with this team influenced early research directions. Wei Sen Loi and Abraham Beauferris were key contributors in these early meetings, helping me get up to speed in the field of computer graphics and providing valuable feedback. Wei Sen Loi also played a significant role in designing the SIGGRAPH poster, and together with the team, they supported editing the poster abstract. The research presented in this thesis has resulted in the publication of a SIGGRAPH Asia 2023 poster titled “Deep Albedo: A Spatially Aware Autoencoder Approach to Interactive Human Skin Rendering,” co-authored with Joel Johnson, Kenneth Chau, Wei Sen Loi, Abraham Beauferris, Swati Kanwal, and Yingqian Gu. The content of this publication is incorporated in Section 4.2, “Biophysically Based Texturing,” of this thesis. Additionally, Figures 4.1 and 5.1 incorporate rendered results used in the poster.

Generative Artificial Intelligence (AI) was utilized in the following ways: GitHub Copilot was used to assist with programming tasks, and ChatGPT and Claude were employed for grammar, spelling, and writing feedback. These tools provided support in refining the technical and written aspects of the thesis.

The MetaHuman assets used in this research (Figures 3.7, 3.10, 3.11, and 4.14) are subject to the MetaHuman Creator End User License Agreement. By using MetaHuman Creator and its assets, I have agreed to the terms of this agreement, which can be found at <https://www.unrealengine.com/en-US/eula/mhc>. The use of MetaHuman Technology in this thesis complies with the terms of this agreement, including the license grant and usage restrictions outlined therein.

This research also utilizes the ICT-3DRFE Database (3D Relightable Facial Expression) from the University of Southern California’s Institute for Creative Technologies. The use of this database is subject to the End User License Agreement available at <http://projects.ict.usc.edu/3drfe/>. In accordance with this agreement, the database has been used solely for research purposes, and all publications resulting from this research will properly

acknowledge its use by citing the following papers [SGDM11, MHP⁺07].

No research ethics approval was required for the work presented in this thesis.

Table of Contents

Abstract	iii
Lay Summary	iv
Preface	v
Table of Contents	vii
List of Tables	x
List of Figures	xi
Acknowledgements	xv
1 Introduction	1
1.1 Research Objectives and Key Contributions	2
1.2 Physically Based Rendering	2
1.2.1 Key Definitions	3
1.3 Real-Time Physically Based Rendering	6
1.4 Biologically Based Human Skin Appearance	6
1.4.1 Overview of Human Skin Structure	6
1.4.2 Relationship Between Skin Structure and Appearance	8
1.4.3 Rendering Human Skin	9
1.5 Machine Learning and Rendering	12
1.6 Summary and Roadmap	13
2 Simulation of Light Transport	15
2.1 Monte Carlo Methods	16
2.2 Scattering and Absorption in Human Skin	18
2.3 Color Spaces	19
2.3.1 CIE XYZ Color Space	20
2.3.2 Linear RGB color space	22
2.3.3 sRGB Color Space	23
2.3.4 Metamers	23

2.4	Results and Discussion	23
2.4.1	Convergence of the Simulation	23
2.4.2	Reflectance Distributions, Color and Parameters	24
2.4.3	Top Layer Saturation	26
2.5	Conclusion	28
3	Mapping Between Skin Structure and Color	30
3.1	Methodology	30
3.1.1	LUT Search Based Mapping	30
3.1.2	Autoencoders	31
3.1.3	Performance Metrics	35
3.2	Results and Discussion	37
3.2.1	Implementation Details	37
3.2.2	LUT Search Maps	38
3.2.3	SAE Training	40
3.2.4	SAE Validation	41
3.3	Use of Real Images	49
3.4	Conclusion	50
4	Dense Landmarks and Spatial Distribution of Chromophores	51
4.1	Introduction	51
4.2	Biophysically Based Texturing	52
4.2.1	Age	53
4.2.2	Emotion	54
4.2.3	Re-topology and Texturing	54
4.3	Synthetic Data Pipeline	55
4.3.1	Affine Transformations	57
4.3.2	Coordinate Space Conversion	58
4.3.3	Procedurally Generated Humans	60
4.4	Data Augmentation Using SAEs	61
4.5	Training a Dense Landmark Model	63
4.6	Implementation	65
4.7	Results and Discussion	65
4.7.1	Synthetic Dataset	65
4.7.2	DLM	66
4.7.3	Spatially Coherent Texturing	67
4.8	Conclusion and Future Work	70
5	Conclusion	71
5.1	Limitations	72
5.2	Future Work	73

Table of Contents

Bibliography	74
Appendices	81
Appendix A: Figures	82

List of Tables

2.1	Biophysical parameter range and frequency of sampling used in the MC light transport simulation. The sampling frequency is selected approximately proportional to each parameter’s sum correlation with R, G, and B color channels.	19
2.2	Correlation of biophysical parameters with RGB channels.	26
3.1	Summary of function definitions and their gradients, where y and $\hat{y}$ are vectors representing the true and predicted values, respectively, for an arbitrary batch of data points. The ReLU activation function, as well as the L_1 and L_2 loss functions, are defined and computed element-wise for each component of these vectors. The gradients are similarly computed with respect to each predicted value $\hat{y}_i$ in the batch.	35
3.2	Perception of color differences based on ΔE values [MMC21].	36
3.3	Table of Delta E statistics for each texture.	44
3.4	Demonstration of RGB inputs outside the range of expected values and resulting latent and recovered values, as seen in textures 1 and 2 of Figure 3.2.2. We expect outliers in non-scan based or lit “in-the-wild” images. . . .	45
3.5	Average times to encode and decode diffuse albedo textures using the SAE and LUT search methods.	48

List of Figures

1.1	Visual depiction of ray tracing to accumulate physically based values for screen fragments (pixels), where (a) is the camera origin, (b) is the screen space fragment, (c) is the first intersection with a surface (vertex fragment), (d) is one light path from x_1 to the light source, (e) is the light source. . . .	5
1.2	Physically based material properties of 3D objects encompass specular and diffuse light interacting with complex meshes wrapped with various texture maps. Left to right: (a) rendered 3D model, (b) specular render pass, (c) diffuse render pass, (d) 3D mesh, and (e) diffuse albedo texture (skin). . . .	5
1.3	Structural representation of human skin. (a) Detailed 3D structure of human skin, illustrating various layers and components (image adapted from [Ber04]). (b) Simplified five-layer homogeneous model of skin commonly used in biomedical optics and physically based rendering, showing the approximation of complex skin structures into distinct optical layers (image adapted from [MKS ⁺ 19]).	8
1.4	Spectral absorption profiles for key chromophores in human skin (image adapted from [IGAJG15]).	9
1.5	(a) BRDFs represent the relationship between incoming and outgoing light that hits a surface, with outgoing rays originating at the same point as the incoming rays. (b) BSSRDFs model the effect of subsurface scattering by accounting for light that is scattered inside a material, in which outgoing rays can exit the material at an arbitrary location (image adapted from “A Practical Model for Subsurface Light Transport” [JMLH01]).	10
1.6	(a) Histogram of reflectance (R) across different radial distances (r), depicting a Sum of Gaussians Diffusion Profile alongside Reflectance Bins. (b) Radial distribution of light that has escaped, scattered, and absorbed in human skin, visualized through a Monte Carlo simulation to model the random walk of photons [JW95].	11

2.1	Cross-sectional view of a Monte Carlo simulation of photon transport in human skin, modeled with two layers: an epidermis of thickness T and a semi-infinite dermis. The blue vector shows the initial photon trajectory, while dashed lines trace its path at a specific wavelength, with direction and step size determined probabilistically. Green and red vectors depict escaped, and internally reflected photons, respectively.	15
2.2	Number of photons simulated per wavelength vs. convergence of average reflectance.	24
2.3	Reflectance distributions for the various rows of the dataset, with the colors corresponding to the derived RGB color from the LUT.	25
2.4	Simulation output of 45 unique C_m and C_h values. With C_m from 0.001 to 0.5, C_h from 0.001 to 0.32, and B_m , B_h , and T fixed at 0.5, 0.75 and 0.25 respectively.	27
2.5	Histogram of common metamers in the LUT.	28
3.1	Pixel-wise RGB to biophysical parameter supervised-autoencoder (SAE) flow chart with example input, latent and output values.	30
3.2	SAE high level diagram, showing the bidirectional and pixel-wise mapping between color and parameter values.	31
3.3	Encoder and decoder computation graphs, top and bottom respectively. . .	32
3.4	Maps generated by searching the LUT for each pixel in a given RGB texture. Top to bottom shows Fitzpatrick types 1 through 6. C_m [0.001,0.5], C_h [0.001,0.32], B_m [0,1], B_h [0,1], T [0.01,0.25]mm as in Table 2.1	39
3.5	SAE training and validation loss by epoch. Where loss = encoder loss (L1 norm) + decoder loss (L2 norm) + end to end loss (L1 norm).	40
3.6	Maps generated by pixel-wise SAE. Top to bottom shows Fitzpatrick types 1 through 6. C_m [0.001,0.5], C_h [0.001,0.32], B_m [0,1], B_h [0,1], T [0.01,0.25]mm as seen in Table 2.1	42
3.7	SAE recovered and original diffuse albedo texture and the rendered model for each.	43
3.8	Original, recovered and delta E color difference for no changes in latent space.	44
3.9	Columns left to right: Fitzpatrick types 2,4,5,6. Row 1: scaled down by 3. Row 2: melanin scaled up by a factor of 3. Row 3: hemoglobin scaled down by 3. Row 4: hemoglobin scaled up by a factor of 3.	46
3.10	Rendered models with $C_h * 1, 1.5, 2, 3$	47
3.11	Rendered models with $C_m * 1, 1.5, 2, 3$	48
3.12	Diffuse light cage images of faces from the 3DRF dataset [USC24, SGDM11, MHP ⁺ 07], decomposed into biophysical parameters using the SAE.	49

4.1	High level overview of our spatially aware, physically based texturing pipeline. The target texture is modified using an age example (Example ₁), and an increased blood perfusion example (Example ₂). Stage (a) the examples are transformed using point correspondence employing Dense Landmarks or manually using FaceForm Wrap. Stage (b) encodes the images from RGB to biophysical parameter maps. Stage (c) then blends the maps between the Target and Examples. Finally, the blended maps are decoded to produce biophysically plausible changes to skin color.	53
4.2	Example re-topology, a 3DMM generated from an image of the author is re-topologised in order to apply the texture to a Texturing XYZ model, using FaceForm Wrap to manually find point correspondences. (a) and (b) Show the manual point selection of Texturing XYZ model and 3DMM respectively. (c) 3DMM (top) and its diffuse albedo texture (bottom). (d) The wrapped 3DMM texture applied to the Texturing XYZ model (top) and the re-topologised texture (bottom). (e) Original Texturing XYZ model (top) and original texture (bottom).	55
4.3	Procedurally generated characters from our synthetic dataset.	56
4.4	Annotations for one of the many procedurally generated characters, and the corresponding annotated diffuse albedo map and eye texture map. (a) is the annotated render, (b) is the head texture, (c) is the eye texture(s).	57
4.5	Cameras are spawned at random vertices of a hemisphere set to the centroid location of our synthetic landmarks for each character.	60
4.6	In addition to generating the render/landmarks for each character, we save the UV coordinates, z values, and additional render passes to facilitate a useful open source dataset for various computer vision tasks.	61
4.7	SAE-based augmentations used to create a larger and more comprehensive set of textures in our synthetic data pipeline. (a) Left is the original texture dataset, right is the augmented textures with a wider range and more variance in melanin and hemoglobin. (b) The offline texture editing GUI we made for visual/interactive adjustments to skin chromophores and thickness.	62
4.8	Example augmentations applied to the rendered images and landmarks using pre-trained segmentation model [YWP ⁺ 18]. Where the top row is the original, and the bottom row (mirrored) has changes to skin, and hair using the SAE in addition to changes in clothing color.	63
4.9	Schematic of our DLM with MobileNetV2 backbone, showing the patch embedding, encoding and regression for a synthetic data point. Image adapted from “An Image is Worth 16x16 Words” [DBK ⁺ 20, VSP ⁺ 23].	64

4.10	Landmark annotations on procedurally generated humans. Object vertices are converted to screen space and used as synthetic landmarks. Color-coded annotations: green for general face structure, red for lips, pink for nostrils, light blue for ears, and blue for eye contours. The dataset includes additional vertices and extra render passes beyond those used in this work.	66
4.11	ViT based DLM training and validation loss by epoch.	67
4.12	DLM inference results for 107 landmark inference, where (a) shows the synthetic data, and (b) shows the diffuse light cage images from the 3DRF dataset [USC24]. The green markings are the true value of selected screen space vertices and the red is predicted.	68
4.13	Digital human rendered in Maya with latent biophysical parameters transferred from example images and textures using dense landmarks. Anger and age properties are transferred from the latent space of FaceScape images and a Texturing XYZ example texture, respectively [JCL ⁺ 23].	69
4.14	Metahumans rendered in Unreal Engine with our biophysically based texturing pipeline.	70
5.1	Maya render used in our work “Deep Albedo [JCL ⁺ 23],” where (a) is the original diffuse albedo texture, (b) shows increased blood perfusion, and (c) shows aged diffuse albedo texture.	71
2	Cm vs Ch color maps for fixed $T = 2.5mm$ and varying Bm and Bh	82
3	Two-dimensional scatter plot of escaped photon weights in polar coordinates. With the red weights, representing “overflow” weights, are placed in the outermost reflectance bins [JW95].	83
4	Diffuse light cage images of human faces [USC24, SGDM11, MHP ⁺ 07], decomposed into biophysical parameters using the SAE.	84

Acknowledgements

I express my deepest gratitude to my supervisor, Dr. Kenneth Chau, for opening doors I never thought possible. His leadership extends beyond academia, demonstrating a remarkable balance between research, entrepreneurship, and family. My sincere thanks go to Wei Sen Loi for his mentorship and dedication to improving my 3D computer graphics skills. His tireless efforts and expertise have significantly elevated the quality of this work. Finally, I owe a debt of gratitude to my wife, Domenic, for being patient with my many late nights during the course of this work. This work would not have been possible without your love, support, and the occasional reminder of who I am outside of academia.

Chapter 1

Introduction

Humans have evolved to be highly perceptive of changes in facial appearance. These subtle changes in geometry and color often tell us more than the words being spoken. Capturing these nuances of the human face is critical for immersive and photo-realistic applications across myriad fields such as biomedical imaging, virtual cosmetics, film production, games, and extended reality (XR).

Film production leverages physically based rendering techniques to create the most faithful representation of characters with uncanny material properties. Leveraging high-resolution scans of real actors to create digital twins for their VFX pipelines [BL03]. These digital twins allow actors to be aged or de-aged, as seen in films like Gemini Man and The Irishman [Gia19]. Companies like Weta FX rely on massive GPU render farms, often turning to cloud providers like AWS to scale up compute power with big projects, with scenes in movies like Avatar 2 taking millions of threaded hours to render individual shots [SLI⁺23]. All of this time, money, and compute is devoted to the meticulous simulation of light bouncing, scattering, and being absorbed in each frame. Games and XR experiences don't have the luxury of time, as new views are determined by the user, so physically based effects of light and materials must be streamlined for 30 or more frames per second on the fly [HHAM18].

Modelling and simulation of biological systems are common in medical fields, simulating whole systems or specific tissues at the cellular level. Realistic simulations are increasingly used to simulate challenging surgical procedures, planning procedures, and predictive modelling [ARVT⁺22]. Biophysically based models of human skin are used in biomedical imaging to identify skin cancer, leveraging similar methodologies used in film production to link diffuse reflectance to its biological structure [MVG⁺13]. Similar models for human skin are also used in the cosmetics industry to augment images for virtual make-up try-on applications [KK18].

The increasing power of GPUs has enabled more powerful AI models, which are increasingly trained using synthetic data [BABANM22]. Companies like Waymo and Tesla rely on simulations and reinforcement learning to train their self-driving cars, with Waymo utilizing orders of magnitude more synthetic data than real data [Toe22]. This paradigm is useful beyond training self-driving cars, much of the CV applications we currently use are focused on aspects of the human face. Procedurally generated digital humans imbued with biophysically based material properties give us a limitless and diverse representation of people who may otherwise not be included [WBH⁺21]. With the freedom to create

data using procedural and simulation-based methods, we can build more performant and representative CV-based applications.

With the ability to generate data on demand, we can leverage unbiased metrics and scales to objectively analyze data, ensuring that unconscious bias is not injected into the dataset. The Fitzpatrick scale ranks common skin types based on sensitivity to UV light, inversely proportional to melanin concentration, where type 1 is a very fair skin tone, and type 6 is a very dark skin tone [Fit88]. Our work uses this scale to ensure that our skin models and procedurally generated characters are representative of these six types as a minimum baseline.

1.1 Research Objectives and Key Contributions

This work aims to develop a real-time, biophysically accurate model for skin structure and color, enhancing visual realism in digital media, improving inclusivity in computer vision training data, and addressing the growing demand for realistic and dynamically modifiable skin appearance in virtual and augmented reality applications.

Our Contributions

- Monte Carlo simulation of diffuse reflectance data, statistically driven, to facilitate a learned supervised autoencoder (SAE)-based representation of human skin. We use an iterative approach to light transport simulation and training of SAEs and introduce heuristics for both to attain the most useful data for mapping between skin structure and color.
- Novel integration of SAEs and dense landmark models (DLMs) for realistic texturing and editing of skin appearance in a dynamic and spatially coherent manner.
- Creation of a procedural data pipeline for generating synthetic human/landmark pairs, utilizing SAEs for biophysically based skin color augmentation.

In the next sections, we will outline the rendering process and highlight the unique challenges of representing and modifying human skin. We then explore state-of-the-art methods for physically based human skin rendering and machine learning methods useful in bridging the gap in quality between offline and real-time quality.

1.2 Physically Based Rendering

Physically Based Rendering (PBR) simulates the physical interaction of light with objects and adheres to fundamental physical principles, such as conservation of energy, ensuring that surfaces do not reflect more light than they receive. It also leverages real-world data

and observations to achieve ground truth rendering. This methodology ensures that digital humans rendered with PBR look realistic and behave consistently under various lighting conditions, closely mimicking their real-world counterparts [PJH16].

To efficiently render 3D objects and scenes, we project light and geometry onto a 2D viewing plane. The fidelity of the render is greatly impacted by how light interacts with objects in the scene and how closely the rendering process either simulates or mimics the real world. In the real world, light can reach our eyes directly from a light source or after a practically infinite number of bounces. As infinite bounces would be impossible in a rendering engine, we rely on many clever mathematical tricks and simplifications to capture the most influential rays with the smallest number of bounces that generate satisfactory results.

1.2.1 Key Definitions

- **Real-Time Rendering:** Rendering of 3D objects to the screen on the fly, used in video games and extended reality [HHAM18].
- **Offline Rendering:** Rendering 3D objects to the screen with little or no time constraint, prioritizing quality of the image(s). Used in animation, film production, and architecture visualization [PJH17].
- **Ray Casting:** Used to determine visibility and material properties of 3D models by casting a ray from the camera, through the image plane, and into the scene [Fol96].
- **Ray Tracing:** A rendering technique that simulates light behavior by tracing rays from the camera into the scene. Classical ray tracing provides a limited set of effects including sharp reflections and hard shadows, focusing primarily on direct lighting calculations [PJH17, HHAM18].
- **Path Tracing:** An extension of ray tracing that uses Monte Carlo methods to solve the full rendering equation, accurately simulating both direct and indirect lighting. It traces multiple light paths per pixel, accounting for complex light interactions and global illumination effects, resulting in more physically accurate renderings [Kaj86, HHAM18].
- **Rasterization:** A method that converts 3D geometry to 2D images by projecting object vertices onto the screen and filling in the resulting triangles, widely used in real-time graphics due to its efficiency [Fol96].

The Rendering Equation: The rendering equation (1.1) is essential for modeling light interactions with objects in a scene. We trace a path from the camera through each pixel fragment on the screen, accumulating light from the first surface (vertex fragment) intersecting with this path. The rendering equation allows the accumulation of light from

all directions at a point, sampling direct and indirect incident rays and prioritizing the most significant illumination sources. Its analytical form calculates the outgoing light (L_o) in response to an incident ray (L_i) by integrating over the hemisphere surrounding the point being analyzed ($\mathbf{x}$). This equation is particularly important in the PBR rendering pipeline, where it is used to determine the final value of pixel fragments by accounting for how light interacts with the geometric surfaces, material properties, and light sources in a scene [Kaj86]. It is expressed as follows:

$$L_o(\mathbf{x}, \hat{\omega}_o, \lambda) = L_e(\mathbf{x}, \hat{\omega}_o, \lambda) + \int_{\Omega} f_r(\mathbf{x}, \hat{\omega}_i, \hat{\omega}_o, \lambda) \cdot L_i(\mathbf{x}, \hat{\omega}_i, \lambda) \cdot (\hat{n} \cdot \hat{\omega}_i) d\hat{\omega}_i \quad (1.1)$$

Each term of this equation has a specific role in determining how light is rendered in a scene:

- $L_o(\mathbf{x}, \hat{\omega}_o, \lambda)$ represents the outgoing light (radiance) from a point $\mathbf{x}$ in a particular direction $\hat{\omega}_o$ and wavelength λ .
- $L_e(\mathbf{x}, \hat{\omega}_o, \lambda)$ is the light emitted directly from the surface at point $\mathbf{x}$ in direction $\hat{\omega}_o$ and wavelength λ .
- $\int_{\Omega}$ indicates the integration over the hemisphere around the normal at point $\mathbf{x}$, accounting for all incoming light.
- $f_r(\mathbf{x}, \hat{\omega}_i, \hat{\omega}_o, \lambda)$ is the bidirectional reflectance distribution function (BRDF) at $\mathbf{x}$, which determines how light is reflected at that point for incoming light direction $\hat{\omega}_i$, outgoing direction $\hat{\omega}_o$, and wavelength λ .
- $L_i(\mathbf{x}, \hat{\omega}_i, \lambda)$ denotes the incoming light (irradiance) at $\mathbf{x}$ from direction $\hat{\omega}_i$ with wavelength λ .
- $(\hat{n} \cdot \hat{\omega}_i)$ is the Lambertian reflectance term, representing the attenuation of light due to the angle of incidence, calculated as the cosine of the angle between the normal at $\mathbf{x}$ and the incoming light direction $\hat{\omega}_i$.

In practice, the equation is computed recursively from the camera to the light source as seen in Figure (1.1) using ray tracing or other numerical methods to account for multiple scattering events and complex interactions within the scene.

The recursive form of the rendering equation is given by Equation 1.2 where $\mathbf{x}'$ is the point on the surface visible from $\mathbf{x}$ in direction $\hat{\omega}_i$ [Fol96]. In Figure 1.1 the rendering equation for point x_1 (x) samples light from all directions on the hemisphere Ω_1 with the outgoing light from x_2 (x') being one of the incident rays sampled.

$$L_o(\mathbf{x}, \hat{\omega}_o, \lambda) = L_e(\mathbf{x}, \hat{\omega}_o, \lambda) + \int_{\Omega} f_r(\mathbf{x}, \hat{\omega}_i \rightarrow \hat{\omega}_o, \lambda) \cdot L_o(\mathbf{x}', \hat{\omega}_i, \lambda) \cdot (\hat{n} \cdot \hat{\omega}_i) d\hat{\omega}_i \quad (1.2)$$

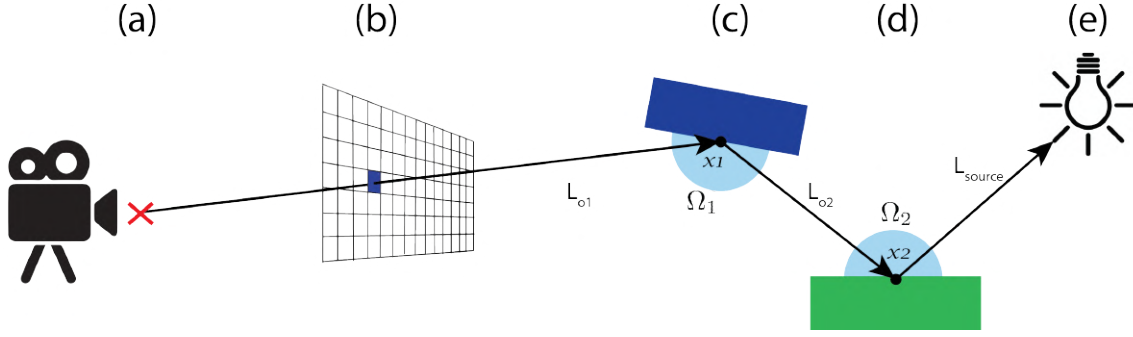


Figure 1.1: Visual depiction of ray tracing to accumulate physically based values for screen fragments (pixels), where (a) is the camera origin, (b) is the screen space fragment, (c) is the first intersection with a surface (vertex fragment), (d) is one light path from x_1 to the light source, (e) is the light source.

In this research, we will primarily focus on the BRDF component of the rendering equation, as the material properties play a key role in producing realistic-looking human skin. Accurately modeling the BRDF is crucial for capturing the subtle nuances of skin appearance, such as glossiness, subsurface scattering, and anisotropic reflections.

Materials: Textures and shaders are fundamental elements in 3D graphics, playing a crucial role in defining the appearance of surfaces in a rendered scene. Textures are essentially image data mapped onto the surfaces of 3D models, adding visual complexity and detail. They can represent diffuse and specular color, normal and displacement maps for simulating surface irregularities, and more. By applying textures, we imbue 3D models with a sense of realism; they mimic the intricate patterns found in real-world materials like wood grain, cloth, or skin [Fol96].



Figure 1.2: Physically based material properties of 3D objects encompass specular and diffuse light interacting with complex meshes wrapped with various texture maps. Left to right: (a) rendered 3D model, (b) specular render pass, (c) diffuse render pass, (d) 3D mesh, and (e) diffuse albedo texture (skin).

Shaders, however, are programs that run on the GPU that control how light interacts with surfaces. They take various inputs, including light positions, texture data, and the geometric properties of the surface, and compute the final color of pixels. Shaders are responsible for a wide range of visual effects, from simple flat shading to complex phenomena like translucency, reflection, and refraction. They control how light reflects off surfaces, scatters within translucent materials, and even bends when passing through transparent objects [Fol96].

1.3 Real-Time Physically Based Rendering

Video games and XR applications are non-deterministic due to the user’s freedom to move within virtual environments. Unlike offline rendering environments, where computational resources are less constrained, real-time rendering requires maintaining high frame rates while rendering objects and materials on demand.

Real-time rendering uses simplified calculations to maintain high frame rates, replacing complex light bounce simulations with efficient approximations like screen space reflections and ambient occlusion [RGK⁺09]. Shading models often use streamlined versions of BRDFs to balance visual quality and performance. Precomputed light-maps and irradiance maps are widely used to simulate complex lighting effects with minimal computational costs [SKS02], enhancing realism while maintaining performance. Sophisticated techniques like deferred rendering and Forward+ rendering efficiently handle multiple light sources and complex scenes [dRL11], optimizing performance without sacrificing visual quality. Hybrid rendering combines the speed of rasterization with advanced lighting calculations from ray tracing, adapting to various hardware capabilities and application needs [CNLE11]. Advances in GPU technology are enabling the integration of computationally intensive techniques like real-time global illumination and ray tracing into interactive applications [PJH17], closing the gap between real-time and offline rendering quality and likely impacting the popularity of certain shading and rendering algorithms.

1.4 Biologically Based Human Skin Appearance

1.4.1 Overview of Human Skin Structure

Human skin is a complex, multi-layered organ, with each layer serving unique structures and functions, including protection, temperature regulation, and sensory perception. Its structure consists of two main layers: epidermis and dermis, both with sub-layers and unique opto-structural properties and functions [Ber04].

In fields such as biomedical optics and physically based rendering, this complex structure is often simplified into layered models with homogeneous properties [MKS⁺19]. These models represent skin as a series of distinct layers, each characterized by specific optical

properties and chromophore concentrations. The number of layers can vary depending on the application and desired accuracy, with common models using anywhere from two to five layers [DJ06]. A typical five-layer model, as shown in Figure 1.3, might include a combined layer for the stratum corneum and epidermis, upper (papillary) dermis, reticular dermis, deep dermis and partial subcutis, and deep subcutis. Each layer in these models is treated as homogeneous, with properties representing the average characteristics of that skin region. This approach allows for more straightforward mathematical modeling of light interaction within the skin while still accounting for the varying concentrations of optically significant components across different skin depths [MKS⁺19].

- **Layer 1 (stratum corneum, epidermis):** Contains eumelanin (eu), pheomelanin (ph), oxygenated hemoglobin (HbO₂), and deoxygenated hemoglobin (Hb). This layer also includes blood components due to its proximity to the next layer.
- **Layer 2 (papillary dermis):** Comprises oxygenated hemoglobin (HbO₂), deoxygenated hemoglobin (Hb), and collagen structures (contributing to base scattering).
- **Layer 3 (reticular dermis):** Similar to Layer 2, containing oxygenated hemoglobin (HbO₂), deoxygenated hemoglobin (Hb), and collagen structures (base).
- **Layer 4 (deep dermis, partial subcutis):** Includes oxygenated hemoglobin (HbO₂), deoxygenated hemoglobin (Hb), water (wat), beta-carotene (car), bilirubin (bil), collagen structures (base), and connective tissue.
- **Layer 5 (subcutis):** Contains oxygenated hemoglobin (HbO₂), deoxygenated hemoglobin (Hb), water (wat), beta-carotene (car), bilirubin (bil), and connective tissue.

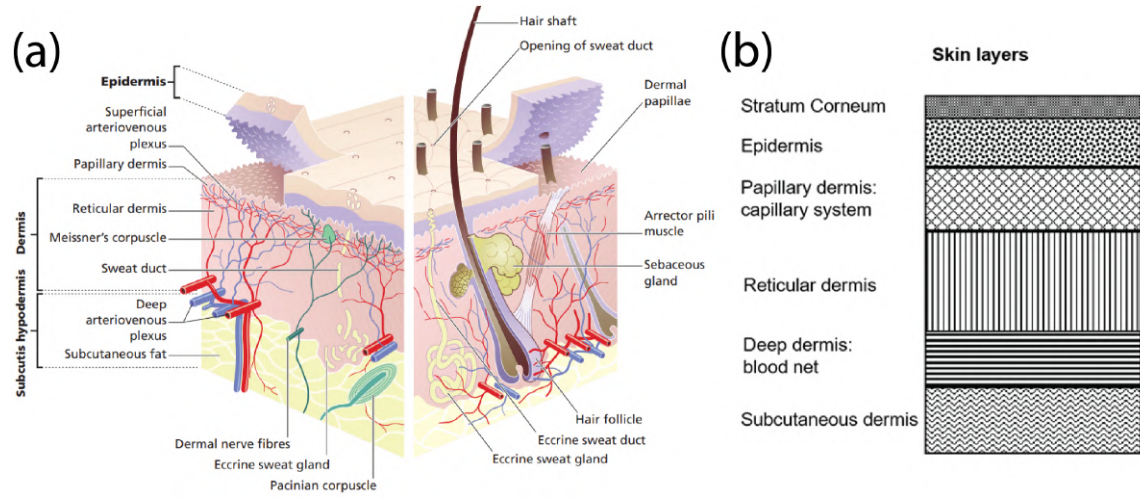


Figure 1.3: Structural representation of human skin. (a) Detailed 3D structure of human skin, illustrating various layers and components (image adapted from [Ber04]). (b) Simplified five-layer homogeneous model of skin commonly used in biomedical optics and physically based rendering, showing the approximation of complex skin structures into distinct optical layers (image adapted from [MKS⁺19]).

In this work, we employ a two-layer homogeneous model for the epidermis and dermis that is practical for modeling the relationship between reflectance and opto-structural parameters of the skin [DJ06].

1.4.2 Relationship Between Skin Structure and Appearance

The appearance of skin is driven by subsurface scattering and absorption of light by different chromophores, each with distinct optical properties [GGD⁺20]. Melanin, produced by melanocytes in the epidermis, is primarily responsible for skin pigmentation. It comes in two main types: eumelanin (brown/black) and pheomelanin (red/yellow), with higher concentrations resulting in darker skin tones. Hemoglobin, present in the blood vessels within the dermis, contributes to the skin's red coloration, especially in areas with high blood flow. The oxygenation state of hemoglobin affects the skin's hue, with oxygenated hemoglobin appearing redder and deoxygenated hemoglobin giving a blue or purple tint. Collagen, found in the dermis, provides structural support and affects light scattering, influencing the skin's texture and tone [KB04].

The combined effects and relative concentrations of these components, in addition to the thickness of each layer, determine the overall appearance of the skin. With increased epidermal thickness leading to more pronounced effects of melanin (pigmentation), while decreased epidermal thickness, associated with aging, results in paler skin with greater visibility of blood vessels (hemoglobin) in the dermis. While dermal thickness is propor-

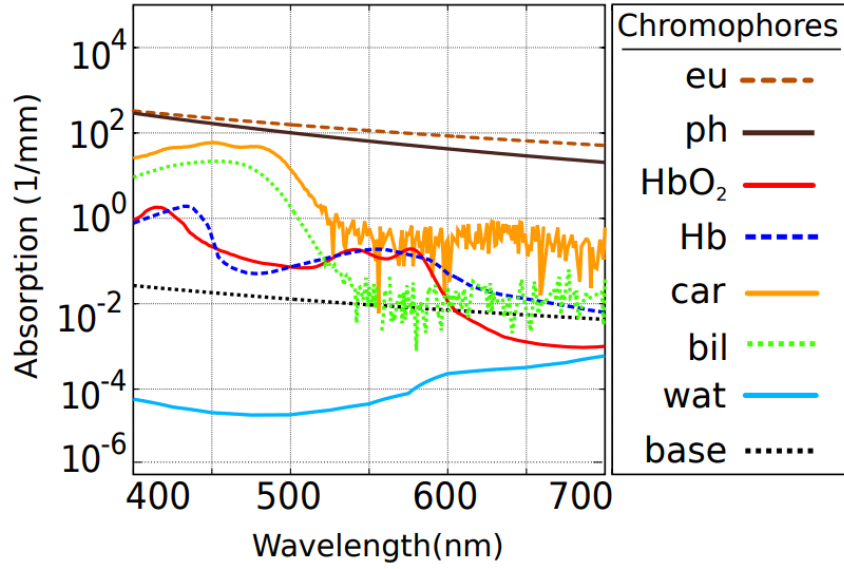


Figure 1.4: Spectral absorption profiles for key chromophores in human skin (image adapted from [IGAJG15]).

tional to the amount of diffusion of light and the effects of hemoglobin. The levels of both melanin and hemoglobin change with age, and aging typically leads to a thinner epidermis, decreases in collagen, and variance in melanin and hemoglobin [IGAJG15]. Dynamic changes in skin appearance can occur due to variations in blood flow, such as blushing, or environmental factors like UV exposure, which can alter melanin production leading to tanning or sunburn. This complex interaction of biological components and their optical properties creates the diverse range of skin appearances observed across different individuals and under varying conditions.

1.4.3 Rendering Human Skin

Capturing the effects of subsurface scattering is essential for realistic rendering of human skin, creating a soft, translucent appearance [DJ06].

BSSRDFs vs BRDFs: Bidirectional Surface Scattering Reflectance Distribution Functions (BSSRDFs) extend the concept of BRDFs to include subsurface scattering (see Figure 1.2). These functions model materials where light penetrates the surface, scatters internally, and exits at a different point. BSSRDFs are essential for rendering materials like skin, wax, and marble, where subsurface scattering plays a significant role in their appearance. BSSRDFs exhibit higher complexity than BRDFs, as they account for the paths light takes inside materials and their effect on outgoing light[Chr15].

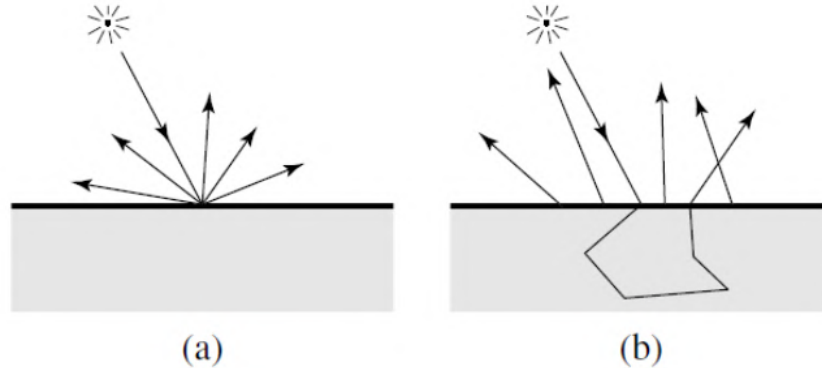


Figure 1.5: (a) BRDFs represent the relationship between incoming and outgoing light that hits a surface, with outgoing rays originating at the same point as the incoming rays. (b) BSSRDFs model the effect of subsurface scattering by accounting for light that is scattered inside a material, in which outgoing rays can exit the material at an arbitrary location (image adapted from “A Practical Model for Subsurface Light Transport” [JMLH01]).

Light hitting human skin is scattered and absorbed in multiple layers before exiting at different points. BSSRDFs represent an approximation of these phenomena, creating a soft and realistic look. Using BSSRDFs allows us to render subtle effects, like the warm glow seen in areas where light passes through the ears or fingers.

Thus, while BRDFs are suitable for simulating light reflection on opaque, non-translucent surfaces, BSSRDFs are indispensable for realistically rendering materials that exhibit subsurface scattering, a key aspect in the lifelike portrayal of human skin [JMLH01].

Approximating the BSSRDF in Real-Time

Due to the computational complexity of BSSRDF models, we require simplifications similar to those used at the level of scene rendering. Diffusion profiles are often used to approximate the spread of light within translucent materials, using precomputed radial reflectance profiles to determine the amount of diffusion based on incident light and surface albedo. By representing subsurface scattering with a set of diffusion profiles that can be precomputed for different skin types, we can avoid the computationally expensive calculation of the radiative transport equation [JMLH01].

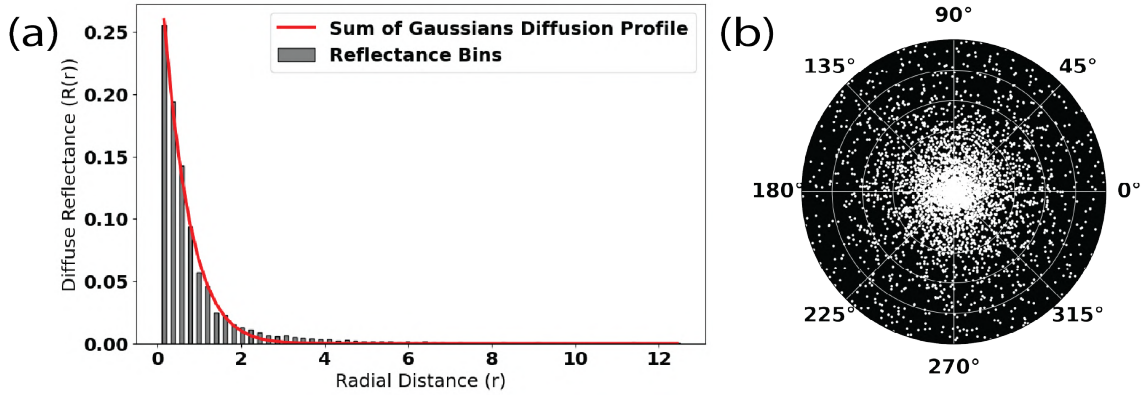


Figure 1.6: (a) Histogram of reflectance (R) across different radial distances (r), depicting a Sum of Gaussians Diffusion Profile alongside Reflectance Bins. (b) Radial distribution of light that has escaped, scattered, and absorbed in human skin, visualized through a Monte Carlo simulation to model the random walk of photons [JW95].

Jensen et al. introduced the dipole diffusion model, which converts an incident ray to a dipole, to estimate the BSSRDF by modeling subsurface scattering using a single scattering term and a dipole approximation for multi-scattering effects [JMLH01]. They later extended this to multipole approximations to handle thin translucent slabs and multi-layered materials [DJ06]. Recent works have approximated diffusion profiles with sum of Gaussian techniques, using convolutions to combine multiple layers with variable scattering, absorption, and refractive indices [JZJ⁺15].

For thin areas such as the ears, incident light is often scattered, absorbed, and transmitted out the side facing away from the light source (shadow side). Translucent shadow maps extend traditional shadow maps to account for diffusion through thin regions by giving shadow areas access to depth, irradiance, and normal information for other parts of the object [DS03]. This technique allows us to render the photorealistic effects you might expect by shining a flashlight through your fingers.

Spatially Varying Properties

Textures are critical for defining the appearance of skin, capturing the spatially varying properties that are essential for realism. These textures, combined with advanced shading techniques, allow for the accurate representation of complex skin features, such as:

- **Freckles and Moles:** Localized variations in melanin concentration.
- **Veins and Blood Vessels:** Visible under the skin due to hemoglobin.
- **Wrinkles and Pores:** Surface irregularities that affect light scattering and reflection.

Texture Mapping: Texture mapping utilizes high-resolution 2D maps to capture the spatially varying properties of real-world materials such as human skin. These include albedo maps, which represent the base color of the skin; normal maps, which capture fine details and surface irregularities; spectral maps, which indicate the reflectivity and glossiness of different areas; and subsurface scattering maps, which represent the extent of light scattering within the skin [Fol96].

Using high-resolution scans of real human faces, we can capture biophysically accurate color variation, allowing us to describe each pixel using a layered homogeneous model to analyze and augment diffuse images of human skin. With our aim of real-time modification of diffuse albedo textures of human skin, we next cover common machine learning models used for image manipulation and data compression.

1.5 Machine Learning and Rendering

Many of the techniques surrounding Physically Based Rendering (PBR) are too computationally expensive for real-time graphics pipelines. For example, the simulation of lighting used in movie productions often requires weeks or months to render a single scene. real-time graphics used in games, extended reality, and medical diagnostics don't have the luxury of time.

If we consider typical values used in multi-layer light transport simulations of skin such that 1 million photons, each undergoing up to 10,000 bounces, are simulated for 16 wavelengths, the computational cost becomes evident if we apply this to a 4k texture

$$\text{Photon Interactions} = 4096 \times 4096 \times 1,000,000 \times 10,000 \times 16 \quad (1.3)$$

$$= 2.684 \times 10^{18} \quad (1.4)$$

Each of these interactions has several non-deterministic steps to model a photon's path through tissue, preventing the use of parametric models. We must turn to pre-computation of reflectances, which can then be searched to determine the underlying tissue parameters, yet this approach is still computationally intensive

$$\text{Search Space} = 4096 \times 4096 \times 100,000 \quad (1.5)$$

$$= 1.678 \times 10^{12} \quad (1.6)$$

This is where pre-computation and machine learning become indispensable in the quest for real-time photo-realism. By utilizing high-resolution texture maps and learned representations of light transport phenomena, we can efficiently and realistically model the full complexity and range of human skin color. In the next section, we will introduce a few of the tools suited for our real-time skin texturing model.

Autoencoders: Autoencoders (AE) are a type of neural network used for learning an encoded (latent) representation of data that can be decoded with minimal loss [CG23]. In this research, we leverage supervised autoencoders (SAEs) to learn a mapping between skin color and its biophysical origins.

Dense Landmarks: Dense landmark models (DLM)s are widely used to construct 3D models from 2D images by matching key vertices between an image and a 3D model using differentiable rendering [MPS21]. To bridge the gap between 2D training data and the 3D perspective of light transport and rendering, DLMs present an optimal tool for the distribution of texture properties in a spatially coherent manner.

Transformers: Transformers are a type of neural network architecture that gained popularity in 2017, with “Attention Is All You Need” by Vaswani et al. [VSP⁺23]. They revolutionized the field of natural language processing (NLP) due to their effectiveness in handling sequences of data. Unlike previous models that processed data sequentially, Transformers use a mechanism called ‘attention’ to weigh the significance of different parts of the input data. This allows them to process entire sequences of data simultaneously, leading to significant improvements in efficiency and performance in tasks such as language translation, text generation, and sentiment analysis.

Vision Transformers: Vision Transformers (ViTs) are an adaptation of the Transformer architecture for image processing tasks. In ViTs, an image is divided into patches, which are then linearly embedded along with positional encodings to maintain spatial information. The model then uses self-attention mechanisms to process these patches collectively, allowing it to capture global dependencies within the image [DBK⁺20]. This approach has shown remarkable results in image classification tasks, challenging the dominance of convolutional neural networks (CNN)s in the field of computer vision. We leverage a pre-trained ViT as the backbone of our DLM in Chapter 4, enabling a model that generalizes well with limited computational resources.

1.6 Summary and Roadmap

The integration of advanced machine learning techniques with traditional rendering methods holds the potential to significantly enhance the realism and efficiency of real-time rendering and texturing. In the following chapters, we will utilize such methods to simulate, compress, and leverage biophysically based skin color.

- **Chapter 1: Introduction**

This chapter introduces physically based rendering and related techniques used to model photo-realistic human skin.

- **Chapter 2: Simulation of Light Transport**

In Chapter 2, we construct an optical simulation model for subsurface light transport in human skin. We use this simulation-based model to create a large tabular look-up table for skin structure and color.

- **Chapter 3: Mapping Between Skin Structure and Color**

In Chapter 3, we use supervised autoencoders to learn a bidirectional relationship between skin’s optical and structural parameters and resulting color.

- **Chapter 4: Dense Landmarks and Spatial Distribution of Chromophores**

Chapter 4 first introduces dense landmark models for the spatial coherent distribution of chromophores and the creation of diverse computer vision datasets. It then demonstrates a novel combination of SAEs and DLMs.

- **Chapter 5: Conclusion**

The final chapter summarizes the research’s key findings, discusses the implications for real-time rendering and computer vision, and suggests potential directions for future work in realistic digital human rendering.

Chapter 2

Simulation of Light Transport

Human skin appearance remains a challenging task in computer graphics due to its complex, layered, and highly scattering optical properties. Due to this complexity, we require brute-force simulation methods to model the relationship between skin structure and color accurately. We build on previous works that leverage Monte Carlo (MC) simulation techniques from particle physics to simulate diffuse reflectance for different permutations of input parameters of a two-layer homogeneous skin model [DJ06], as seen in Figure 2.1. The epidermal layer is parameterized by melanin volume fraction (C_m), the ratio of eumelanin to pheomelanin (B_m), and thickness (T). The dermis is parameterized by hemoglobin volume fraction (C_h), and ratio of oxy-hemoglobin to deoxy-hemoglobin (B_h).

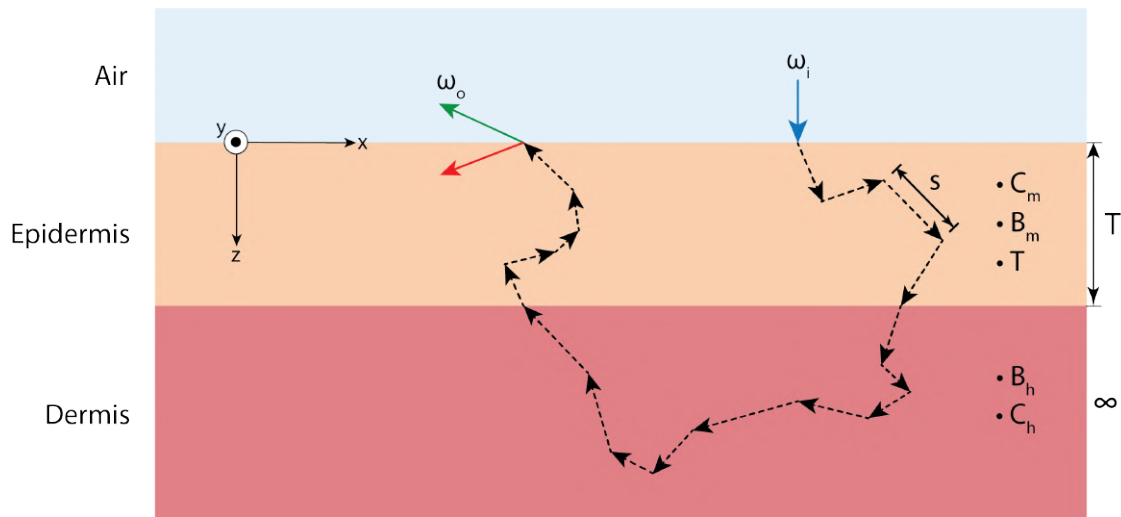


Figure 2.1: Cross-sectional view of a Monte Carlo simulation of photon transport in human skin, modeled with two layers: an epidermis of thickness T and a semi-infinite dermis. The blue vector shows the initial photon trajectory, while dashed lines trace its path at a specific wavelength, with direction and step size determined probabilistically. Green and red vectors depict escaped, and internally reflected photons, respectively.

In the next sections, we derive absorption and scattering equations in terms of key opto-structural parameters for human skin. We then use these to construct a simulation for diffuse skin reflectance. We simulate an exhaustive set of possible parameter permu-

tations based on medical literature for wavelengths across the visible spectrum. Next, we convert the resulting reflectance distributions to the RGB color space. We iteratively repeat this process, evaluating the resulting color distributions, occurrence of metamers (see Section 2.3.4), and ability to represent diffuse albedo textures across the Fitzpatrick scale.

2.1 Monte Carlo Methods

Due to the variable, and stochastic nature of light scattering in a turbid medium—such as human skin, Monte Carlo (MC) random sampling (MCRS) is an ideal way to generate realistic photon paths. MCRS allows us to model light transport into and out of the skin accurately, albeit with high computational complexity. Over many thousands of iterations, photons are launched with a starting weight of one and scattered through multi-layer tissue, losing weight and changing paths at each interaction event. Where each interaction event uses MC sampling of probability density functions, random variables, and Russian roulette to determine step length (s), angle of deflection, and weight lost by the photon. A photon is terminated when its remaining weight falls below a threshold or exits the top layer into the ambient medium (air). The average reflectance of photons at each wavelength sampled in the visible spectrum gives the spectral reflectance distribution for each desired permutation of tissue absorption (μ_a), scattering (μ_s), and thickness (T). For an in-depth review of Monte Carlo methods used in this section, we refer the reader to [JW95].

Photons are launched into the skin at normal incidence and scattered through the layers of tissue using stochastic methods to determine step size and angle of deflection at boundaries. Each layer in the simulation is parameterized by absorption and scattering coefficients μ_a and μ_s , respectively, while only the top layer is parameterized by thickness. The dermis is treated as semi-infinitely thick as negligible amounts of light reach subcutaneous fatty layers [DJ06].

Coordinate systems are chosen for computational simplicity and readability. The photon's trajectory, step size, and location are monitored in Cartesian coordinates. Escaped weight is accounted for in cylindrical coordinates, with reflectance bins accumulating escaped weight. The outermost radial bins are used to store the weight of photons that have a radius greater than $R = 2.5\text{cm}$ (see Appendix (5.2) for further discussion).

The step size and deflection angle are randomly sampled from analytical expressions derived from the optical properties of the dielectric medium being studied. The coefficient of absorption, μ_a , signifies the likelihood of a photon being absorbed per unit path length. The coefficient of scattering, μ_s , indicates the likelihood of a photon scattering over each infinitesimal path length. The sum of μ_a and μ_s gives the interaction coefficient, μ_t , which is equivalent to the probability of photon interaction per unit path length

$$\mu_t = \mu_a + \mu_s \quad (2.1)$$

For variable x with probability density function $p(x)$, ζ represents the integral of $p(x)$,

which is used in Monte Carlo methods to sample a variable of interest

$$\zeta = \int_0^x p(x)dx \quad (2.2)$$

Solving ζ for step size (s), based on extinction coefficient $\mu_t(x = s)$

$$p(s) = -\frac{\ln(1 - \zeta)}{\mu_t} \quad (2.3)$$

$$s = -\frac{\ln(\zeta)}{\mu_t} \quad (2.4)$$

Solving ζ for scattering angle of deflection ($x = \cos(\theta)$)

$$p(\cos(\theta)) = \frac{1 - g^2}{2(1 + g^2 - 2g \cos(\theta))} \quad (2.5)$$

$$\cos(\theta) = \begin{cases} \frac{1}{2g}[1 + g^2 - (\frac{1-g^2}{1-g+2g\zeta})] & \text{if } g \neq 0 \\ 2\zeta - 1 & \text{if } g = 0 \end{cases} \quad (2.6)$$

Equation 2.5 is the Henyey Greenstein scattering equation [HG41] where g represents the anisotropy factor—the mean cosine of the deflection angle. The parameter 2.6 is utilized to sample the angle of deflection of the photon at each interaction event. At each boundary crossing, the refractive indices n_1 and n_2 and the angle of incidence α_i determine the resulting trajectory and reflectance; internal $R_i(\alpha_i)$ and external, $R_e(\alpha_i)$.

$$n_i \sin(\alpha_i) = n_t \sin(\alpha_t) \quad (2.7)$$

Where α_t is the transmission angle of the photon in the new layer (air). For α_i greater than the critical angle

$$R(\theta_t) = 1 \quad (2.8)$$

For α_i less than the critical angle, Fresnel's equations give the average internal reflectance:

$$R(\alpha_i) = \frac{1}{2} \left[\frac{\sin^2(\alpha_i - \alpha_t)}{\sin^2(\alpha_i + \alpha_t)} + \frac{\tan^2(\alpha_i - \alpha_t)}{\tan^2(\alpha_i + \alpha_t)} \right] \quad (2.9)$$

Let $W_{remaining}$ be the remaining photon weight. Using the above equation for $R(\alpha_i)$, the escaped weight for a given boundary interaction is

$$W_{escaped} = W_{remaining} * (1 - R(\alpha_i)) \quad (2.10)$$

When the remaining photon weight $W_{remaining}$ is less than threshold W_{th} , it is terminated conditionally using the Russian Roulette method

$$W = \begin{cases} mW & \text{if } \zeta \leq 1/m \\ 0 & \text{if } \zeta > 1/m \end{cases} \quad (2.11)$$

Where $m = 10$ and ζ is a uniform random number $0 \leq \zeta \leq 1$. We save compute by avoiding the simulation of 9 out of 10 photons with negligible weight, and account for this lost weight by the 10 fold increase in 1 out of 10 photons. This method reduces computation while still accurately accounting for energy conservation [AK90].

The above methods are invoked for each layer of skin at discrete wavelengths across the visible spectrum. Due to the stochastic nature of the simulation, 100,000 photons are simulated for each wavelength. The average of the resulting reflectance is used as the final value for each cylindrical bin used to accumulate escaped photons. Next we will address the particular values used in the simulation of light transport in human skin.

2.2 Scattering and Absorption in Human Skin

For the purpose of modeling human skin color, we derive the absorption and scattering parameters—introduced in Section 2.1—using optical properties of melanin and hemoglobin and the geometry of skin. The inputs to the simulation are melanin concentration (C_m), hemoglobin concentration (C_h), melanin blend (B_m), hemoglobin blend (B_h) and epidermal thickness (T). The output is reflectance at discrete wavelengths. The resulting spectral reflectance distribution is converted to RGB color space using the CIE 1931 color transformation system [FR98].

The epidermis layer has a variable thickness and refractive index $n = 1.4$. All absorption and scattering coefficients are in units of mm^{-1} . The absorption coefficients of the epidermis layer are defined as

$$\mu_a^{em}(\lambda) = 6.6 \times 10^{10} \times \lambda^{-3.33} \quad (2.12a)$$

$$\mu_a^{pm}(\lambda) = 2.9 \times 10^{14} \times \lambda^{-4.75} \quad (2.12b)$$

$$\mu_a^{base}(\lambda) = 0.0244 + 8.53e^{-(\lambda-154)/66.2} \quad (2.12c)$$

$$\mu_a^{epi}(\lambda) = C_m [B_m \mu_a^{em}(\lambda) + (1 - B_m) \mu_a^{pm}(\lambda)] \quad (2.12d)$$

where $\mu_a^{em}(\lambda)$, $\mu_a^{pm}(\lambda)$, and $\mu_a^{base}(\lambda)$ are absorption coefficients for eumelanin, pheomelanin and baseline, respectively [SJT95, JM91] and $\mu_a^{epi}(\lambda)$ is the overall epidermal absorption coefficient [NBH⁺08, DJ06]. The melanocyte absorption equations are derived by fitting a power law for C_m and B_m as a function of λ . The scattering coefficient of the epidermis layer is defined as:

$$\mu_s'(\lambda) = 14.74\lambda^{-0.22} + 2.2 \times 10^{11} \times \lambda^{-4} \quad (2.13)$$

where μ_s' is the scattering coefficient based on a combination of Mie and Rayleigh scattering. $\mu_s'(\lambda)$ is scaled by 0.5 for scattering in the dermis, giving the reduced dermal scattering coefficient [BGT11].

$$\mu_s'(\lambda) = 7.37\lambda^{-0.22} + 1.1 \times 10^{11} \times \lambda^{-4} \quad (2.14)$$

The absorption coefficient of the semi-infinite dermis layer is defined as

$$\mu_a^{derm}(\lambda) = C_h \left[B_h \mu_a^{oxy}(\lambda) + (1 - B_h) \mu_a^{deoxy}(\lambda) \right] + (1 - C_h) \mu_a^{base} \quad (2.15)$$

where μ_a^{oxy} and μ_a^{deoxy} are the absorption coefficients of oxygenated (oxy) and deoxygenated (deoxy) hemoglobin, respectively.

The ranges of sample values for C_m , C_h , B_m , B_h and T parameters are based on previous works and ranges common in medical literature [DJ06, BGT11, NMM⁺19]. To best account for non-uniform impact on color and reduce metamers, the sampling frequency of each parameter is chosen proportional to its correlation with R, G, and B color channels. We demonstrate these correlations in Table 2.2 in Section 2.4.2.

Parameter	Low	High	Samples
C_m	0.001	0.5	13
C_h	0.001	0.32	9
B_m	0.000	1.0	5
B_h	0.000	1.0	5
T	0.01	0.25	7

Table 2.1: Biophysical parameter range and frequency of sampling used in the MC light transport simulation. The sampling frequency is selected approximately proportional to each parameter’s sum correlation with R, G, and B color channels.

The simulation outputs a spectral reflectance distribution for each set of parameters. This must be converted to a perceptible color space in order to be useful for our purposes. In the next section, we will cover the conversion methods used to transform the spectral reflectance distribution into the CIE XYZ and RGB color spaces.

2.3 Color Spaces

Colors are an integral aspect of human perception and experience, enabling us to differentiate objects, understand our surroundings, and communicate emotions and ideas. However, in the realm of digital technology, representing colors as we perceive them requires a standardized and numeric representation. A color space provides a systematic and consistent method to describe, categorize, and represent colors using numerical values [FL13]. Typically represented in a three-dimensional coordinate system, the axes of a color space correspond to primary colors, which can be mixed in various proportions to reproduce a wide gamut of colors.

The choice of a color space is not trivial. Depending on the domain—whether it’s digital graphics, film production, medical imaging, or scientific research—the selection of

color space can profoundly impact how color information is interpreted, processed, and rendered [SWD05]. Misinterpretations can lead to color shifts, loss of detail, and other unintended consequences.

The Commission Internationale de l'Éclairage (CIE) introduced the XYZ color space in 1931. It's a result of rigorous human vision experiments and is designed to link the electromagnetic spectrum to perceived color based on experimental data [SG31]. We align our analysis with the D65 illuminant, a standard representation of daylight with a correlated color temperature of approximately, 6500K. Additionally, we make use of the 2° field of view (FOV) standard observer and the CIE 1931 XYZ color space as a bridge between HSI data, spectral reflectance distributions and the RGB/sRGB color spaces ubiquitous in modern displays [RKOAJ08].

In the following sections, we will cover the process of transforming spectral reflectance distributions into RGB, touching on each intermediate color space and its origin.

2.3.1 CIE XYZ Color Space

The CIE XYZ color space, serves as a bridge between the physiological responses of the human eye and the physical properties of light. Foundational to this work as the CIE XYZ space is easily converted to and from the RGB color space. This space uses three tristimulus values: X , Y , and Z [RKOAJ08]:

- X : A mix of response curves from luminance (Y) and the long-wavelength sensitive cones, often associated with reddish colors.
- Y : Luminance or brightness. It peaks at the green wavelengths, corresponding to the peak sensitivity of the human eye.
- Z : A mix of cone response curves from the short-wavelength sensitive cones, often associated with bluish colors.

While it is common to rely on tabular versions of the color matching functions, this limits the chosen wavelengths to those stored in the table. In order to facilitate the simulation of any wavelength, we have utilized analytical approximations of the XYZ color-matching functions to convert the spectral reflectance distributions into the XYZ tristimulus values. The analytical approximations of the 1931 CIE XYZ color matching functions are defined

as [WSS13]

$$\begin{aligned} x(\lambda) = & 0.362e^{-0.5[(\lambda-442.0) \times u(\lambda, 442.0)]^2} \\ & + 1.056e^{-0.5[(\lambda-599.8) \times u(\lambda, 599.8)]^2} \\ & - 0.065e^{-0.5[(\lambda-501.1) \times u(\lambda, 501.1)]^2} \end{aligned} \quad (2.16)$$

$$\begin{aligned} y(\lambda) = & 0.821e^{-0.5[(\lambda-568.8) \times u(\lambda, 568.8)]^2} \\ & + 0.286e^{-0.5[(\lambda-530.9) \times u(\lambda, 530.9)]^2} \end{aligned} \quad (2.17)$$

$$\begin{aligned} z(\lambda) = & 1.217e^{-0.5[(\lambda-437.0) \times u(\lambda, 437.0)]^2} \\ & + 0.681e^{-0.5[(\lambda-459.0) \times u(\lambda, 459.0)]^2} \end{aligned} \quad (2.18)$$

where u represents the piece-wise nature of the multi lobe Gaussian fit functions [WSS13]

$$u(\lambda) = \begin{cases} 0.0624, & \text{if } \lambda < 442.0 \\ 0.0374, & \text{if } \lambda \geq 442.0 \end{cases}$$

To obtain the X, Y and Z values from spectral reflectance data, we integrate the product of the reflectance $S(\lambda)$, the illuminant $I(\lambda)$ and the fit function for each $x(\lambda), y(\lambda), z(\lambda)$ [RKOAJ08]. For the continuous case:

$$X = \frac{K}{N} \left(\int S(\lambda) \times I(\lambda) \times x(\lambda) d\lambda \right) \quad (2.19)$$

$$Y = \frac{K}{N} \left(\int S(\lambda) \times I(\lambda) \times y(\lambda) d\lambda \right) \quad (2.20)$$

$$Z = \frac{K}{N} \left(\int S(\lambda) \times I(\lambda) \times z(\lambda) d\lambda \right) \quad (2.21)$$

For the discrete case:

$$X = \frac{K}{N_d} \left(\sum_{\lambda} \Delta\lambda \times S(\lambda) \times I(\lambda) \times x(\lambda) \right) \quad (2.22)$$

$$Y = \frac{K}{N_d} \left(\sum_{\lambda} \Delta\lambda \times S(\lambda) \times I(\lambda) \times y(\lambda) \right) \quad (2.23)$$

$$Z = \frac{K}{N_d} \left(\sum_{\lambda} \Delta\lambda \times S(\lambda) \times I(\lambda) \times z(\lambda) \right) \quad (2.24)$$

where K/N is a normalization factor with $K = 1$ or $K = 100$:

$$N = \int y(\lambda) \times I(\lambda) d\lambda \quad (\text{for the continuous case}) \quad (2.25)$$

$$N_d = \sum_{\lambda} \Delta\lambda \times y(\lambda) \times I(\lambda) \quad (\text{for the discrete case}) \quad (2.26)$$

Since the simulation weights begin at 1 for each wavelength, we assume an even energy illuminant, $I(\lambda) = 1$.

$$X = \frac{K}{N_d} \left(\sum_{\lambda} \Delta\lambda \times S(\lambda) \times x(\lambda) \right) \quad (2.27)$$

$$Y = \frac{K}{N_d} \left(\sum_{\lambda} \Delta\lambda \times S(\lambda) \times y(\lambda) \right) \quad (2.28)$$

$$Z = \frac{K}{N_d} \left(\sum_{\lambda} \Delta\lambda \times S(\lambda) \times z(\lambda) \right) \quad (2.29)$$

Where $\Delta\lambda$ is the difference between consecutive wavelength values (step size). The resulting X, Y and Z values are converted from even energy illuminant to D65 using the Bradford transformation matrix(M) [RKOAJ08].

$$\begin{bmatrix} X_D \\ Y_D \\ Z_D \end{bmatrix} = M \begin{bmatrix} X_S \\ Y_S \\ Z_S \end{bmatrix} \quad (2.30)$$

$$M = \begin{bmatrix} 0.9531874 & -0.0265906 & 0.0238731 \\ -0.0382467 & 1.0288406 & 0.0094060 \\ 0.0026068 & -0.0030332 & 1.0892565 \end{bmatrix} \quad (2.31)$$

As an alternative to this step, the simulation can be run using starting weights equivalent to the D65 illuminant, in which case the calculated X, Y, Z values can be directly converted to linear RGB.

2.3.2 Linear RGB color space

The D65 illuminant represents average daylight with the 2° observer from the 1931 CIE standard. Where M is the transformation matrix for this standard and illuminant, we multiply M with XYZ values to convert them to RGB [RKOAJ08]

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = [M] \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \quad (2.32)$$

$$M = \begin{bmatrix} 3.2404542 & -1.5371385 & -0.4985314 \\ -0.9692660 & 1.8760108 & 0.0415560 \\ 0.0556434 & -0.2040259 & 1.0572252 \end{bmatrix} \quad (2.33)$$

The result is a linear RGB that requires a final step to reach the widely accepted sRGB standard.

2.3.3 sRGB Color Space

The sRGB color space is established for standard display devices. The conversion from linear RGB values to sRGB involves a gamma correction. Where C represents an arbitrary channel of a linear RGB pixel. The gamma correction is defined as [RKOAJ08]:

$$\gamma(C) = \begin{cases} 12.92 * C & \text{if } |C| \leq 0.0031308 \\ 1.055 * |C|^{1/2.2} - 0.055 & \text{otherwise} \end{cases} \quad (2.34)$$

Due to the non-linearity of the sRGB color space, the convention is always to apply this correction as the last step in the rendering pipeline. Diffuse albedo textures we work with are primarily in the linear RGB space. Most importantly, linear RGB values are easier to search using nearest neighbours and more effective training data for subsequent chapters.

2.3.4 Metamers

Metamers are different color stimuli that appear identical despite having varied spectral power distributions [Pal99]. This simple phenomenon presents one of the biggest challenges in mapping between biophysical parameters such as melanin and hemoglobin and the resulting RGB skin color. The step where we convert from reflectance distribution to the CIE XYZ color space is many-to-one, which carries over to subsequent RGB and sRGB color spaces. The result is a tabular dataset with a many-to-one relation between skin parameters and color. For simplicity, we will refer to the many-to-one relationship between RGB skin color and biophysical parameters as metamers for the remainder of the text.

2.4 Results and Discussion

The following results were achieved on a workstation equipped with a 13th Gen Intel(R) Core(TM) i9-13900K CPU at 3.00 GHz and 64 GB of RAM and GeForce RTX 4090 with 24 GB GDDR6X VRAM. The simulations were run on the CPU using C++ multiprocessing, while the subsequent LUT search and image editing tasks were largely run on the GPU using python and CUDA.

2.4.1 Convergence of the Simulation

Monte Carlo methods require many iterations to reach convergence. As a result there is a trade off between the time and compute required and the quality of the reflectance and color values obtained in our LUT.

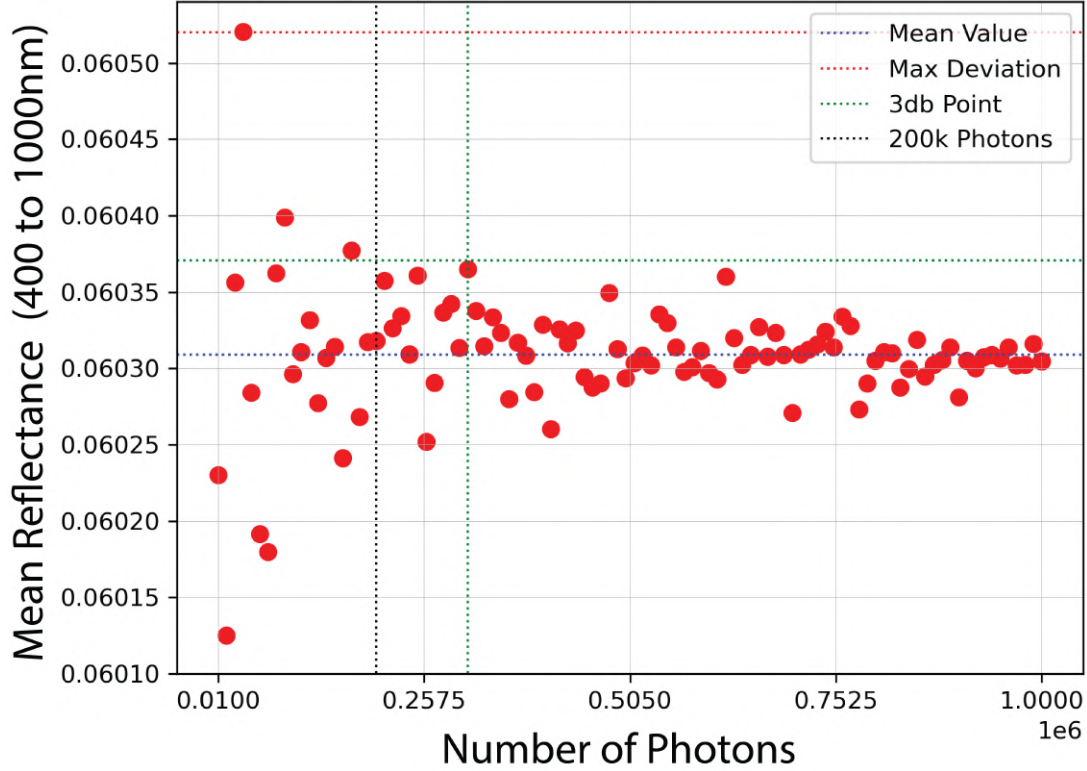


Figure 2.2: Number of photons simulated per wavelength vs. convergence of average reflectance.

For speed of development and debugging of the simulation, the number of photons can be reduced. This allows for much easier adjustment of key factors such as parameter sampling frequency and range. Once the simulation was finalized, we increased to 200k photons for higher accuracy. With greater computational resources a number greater than 260,000 would be ideal as seen by the 3dB point in Figure 2.2.

2.4.2 Reflectance Distributions, Color and Parameters

The first thing that stands out upon plotting the reflectance distributions (Figure 2.3) is the repetition of RGB colors and the clustering of many similar RGB values despite having a wide variety of biophysical parameters.

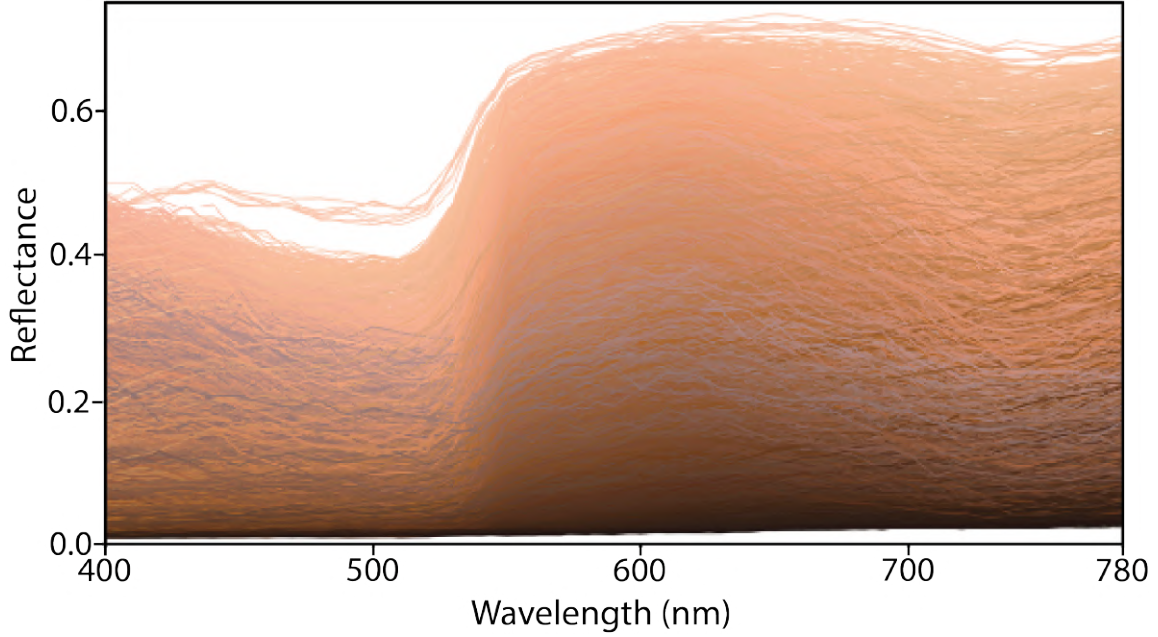


Figure 2.3: Reflectance distributions for the various rows of the dataset, with the colors corresponding to the derived RGB color from the LUT.

The repetition of RGB color as seen in Figure 2.3 is a result of metamers and the disproportionate impact of each of the 5 skin parameters on reflectance and color. Since the former is unavoidable, we focus on the sampling and distribution of parameters fed into the simulation. Where the cumulative correlation with R , G , and B channels for each parameter is found using the the Pearson correlation coefficient in Equation 2.4.2 below, where $X \in \{C_m, C_h, B_m, B_h, T\}$ and $Y \in \{R, G, B\}$

$$\text{Corr}(X, Y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}} \quad (2.35)$$

This cumulative correlation provides a metric that quantifies the overall relationship between each biophysical parameter and the RGB color space, combining the individual correlations with each color channel. It is worth noting that this method works best on linear correlations, so it is important that cubic and quartic transforms are applied to C_m and C_h before doing this analysis.

It's clear from the last column of Figure 2.2, if we rank the effect of each parameter on color in descending order as C_m, C_h, B_m, T and B_h . This tells us that we can expect large color changes from variations in C_m and C_h and much more subtle changes from the other parameters. This can also be used to guide the sampling frequency of the parameters in such a way that we adequately space parameters with low impact on color. For example,

Parameter	R	G	B	Sum Correlation
C_m	-0.7514	-0.7754	-0.7887	-2.3156
C_h	-0.4447	-0.3524	-0.1783	-0.9753
B_m	-0.0999	-0.0747	-0.0243	-0.1990
T	-0.1648	-0.0476	0.0975	-0.1149
B_h	-0.0001	0.0000	0.0001	-0.0001

Table 2.2: Correlation of biophysical parameters with RGB channels.

if we fix C_m, C_h, B_m and T and vary only B_h in small enough increments, we can expect to generate many RGB values that are the same. We use this correlation with RGB color to determine the sampling frequency of our input parameters as seen in Table 2.1.

2.4.3 Top Layer Saturation

Once the simulation has been run and the reflectance distributions are in the RGB color space, these changes are saved to a large comma separated look-up-table (LUT). The table has columns for $C_m, C_h, B_m, B_h, T, R, G, B$ with each row representing a unique combination of values for C_m, C_h, B_m, B_h, T . Fixing all but the C_m and C_h parameters, we can visualize the RGB color as follows:

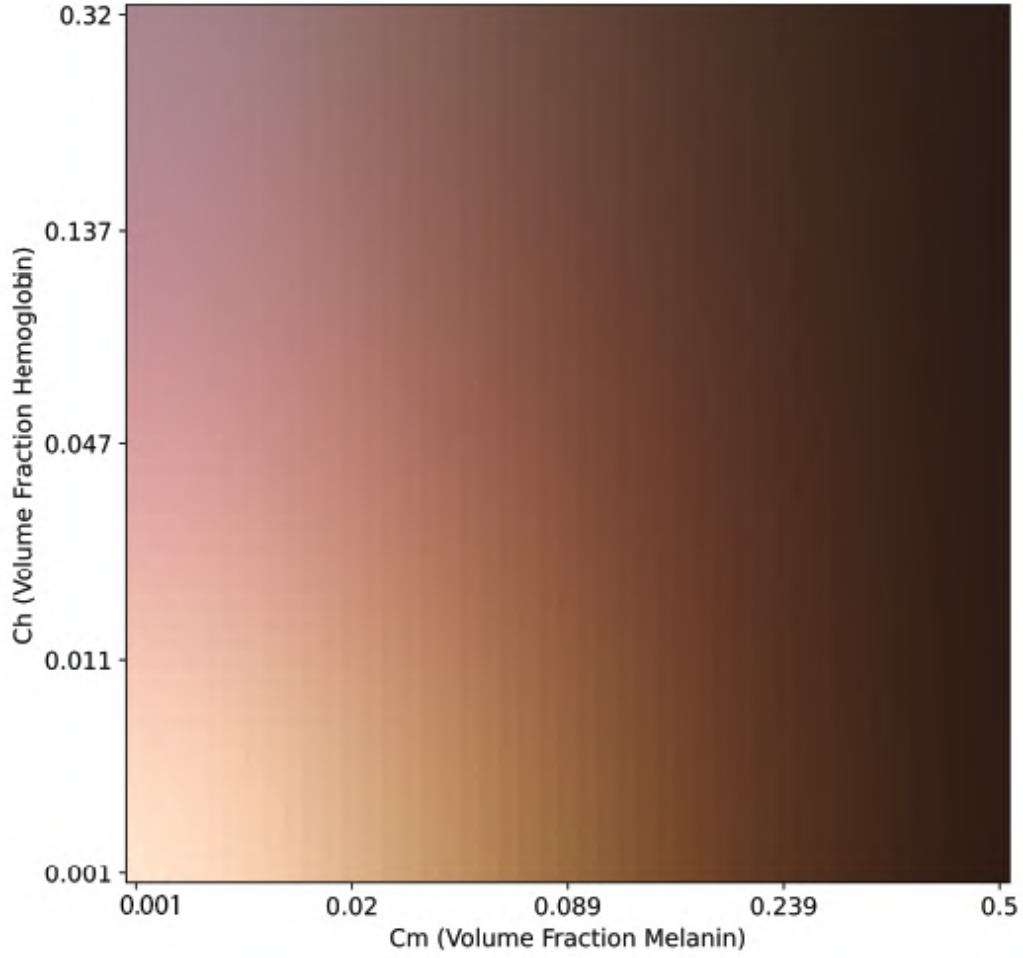


Figure 2.4: Simulation output of 45 unique C_m and C_h values. With C_m from 0.001 to 0.5, C_h from 0.001 to 0.32, and B_m , B_h , and T fixed at 0.5, 0.75 and 0.25 respectively.

One limitation of this model for skin color is that with higher C_m values, it becomes increasingly difficult to evaluate changes in the other parameters. Looking at the max C_m values in Figure 2.4 the color becomes nearly uniform regardless of C_h . This makes sense if we think of the high level concept of absorption and scattering. As more and more light is absorbed in the epidermis, we can expect less of an impact from adjustment of C_h and B_h which are the main drivers of color in the dermis.

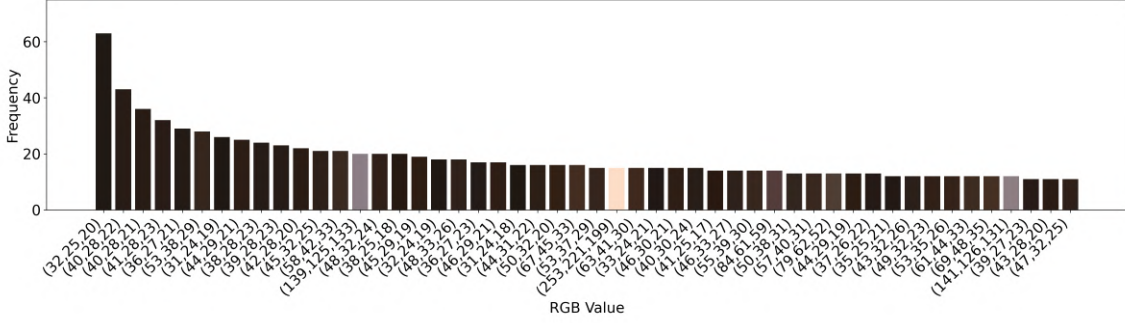


Figure 2.5: Histogram of common metamers in the LUT.

These limitations are an unavoidable consequence of multi-layer volumetric scattering. However, we can mitigate them through thoughtful selection of parameter combinations. The more we can limit metamers, the easier it is to map between skin parameters and color. We must ultimately strike a balance between having enough data to train a learned representation and avoiding the metamers that come with increased sampling.

2.5 Conclusion

We demonstrate a data-driven methodology for simulating light scattering and absorption in multi-layer turbid mediums such as human skin. Using Monte Carlo methods, we model diffuse reflectance as a non-parametric function of absorption and scattering parameters, converting reflectance distributions to RGB color to create a tabular dataset encompassing the majority of biologically plausible skin colors. Finally, we analyze the resulting LUT and provide useful heuristics to improve it for use as both an LUT and a training set to be used in subsequent chapters.

In analyzing the LUT, we identify the challenges posed by metamers—different parameter combinations that produce similar colors. To address this, we propose a correlation-based sampling strategy that emphasizes parameters with a more significant impact on color variation. This ensures a more efficient and representative sampling of the parameter space, reducing redundancy and enhancing the quality of the data for subsequent use. By leveraging our simulation results, we can improve the realism and inclusivity of digital humans and datasets with human skin.

Our Contributions

- The use of analytical functions for XYZ color matching to enable flexibility in the choice of simulated wavelengths.

- Creation of heuristics for iterative improvement of the simulation data to reduce metamers and allow larger, more descriptive skin datasets.
- Leveraging correlation between R, G, and B color channels to select the sampling frequency of the input parameters.
- Integration with training supervised autoencoders (see Chapter 3) for iterative improvement of the data and resulting learned representation.

Chapter 3

Mapping Between Skin Structure and Color

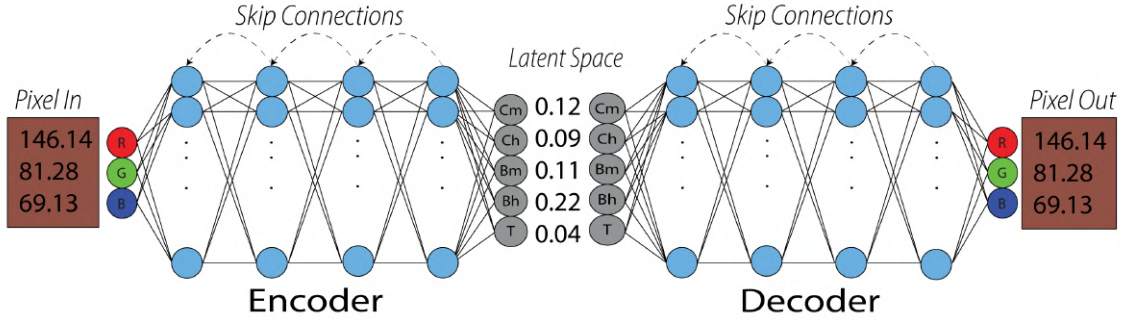


Figure 3.1: Pixel-wise RGB to biophysical parameter supervised-autoencoder (SAE) flow chart with example input, latent and output values.

The mapping between skin structure and color is inherently stochastic and non-linear, requiring light transport simulations which are pre-computed and stored in a tabular look-up table (LUT). This LUT-based model allows us to search using a pixel-wise nearest neighbor approach, to generate biophysical parameter maps. LUT-based mapping however, suffers from quantization and slow performance [AHX22].

We employ a supervised-autoencoder (SAE)-based approach to learn this mapping from the LUT and compare the speed and accuracy of the models [AHX22]. The main utility of the LUT-based approach is in our ability to evaluate how well the dataset represents the range of pixel values in diffuse albedo textures. In the next sections, we outline the architecture used and introduce heuristics to evaluate and iteratively improve the SAE and the data on which it is trained.

3.1 Methodology

3.1.1 LUT Search Based Mapping

The tabular dataset (LUT) simulated in Chapter 2 serves as a non-parametric representation of light transport in human skin. Using this LUT we can map the pixels in a diffuse

albedo texture to maps for the 5 biophysical parameters of interest C_m , C_h , B_m , B_h and T .

Quantization: Because the LUT is limited to discrete values, we require a sampling density high enough to be representative, yet not so high as to produce an excess of metamers. Thus, the range and sampling of parameter values are key considerations. Because of the non-linear relationship between these input parameters and resulting color, we use cubic and quartic sampling of C_m and C_h as in previous works in order to approach a more uniform distribution of colors in our LUT [AHX22]. This allows more consistent results across the full range of the LUT.

3.1.2 Autoencoders

Autoencoders (AE) represent a broad class of unsupervised machine learning methods used for compression, dimensionality reduction, and feature extraction [GBC16]. Composed of an encoder and decoder, these stages work together: the encoder constructs a latent representation of the input, while the decoder aims to reconstruct the original data from the latent space. Our work utilizes a supervised-autoencoder (SAE) model to generate a higher-dimensionality latent space. The model described in this section is largely based on the work of Aliaga et. al [AHX22].

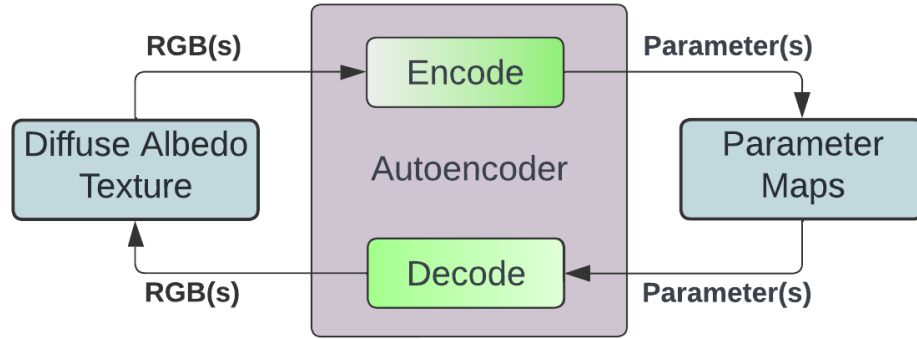


Figure 3.2: SAE high level diagram, showing the bidirectional and pixel-wise mapping between color and parameter values.

The model consists of an encoding and decoding stage, both of which are feed-forward, fully-connected neural networks. The encoder takes 3 inputs representing the RGB values of a single pixel from a diffuse albedo texture. These inputs pass through L hidden layers, each with N neurons. The encoded output—the latent space—has 5 channels corresponding to the biophysical parameters C_m , C_h , B_m , B_h , and T . This output is then fed into a matching 5-channel input layer of the decoder. The decoder mirrors the encoder structure, with L

hidden layers of N neurons, culminating in an output layer of size three to represent the recovered RGB value.

Matrix Formulation: Each layer l of the model is characterized by a weight matrix $W^{(l)}$ and a bias vector $b^{(l)}$, operating on the input vector $x^{(l-1)}$ from the previous layer to produce the linear transformation $z^{(l)}$. Each connection between neuron n of layer $l-1$ and neuron m of layer l in the network is represented by a scalar $W_{n,m}^l$. The output $z^{(l)}$ then undergoes a non-linear activation, with the ReLU function applied element-wise for hidden layers and linear activations for the final outputs of both encoder and decoder:

$$W^{(l)} = \begin{bmatrix} w_{1,1}^{(l)} & \cdots & w_{1,m}^{(l)} \\ \vdots & \ddots & \vdots \\ w_{n,1}^{(l)} & \cdots & w_{n,m}^{(l)} \end{bmatrix}, \quad z^{(l)} = W^{(l)}x^{(l-1)} + b^{(l)}, \quad a^{(l)} = \max(0, z^{(l)})$$

Graph Formulation: Due to the relative simplicity of the model, we can construct a graph based representation of the SAE. The computation graphs for the encoder and decoder are shown below, with only two hidden layers for brevity.

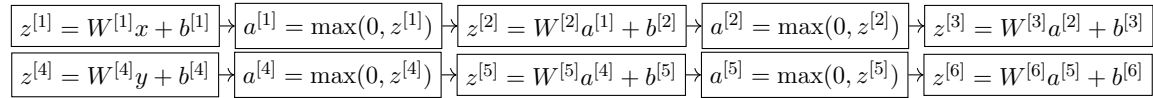


Figure 3.3: Encoder and decoder computation graphs, top and bottom respectively.

These computation graphs are easily translated into code for inference (forward-pass), allowing us low level access to each weight, bias, input and output in the network. Though it is recommended to use libraries such as PyTorch and TensorFlow for training and deployment, this low level instantiation is easily portable to different languages and platforms and often helps to optimize and debug the model. For example, to encode and decode a single pixel—using only matrix multiplications and the trained weights/biases from a TensorFlow model in Python:

```

1: function ENCODE(RGB)
2:   RGB.reshape(3,1)
3:   w1, b1 = encoder.layers[1].get_weights()
4:   w2, b2 = encoder.layers[2].get_weights()
5:   w3, b3 = encoder.layers[3].get_weights()
6:   z1 = np.matmul(w1.T, RGB) + b1
7:   a1 = np.maximum(0, z1)
8:   z2 = np.matmul(w2.T, a1) + b2
9:   a2 = np.maximum(0, z2)
10:  z3 = np.matmul(w3.T, a2) + b3
    
```

```

11:     return z3
12: end function

13: function DECODE(encoded)
14:     w4, b4 = decoder.layers[1].get_weights()
15:     w5, b5 = decoder.layers[2].get_weights()
16:     w6, b6 = decoder.layers[3].get_weights()
17:     z4 = np.matmul(w4.T, encoded) + b4
18:     a3 = np.maximum(0, z4)
19:     z5 = np.matmul(w5.T, a3) + b5
20:     a4 = np.maximum(0, z5)
21:     z6 = np.matmul(w6.T, a4) + b6
22:     return z6
23: end function

24: function AUTOENCODER(RGB)
25:     encoder_pred = Encode(RGB)
26:     latent_space = encoder_pred
27:     decoder_pred = Decode(encoder_pred)
28:     return decoder_pred
29: end function

```

Due to the millions of pixels in high resolution textures, we require fast and efficient batch processing of the above operations. To this end, we run all operations in parallel on the GPU with the largest batch sizes possible.

Pixels from an albedo texture are encoded into a matrix of parameters with dimensions $[w \cdot h, 5]$, where w and h are the image dimensions. These parameters are then decoded back into an array of RGB pixels. With large textures, the utility of GPU optimized batch processing makes the use of the aforementioned libraries essential for inference. It is the optimization of the SAE however, where parallelization is most needed.

Optimization and Training

The end-to-end supervision of the SAE is critical for training both encoder, decoder and end-to-end (SAE) weights effectively. The power of this model lies in the connection of the two stages in order to learn the mapping from RGB to biophysical parameters in a bi-directional manner. We want the latent space to be close to the ground truth, but we also deeply care about the accuracy of the decoded texture. SAE models work well for this purpose of learning a *supervised* latent space.

SAE Loss: The SAE’s loss is a composite measure consisting of encoder-loss, decoder-loss, and end-to-end loss. The encoder-loss is the L1-norm between actual and predicted

biophysical parameter values for RGB inputs. Conversely, decoder-loss uses the L2-norm between the true and predicted RGB for a given set of parameters inputs. Finally, the supervised-autoencoder loss applies the L1-norm to compare the input RGB values with the end-to-end prediction of the two stages. These loss functions collectively drive the optimization of the supervised-autoencoder, as detailed below [AHX22]:

$$\mathcal{L}_{\text{encoder}} = \|p_{\text{true}} - p_{\text{predicted}}\|_2 \quad (3.1)$$

$$\mathcal{L}_{\text{decoder}} = \|I_{\text{true}} - I_{\text{predicted}}\|_1 \quad (3.2)$$

$$\mathcal{L}_{\text{end-to-end}} = \|I_{\text{true}} - I_{\text{predicted}}\|_1 \quad (3.3)$$

The overall loss function for the SAE is a weighted sum of Equations 3.1.2, 3.1.2, and 3.1.2, where α , β , and γ are the weights controlling the importance of each loss term

$$\mathcal{L}_{\text{SAE}} = \alpha \mathcal{L}_{\text{encoder}} + \beta \mathcal{L}_{\text{decoder}} + \gamma \mathcal{L}_{\text{end-to-end}} \quad (3.4)$$

Optimization of the SAE involves computing the gradients of the loss functions with respect to the weights and biases of each layer, and updating these parameters to minimize the loss. This process is based on the chain rule of calculus to efficiently propagate errors backward through the network.

Back Propagation: The gradient of the loss function $\mathcal{L}$ with respect to any parameter θ in the Supervised Autoencoder (SAE) can be expressed using the chain rule as

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z^{[L]}} \frac{\partial z^{[L]}}{\partial a^{[L-1]}} \cdots \frac{\partial z^{[l+1]}}{\partial a^{[l]}} \frac{\partial a^{[l]}}{\partial z^{[l]}} \frac{\partial z^{[l]}}{\partial \theta} \quad (3.5)$$

where $\hat{y}$ is the network output, $z^{[l]}$ and $a^{[l]}$ are the input and output of the activation function in layer l , and L is the total number of layers [GBC16]. Below is a table summarizing the ReLU activation function and the L_1 and L_2 loss functions, along with their gradients

For a given layer l , the gradients of L with respect to the weights $W^{[l]}$ and biases $b^{[l]}$ are

$$\frac{\partial \mathcal{L}}{\partial W^{[l]}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z^{[L]}} \frac{\partial z^{[L]}}{\partial a^{[L-1]}} \cdots \frac{\partial z^{[l+1]}}{\partial a^{[l]}} \frac{\partial a^{[l]}}{\partial z^{[l]}} \frac{\partial z^{[l]}}{\partial W^{[l]}} \quad (3.6)$$

$$\frac{\partial \mathcal{L}}{\partial b^{[l]}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z^{[L]}} \frac{\partial z^{[L]}}{\partial a^{[L-1]}} \cdots \frac{\partial z^{[l+1]}}{\partial a^{[l]}} \frac{\partial a^{[l]}}{\partial z^{[l]}} \frac{\partial z^{[l]}}{\partial b^{[l]}} \quad (3.7)$$

Function	Definition	Gradient
ReLU ($z^{(l)}$)	$a^{(l)} = \max(0, z^{(l)})$	$\frac{\partial a^{(l)}}{\partial z^{(l)}} = \begin{cases} 1, & \text{if } z^{(l)} > 0 \\ 0, & \text{if } z^{(l)} \leq 0 \end{cases}$
L_1 Loss	$L_1(y, \hat{y}) = \sum_{i=1}^n y_i - \hat{y}_i $	$\frac{\partial L_1}{\partial \hat{y}_i} = -\text{sign}(y_i - \hat{y}_i)$
L_2 Loss	$L_2(y, \hat{y}) = \sum_{i=1}^n (y_i - \hat{y}_i)^2$	$\frac{\partial L_2}{\partial \hat{y}_i} = 2(y_i - \hat{y}_i)$

Table 3.1: Summary of function definitions and their gradients, where y and $\hat{y}$ are vectors representing the true and predicted values, respectively, for an arbitrary batch of data points. The ReLU activation function, as well as the L_1 and L_2 loss functions, are defined and computed element-wise for each component of these vectors. The gradients are similarly computed with respect to each predicted value $\hat{y}_i$ in the batch.

As gradients propagate recursively using the chain rule, small or large values compound, leading to vanishing and exploding gradients. As we propagate a gradient from the output of the decoder, the end-to-end loss gradient has diminishing or excessively increasing strength, especially once it reaches the first few hidden layers of the encoder. Despite only having 3 to 5 layers of neurons each, the combined effect of large width and moderate depth in an end-to-end setup results in significant issues with vanishing and exploding gradients due to the non-negligible compounding of consistently small or large gradients [GBC16].

Skip Connections: To mitigate vanishing and exploding gradients, we can introduce skip connections to more uniformly distribute the gradient-based updates of weights and biases throughout the network [GBC16]. The dashed lines, seen in Figure 3.1, illustrate the backward flow of the gradient during the training of the SAE.

Hyper-parameter Optimization: The optimization process extends beyond loss minimization to include a grid search for hyper-parameter tuning. Parameters such as the number and size of dense layers, the weightings of each loss component, batch size, and number of epochs. Due to the limitations of pixel-wise loss as a sole metric, considering the spatial relationships between pixels, a holistic evaluation is employed. The supervised-autoencoder is assessed on whole diffuse albedo textures for Fitzpatrick skin types 1 through 6, ensuring that color adjustments via latent space modifications align with high-level visual objectives.

3.1.3 Performance Metrics

We use both pixel-wise and whole image metrics that combine qualitative and quantitative approaches to fine tune both the simulation data and the SAE hyper parameters. This

process was repeated using an iterative approach to adjust the range and sampling used in the simulation of the LUT data and in converging on the optimal SAE hyper parameters.

Quantitative

Delta E: The CIE Lab color space was designed to be more perceptually uniform than RGB color, which gives it utility in comparison of color. To gauge reconstruction accuracy of the SAE we use the Delta E (dE) color difference between original and recovered textures [ORL06]. We also employ the same metric for pixel-wise LUT search which benefits from a uniform search space for nearest neighbours.

The Delta E color difference can be calculated using the formula:

$$\Delta E = \sqrt{(L_2 - L_1)^2 + (a_2 - a_1)^2 + (b_2 - b_1)^2} \quad (3.8)$$

where L , a , and b are the color coordinates in CIE Lab color space for two samples denoted as 1 and 2 [ORL06, SG31]. Table 3.2 shows ranges of ΔE values and their perceptual meanings.

ΔE	Perception
≤ 1.0	Not perceptible by human eyes.
1 - 2	Perceptible through close observation.
2 - 10	Perceptible at a glance.
10 - 50	Colors are more similar than different.
50 - 99	Colors are more different than similar.
100	Colors are the exact opposite.

Table 3.2: Perception of color differences based on ΔE values [MMC21].

Latent Space: The maps generated from an albedo texture should be close to the range of values in our LUT, making this a good way to test the encoded maps generated by our SAE. It is common to end up with negative or larger than expected values if the encoder is over or under fit. This metric is inherently tied to the weights of the 3 loss functions, presenting a subtle but important trade off between reconstruction ability and accuracy of the latent space.

Speed: We use the speed of inference to justify the SAE, though it is not used to choose between SAE models in the grid search, as we prioritize accuracy and *biophysical plausibility*.

Qualitative

Plausibility: The lips and hair (if present) are two good heuristics to gauge the accuracy of the generated maps. For example, for pixels in the lip region we expect C_h above the meanwhile T and C_m should be less than the mean. We formalize this as a qualitative test based on expected trends from the dermatology literature:

- Lips (high C_h , low T , low C_m)
- Hair (high C_m)

Metamers can be addressed using the plausibility heuristic to break ties between rows of the LUT with the same RGB color value and different biophysical parameter permutations.

Noise: We can also assess the level of noise in the generated maps, recovered and modified textures to evaluate how well the LUT and SAE represent the various skin types. As mentioned in Section 3.1.1 the LUT suffers quantization inversely proportional to sampling density. We can use noise from quantization in the LUT and SAE results to assess the *interpolation ability* of the SAE.

Modification: Finally, we rely on qualitative evaluation of uniform changes to the latent biophysical parameter maps and the resulting decoded texture. In particular, changes to C_m and C_h are expected to produce darker and more reddish skin tones respectively.

In summary, we can assess the LUT and SAE models through six key heuristics: *Delta E (dE)*, *Latent Space*, *Speed*, *Plausibility*, *Noise*, and *Modification*.

3.2 Results and Discussion

We generate a set of benchmark biophysical parameter maps using LUT search based mapping. Next, we train an SAE to represent a learned neural representation of the LUT data and validate mapping between color and biophysical parameters. Finally, we evaluate the utility of the SAE in making biophysically based color changes to diffuse albedo textures.

3.2.1 Implementation Details

Hardware and Software: The following results were achieved on a workstation equipped with a 13th Gen Intel(R) Core(TM) i9-13900K CPU at 3.00 GHz and 64 GB of RAM and GeForce RTX 4090 with 24 GB GDDR6X VRAM. All search based methods, machine learning and inference were run on the GPU using python and CUDA.

Code: Trained models are available in ONNX format, with example code for python and C++ inference: <https://github.com/jj-tech-io/Deep-Albedo>

Data Sources: For most of this work we utilized high resolution texture maps from Texturing XYZ [Tex24] (types 2-6). For the type 1 textures we used a Metahuman texture [Epi21], and 1 generated texture from the FFHQ-UV dataset [BKZ⁺23] for LUT and SAE respectively.

3.2.2 LUT Search Maps

For each texture, we search the LUT for the nearest RGB neighbors and assign the parameters from the corresponding row. Below is the result of our nearest RGB neighbor search:

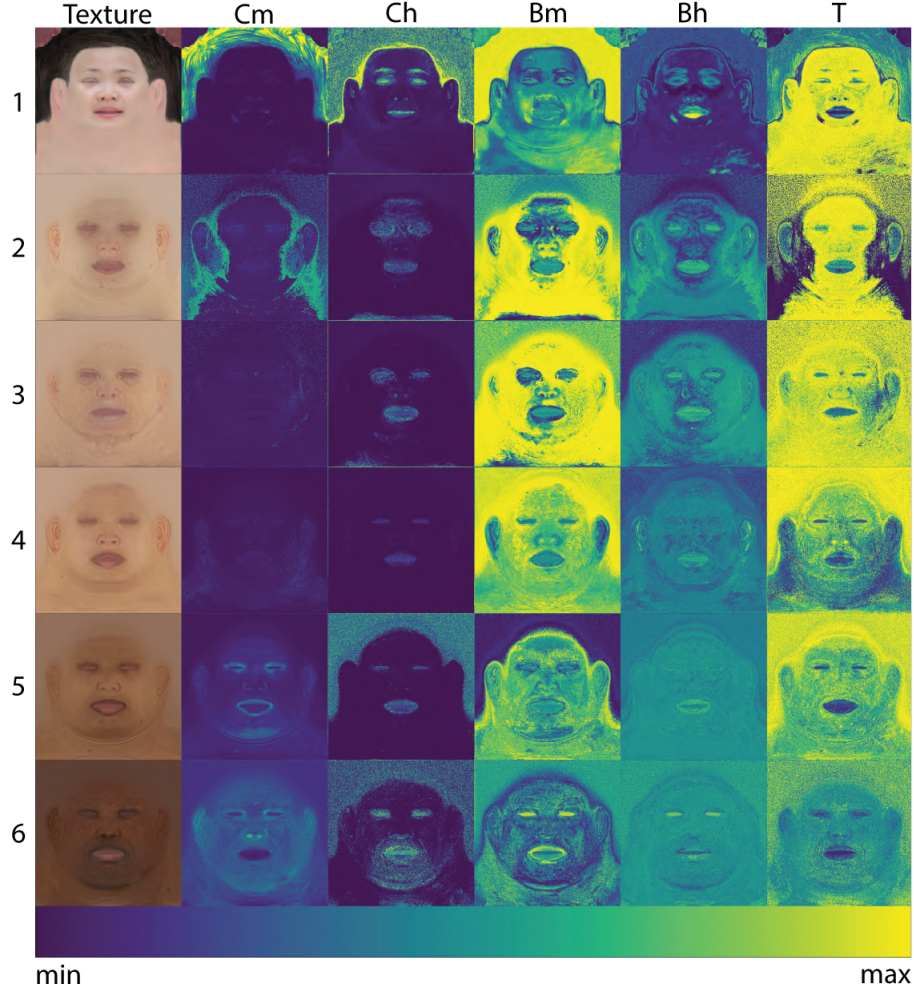


Figure 3.4: Maps generated by searching the LUT for each pixel in a given RGB texture. Top to bottom shows Fitzpatrick types 1 through 6. C_m $[0.001, 0.5]$, C_h $[0.001, 0.32]$, B_m $[0, 1]$, B_h $[0, 1]$, T $[0.01, 0.25]$ mm as in Table 2.1

This search based mapping helps to validate the simulation data without needing to train the SAE. In Figure 3.2.2 we see promising results for the *plausibility heuristic* with higher C_h , lower C_m and lower T in the lips. We also see the C_m trend higher for increasing Fitzpatrick skin types. As a result of constructing the maps using a discrete dataset, we get quantization noise inversely proportional to the density with which the parameters are spaced. Despite the slow performance and quantization, this mapping is useful in defining expected parameter ranges and distributions.

We decompose various scan-based textures across the Fitzpatrick scale in order to assess

how representative our LUT is. We can then adjust the sampling range and frequency to better represent regions of the color space that may be overlooked.

3.2.3 SAE Training

We train an SAE using grid search optimal hyperparameters as follows:

- Number of Layers: 4
- Number of Neurons: 75
- Encoder Loss Weight: 0.3
- Decoder Loss Weight: 0.3
- End to End Loss Weight: 0.6
- Batch Size: 4096
- Epochs: 200
- LUT Size: 750,000 rows (with ranges and sampling density as seen in Table 2.1)
- Train test split: 0.9/0.1

Below is the loss per epoch with a train test split of 0.9 and 0.1 respectively. The time to train is directly proportional to the size of the LUT and batch size.

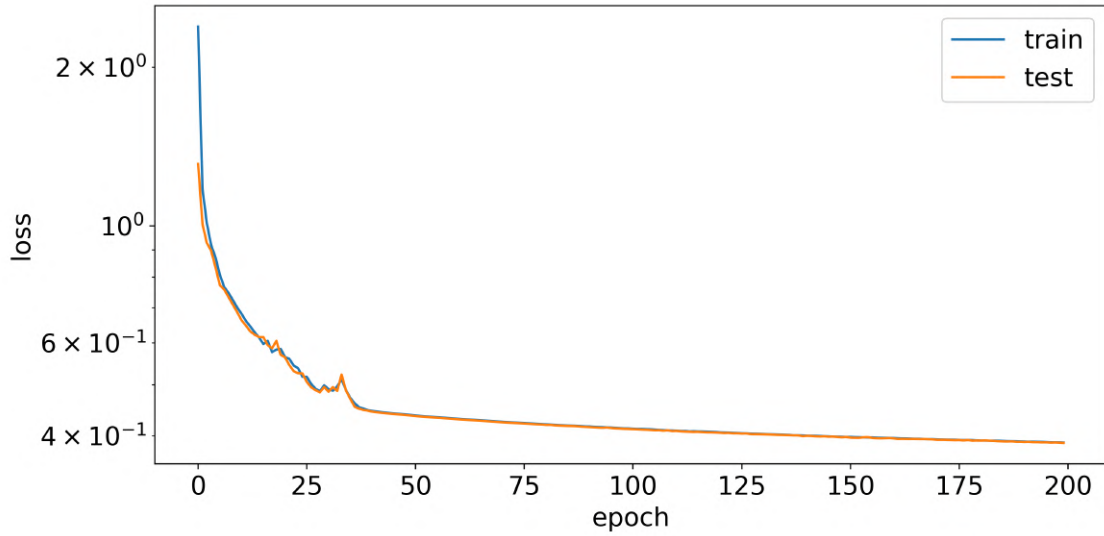


Figure 3.5: SAE training and validation loss by epoch. Where loss = encoder loss (L1 norm) + decoder loss (L2 norm) + end to end loss (L1 norm).

3.2.4 SAE Validation

For diffuse albedo texture types 1 – 6, we employ the SAE to predict parameter maps and recovered textures—with and without modifications to the latent space. We then evaluate the resulting outputs using the performance heuristics from section 3.2.3. Additionally, using the LUT search generated maps as a benchmark for chromophore range and distribution. Finally, we motivate the existence of the SAE approach through speed and accuracy comparison with LUT search based methods.

SAE Mapping: The SAE approach does the same mapping with a learned neural representation of the LUT. Since it is trained to output continuous values, the resulting parameter maps are much smoother.

3.2. Results and Discussion

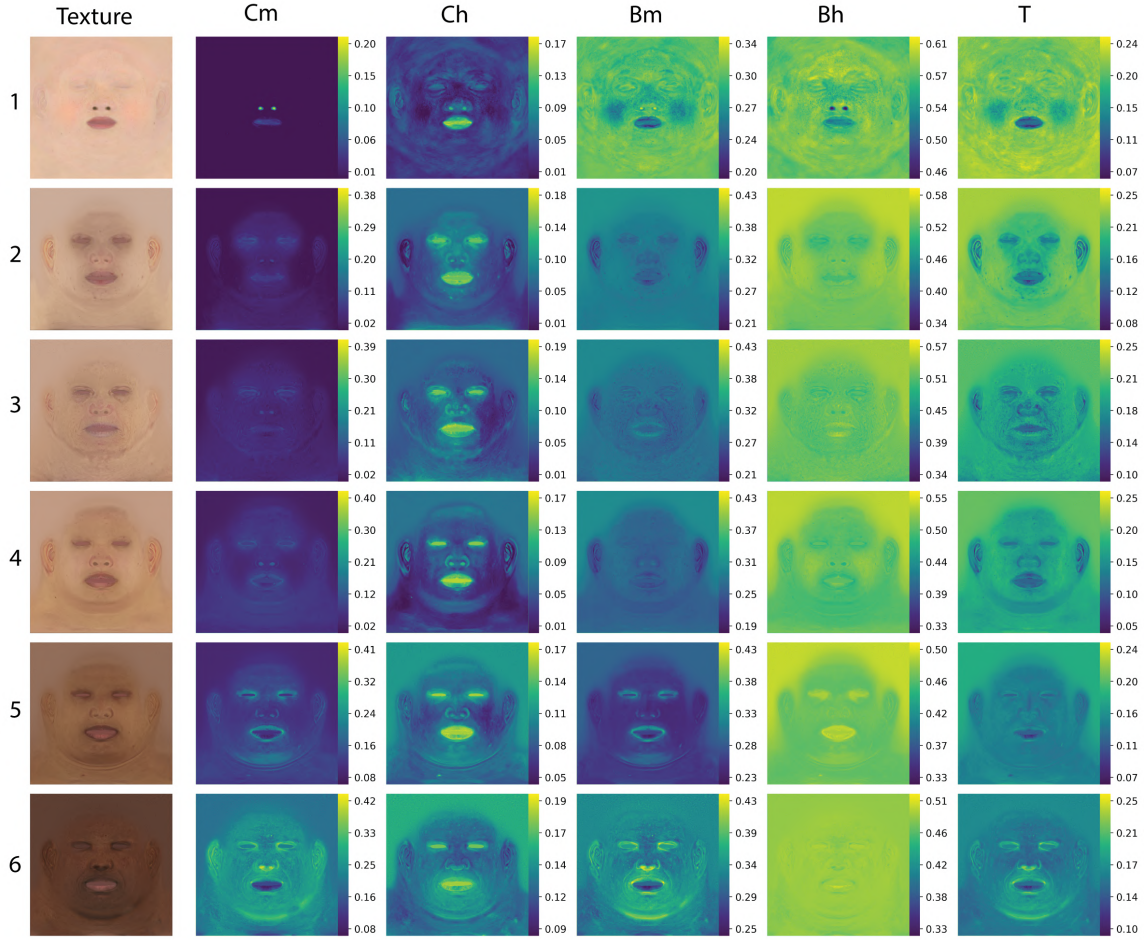


Figure 3.6: Maps generated by pixel-wise SAE. Top to bottom shows Fitzpatrick types 1 through 6. $C_m[0.001,0.5]$, $C_h[0.001,0.32]$, $B_m[0,1]$, $B_h[0,1]$, $T[0.01,0.25]$ mm as seen in Table 2.1

Image Recovery: We want to retain all the information in a texture to allow textures to be modified in 1 or more of the latent parameters with even the most subtle changes. To this end, the recovery heuristic is used to assess the difference between original and recovered image passed through the SAE with no changes to the latent space.

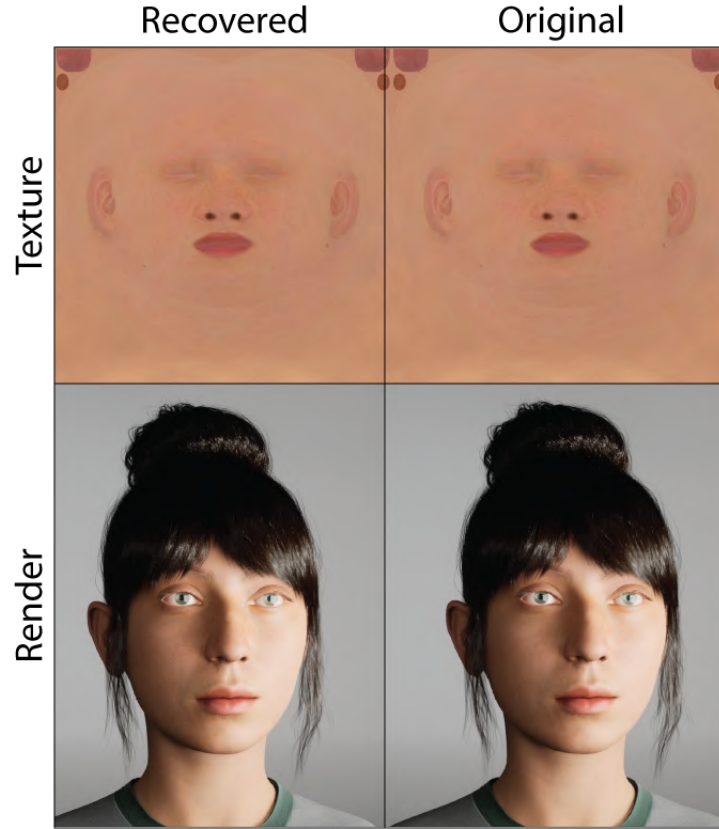


Figure 3.7: SAE recovered and original diffuse albedo texture and the rendered model for each.

Our ability to reconstruct textures depends largely on how complete we sample the range of parameters found in any given image. For this reason, we test one of each skin type to verify a consistent dE loss between original and recovered. It is often acceptable to test types 1 and 6 as these represent the edge cases and are most often where the model breaks down.

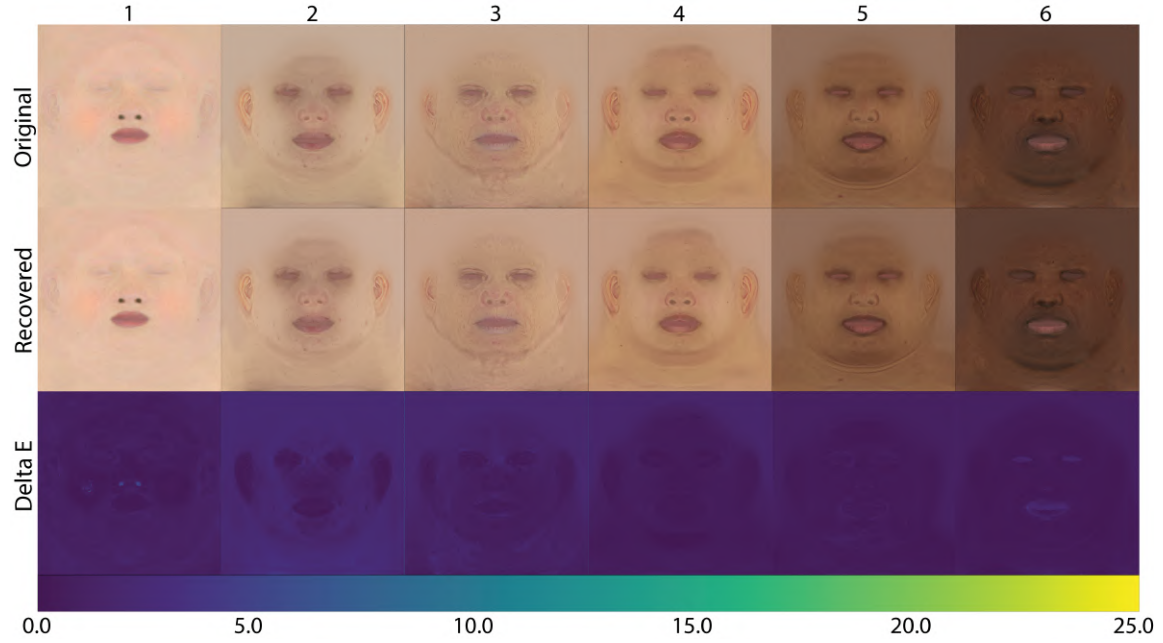


Figure 3.8: Original, recovered and delta E color difference for no changes in latent space.

Fitzpatrick	Min dE	Mean dE	Max dE
1	0.000	0.771	13.171
2	0.000	1.681	3.929
3	0.000	1.125	4.154
4	0.000	0.612	2.414
5	0.000	0.488	1.983
6	0.000	0.407	6.037

Table 3.3: Table of Delta E statistics for each texture.

The low average dE loss across all skin types suggests the model has a good distribution of parameters and colors. Typically, mean delta E values below 5 are nearly imperceptible. Because we care about *changes* in the texture, it is best to prioritize an SAE that allows more plausible modifications to the colors, even in the case of a slight increase in dE.

Outliers: Higher than average max dE values can occur from input RGB values far outside the LUT range or from inputs that correspond to the most common metamers. Some of the highest loss pixel regions are noticeable in the hair of texture 1 which is not

surprising as it is a lower quality texture that is not fully unlit. If we find the indices of the highest loss pixels, this can provide insight into the source of the errors. For example, the following pixels are common in textures 1 and 2 (see Figure 3.2.2) respectively:

Texture	RGB Input	Latent Space	RGB Prediction
1	[57, 85, 81]	Cm: 0.28 Ch: 0.47 Bm: 0.40 Bh: 0.20 T: 0.40	[84, 87, 92]
2	[214, 152, 101]	Cm: 0.05 Ch: -0.03 Bm: 0.40 Bh: 0.60 T: 0.20	[215, 192, 135]

Table 3.4: Demonstration of RGB inputs outside the range of expected values and resulting latent and recovered values, as seen in textures 1 and 2 of Figure 3.2.2. We expect outliers in non-scan based or lit “in-the-wild” images.

If we find the indices of the highest loss dE values, we can then compare the input and recovered pixel values, if they are vastly different or the latent/recovered values don’t make sense, it’s likely one of two issues. The first is inputs outside the LUT domain, which could mean a non-biologically based texture or an LUT of insufficient size. The second possibility is input pixels that correspond to *metamers* that are repeated frequently in the LUT.

The highest max dE values occur in textures 1 and 2. Though a small number of outlier pixels doesn’t impact the final image, they can give us valuable insight into the LUT and the learned representation. In the case of Texture 0, the input pixel is clearly far outside the range of normal skin color. As for Texture 1, the pixel is metamer found 5 times in the LUT, which hinders reconstruction. We can use this information to iteratively add or remove LUT values in order to fill in or mitigate gaps that may exist.

Image Editing: Through modifications applied to the latent space parameter maps, we can generate color changes with biophysical accuracy. It is worth noting that while at a pixel level, the changes are biophysically based, at the texture level, they require the work of subsequent chapters to be spatially coherent. We demonstrate the effect of changes to C_m and C_h using simple scaling of the two parameters.

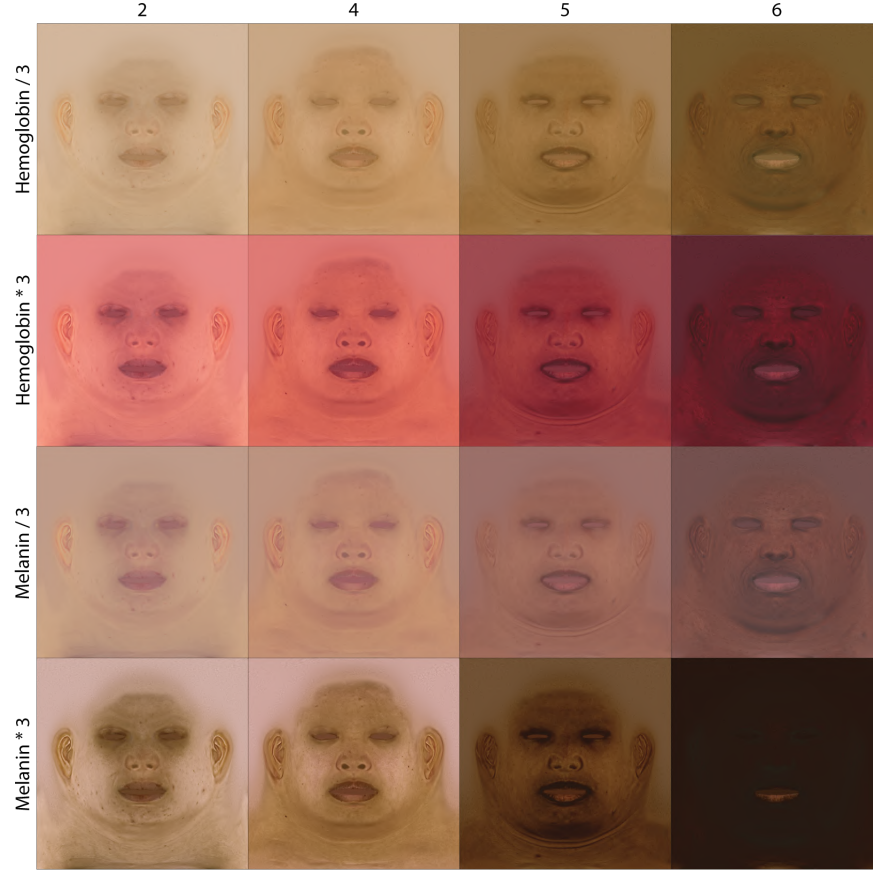


Figure 3.9: Columns left to right: Fitzpatrick types 2,4,5,6. Row 1: scaled down by 3. Row 2: melanin scaled up by a factor of 3. Row 3: hemoglobin scaled down by 3. Row 4: hemoglobin scaled up by a factor of 3.

Hemoglobin Scaling: In Figure 3.9 we see the effect of uniform scaling of C_m and C_h . In row 1 we notice a more pale skin tone as expected with decreased C_h which is associated with blood perfusion and redness. In row 2, we see a dramatic increase in redness and disproportionate modification in certain areas that have naturally high C_h to begin with, such as the lips.



Figure 3.10: Rendered models with $C_h * 1, 1.5, 2, 3$

Melanin Scaling: If we decrease the melanin for a very fair skin type 1 texture or increase it for a darker type 6 texture, we can very quickly extend beyond the bounds of plausible skin colors. Additionally, if we increase the C_m of an already dark type 6 texture, the amount of light absorbed in the epidermis will overpower the effect of any other chromophores, leading to substantial loss of detail—as seen in Figure 3.9 where texture 6 has the C_m map scaled up by a factor of 3.

Figure 3.11: Rendered models with $C_m * 1, 1.5, 2, 3$

Performance

If we compare the encoding and decoding times for SAE and the LUT search method, the power of the SAE becomes increasingly salient as we increase the texture sizes.

Image Size	SAE Encode(s)	SAE Decode(s)	LUT Encode(s)	LUT Decode(s)
(128, 128, 3)	0.0086	0.0070	1.4814	3.5325
(256, 256, 3)	0.0077	0.0080	5.8544	14.3369
(512, 512, 3)	0.0732	0.0829	269.4374	307.6881
(1024, 1024, 3)	0.0752	0.0867	—	—
(2048, 2048, 3)	0.0846	0.0986	—	—
(4096, 4096, 3)	0.1337	0.3133	—	—

Table 3.5: Average times to encode and decode diffuse albedo textures using the SAE and LUT search methods.

Even for a smaller 512x512 size texture, we see the LUT search takes several orders of magnitude longer for encoding and decoding, respectively. Besides, even if we employ multithreaded, or GPU accelerated search methods, we would still suffer quantization. This supports our claim that the main utility of the search based method is to have reference

maps that are constrained to the simulated values only.

3.3 Use of Real Images

We can also utilize the SAE to decompose 3D diffuse light cage images into the 5 parameter maps, as seen in Figure 3.12. This has many potential use cases, from biomedical diagnostics to example-based texturing (see Chapter 4). The resulting maps give us real-world distributions of chromophores and thickness. The plausibility heuristic is satisfied as we

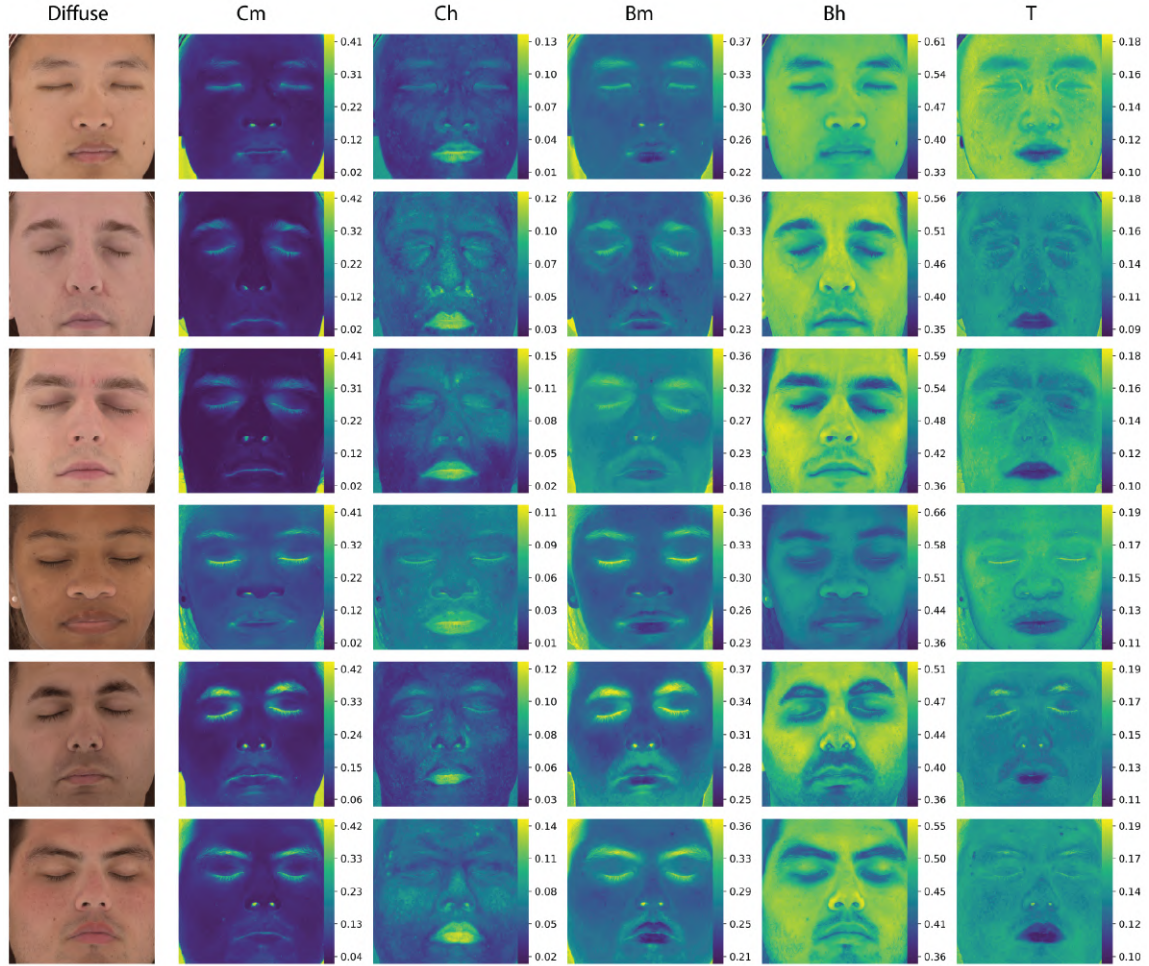


Figure 3.12: Diffuse light cage images of faces from the 3DRF dataset [USC24, SGDM11, MHP⁺07], decomposed into biophysical parameters using the SAE.

see increased C_m in hairy regions, increased C_h and decreased T in the lips. We also notice

very convincing blood perfusion in areas like the cheeks. The ranges of parameter values are also comfortably within those expected in the medical literature, satisfying the latent heuristic.

3.4 Conclusion

We demonstrate a methodology for training deep supervised autoencoders (SAE) using simulated data (LUT) to represent light transport in human skin. Using subtractive pre-processing methods on an exhaustive pre-computed simulation LUT, we pre-filtered rows of the LUT with the highest repetition of metamers. Additionally, we introduced heuristics for evaluating SAE performance and LUT quality. The ability to extract and manipulate properties related to skin color has applications in biomedical diagnostics, real-time graphics, cosmetics, and data augmentation. With the ability to make fast and meaningful adjustments to skin color in a pixel-wise manner, a clear need to address spatial coherence is addressed in Chapter 4.

Our Contributions

- Developing a supervised-autoencoder (SAE) model that learns the mapping between RGB values and biophysical parameters, significantly improving the performance and accuracy compared to LUT-based methods.
- Creating a comprehensive dataset (LUT) using light transport simulations that serves as the basis for training and evaluating the SAE model.
- Introducing heuristics for evaluating the SAE model’s performance, including Delta E color difference, latent space validation, speed of inference, and plausibility of the generated biophysical parameter maps.
- Demonstrating the application of the SAE model in making biophysically accurate color adjustments to diffuse albedo textures, with potential applications in biomedical diagnostics, real-time graphics, cosmetics, and data augmentation.
- Iterative simulation and training methodology, including data filtering methods to reduce metamers and allow larger, more descriptive LUTs for training.
- Addition of skip connections in the SAE to mitigate vanishing and exploding gradients.

Chapter 4

Dense Landmarks and Spatial Distribution of Chromophores

Though realistic digital humans are an essential part of games, movies, and XR applications, they still pose challenges due to the spatially varying and dynamic effects of subsurface scattering of light and its impact on color. Skin color can be modified in a biophysically accurate manner using supervised-autoencoders (SAE)s, but the pixel-wise nature of these models doesn't fully capture the spatially varying distribution of chromophores and resulting color. We demonstrate a novel methodology using dense landmarks to transfer latent biophysical properties between images and textures. We then show how one can utilise SAEs as a tool for data augmentation, training a dense landmark model (DLM) with synthetic data to enable coordinate space agnostic landmarks.

4.1 Introduction

The realism of digital humans depends on accurate skin modeling, where subsurface scattering and absorption of light play pivotal roles. Skin color in digital humans is driven by complex interaction of light with the epidermis and dermis layers, a process that can be quantified using Monte Carlo photon simulations to determine the spectral reflectance based on chromophore concentration and skin layer thickness [DJ06]. Key biophysical parameters such as the volume fraction of melanin (C_m) and hemoglobin (C_h), the eumelanin to pheomelanin ratio (B_h), the oxy to deoxy-hemoglobin ratio (B_h), and epidermal thickness ($T[\mu_m]$) are critical for these simulations [DJ06]. We can then convert these spectral reflectance values into RGB values using the CIE 1931 color transformation system, creating a lookup table for various biophysical parameter combinations and corresponding RGB pixel values [SG31].

Leveraging SAEs to learn the mapping between skin parameter values and RGB color gives us a way to edit and analyze color from a biophysically based perspective. This perspective, however, lacks the spatial context required to generate new textures or make meaningful changes to existing textures. Due to the complex and high-level nature of skin, we require a facial coordinate system in order to apply changes to specific regions.

With the ability to change diffuse skin color, we still need a data-driven approach to determine the distribution of each skin parameter. Recent advances in dense landmark

models present an effective tool for image and texture mapping. However, many models fail to capture the entirety of facial features, such as the ears, and do not maintain landmark density across different expressions and movements. These limitations are particularly evident when adapting models to UV unwrapped textures, where traditional DLMs exhibit reduced effectiveness. This shortcoming is a result of the training data being exclusively 3D.

In response to these challenges, this work introduces a Dense Landmark Model (DLM) trained on synthetic data to facilitate biophysically based adjustments to skin color in a spatially coherent manner. Using screen space and UV space projections of selected vertices of digital human faces gives every predicted landmark a meaning in both coordinate systems. This approach allows both 3D images and textures of differing topologies to be used in example-based texture modification.

Utilizing SAEs and DLMs in unison, we efficiently map skin color to its biophysical latent space, allowing for real-time adjustments of skin albedo textures encoded as parameter maps for C_m , C_h , B_m , B_h , and T [AHX22]. This biophysical latent space is crucial for modeling the skin color changes associated with spatially and temporally varying phenomena, such as age and emotion. Our method provides a comprehensive solution for creating and analyzing realistic digital and in-the-wild humans with potential applications in games, extended reality, identity verification, and biomedical diagnostics.

In the following sections, we introduce our spatially coherent and biophysically based skin texturing pipeline. Next, we describe how affine transformations are used to extract screen and UV coordinates from 3D objects and in augmenting data for computer vision applications. We then detail our synthetic data pipeline, which procedurally generates a dataset of 3000 synthetic face images (3D renders) and their corresponding landmarks (screen space vertices), utilizing the SAE to augment the dataset with biologically accurate variations. Next, we use our synthetic data to train a DLM.

4.2 Biophysically Based Texturing

The portrayal of human skin in digital environments traditionally focuses on achieving photorealistic texture and shading, primarily through techniques like high-resolution scanning, light cages, and Monte Carlo photon simulations. The latter method provides a spectral analysis of light absorption and scattering through skin layers but doesn't address the spatial distribution of chromophores that contribute to skin tone variations due to biological or emotional changes.

In order to capture the high-level color dynamics of skin, we take advantage of images and 3D scans of real humans. This data-driven approach is the most practical way to portray the endless complexity that occurs at many levels of resolution to create the resulting color distributions.

We propose the four-step pipeline seen in Figure 4.1 in order to leverage 3D images

and scan-based textures for biophysically accurate modifications to digital humans. First, the example and target are aligned using point correspondences and then encoded into their respective biophysical parameter maps. Next, the maps relevant to the biological process being modeled are blended, creating a biological hybrid of that property between the target and the example. Algebraic modifications are optionally applied to the blended result, which is then decoded to reveal spatially coherent and biologically plausible changes in diffuse skin color. The methodology described in this section is related to our previous work [JCL⁺23].

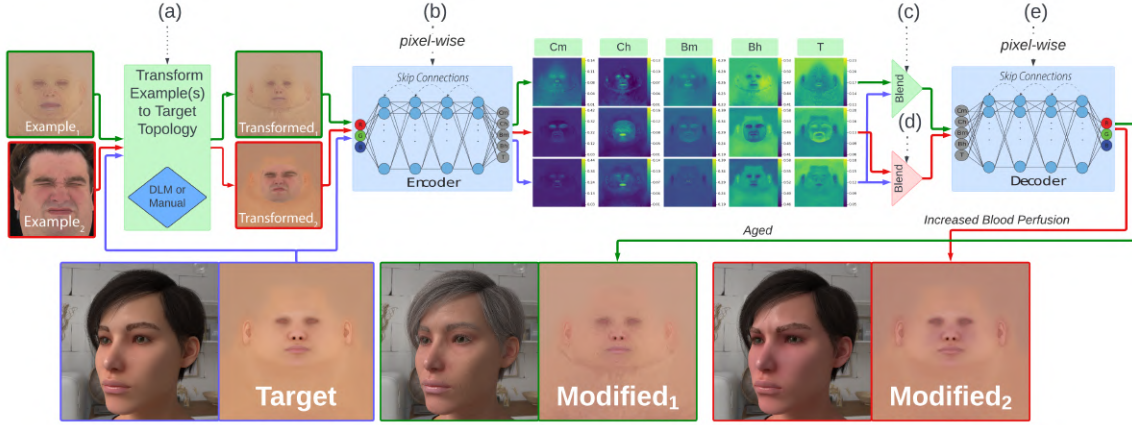


Figure 4.1: High level overview of our spatially aware, physically based texturing pipeline. The target texture is modified using an age example (Example₁), and an increased blood perfusion example (Example₂). Stage (a) the examples are transformed using point correspondence employing Dense Landmarks or manually using FaceForm Wrap. Stage (b) encodes the images from RGB to biophysical parameter maps. Stage (c) then blends the maps between the Target and Examples. Finally, the blended maps are decoded to produce biophysically plausible changes to skin color.

4.2.1 Age

Skin aging is associated with thinning of the epidermis and dermis layers, in addition to the decrease in chromophores [IGAJS15]. We model the uniform changes using a polynomial model for C_m and C_h and scalar transforms for B_h and T as a function of aging [IGAJS15]. To account for non-uniform changes in skin, we use an example albedo texture to provide distributions of C_m and C_h that simulate age-related blemishes, freckles and sun spots. The example texture is matched to the target texture (the texture to be aged) using dense landmarks and Delauney triangulation. The example and target textures are both Texturing XYZ models.

The autoencoder is used to convert aged models into parameter maps that can be used to increase or decrease biophysical parameters for different regions of the face. Regions of melanin and hemoglobin changes in the aged example texture are extracted using the encoder stage and blended with the encoded maps associated with the target texture. The latent space of the modified target texture is decoded to reveal biophysically plausible aging effects on a human face.

4.2.2 Emotion

Skin color variations due to changes in emotion are typically associated with blood perfusion [ATHPLM⁺23, LLZ⁺18]. In this work, we focus on modelling skin color changes to the human face due to the emotion of anger. Our approach for modeling the effect of anger involves leveraging dense landmarks to adapt blood perfusion (C_h , B_h) based on textures associated with neutral and angry expressions. For a given face in the FaceScape dataset [YZW⁺20], we morph the neutral and angry expressions to align with the landmarks of the target texture. Using dense landmarks, the neutral and angry textures are aligned to the target texture to isolate skin color change from geometric changes in expression. The skin color changes are encoded into parameter maps. The discrepancies in the maps associated with neutral and angry expressions are used to generate difference maps that are applied to the target texture and decoded to reveal biophysically plausible effects of anger on a human face.

Our approach to example-based texturing requires mapping between 3D and various UV topologies. In the age example of Section 4.2.1 we manually found point correspondences to align the target and example textures, achieving the best possible alignment. In the emotion example of Section 4.2.2 we utilised dense landmarks to automatically find these points. The latter method is much faster—fractions of a second as opposed to 15 to 20 minutes for the manual re-topology—yet is limited to just the face and is less reliable.

4.2.3 Re-topology and Texturing

The industry standard for wrapping a texture onto a model with a different shape and/or UV topology is a manual workflow to find point correspondences between two characters. In Figure 4.2 we show a texture wrapping and re-topology of a 3D morphable model (3DMM) [LBB⁺17] using Faceform Wrap [Fac24].

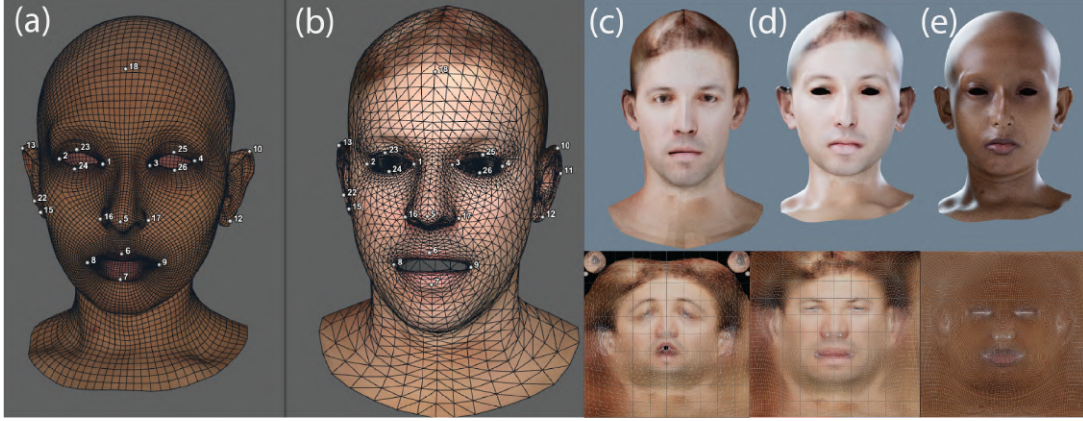


Figure 4.2: Example re-topology, a 3DMM generated from an image of the author is re-topologised in order to apply the texture to a Texturing XYZ model, using FaceForm Wrap to manually find point correspondences. (a) and (b) Show the manual point selection of Texturing XYZ model and 3DMM respectively. (c) 3DMM (top) and its diffuse albedo texture (bottom). (d) The wrapped 3DMM texture applied to the Texturing XYZ model (top) and the re-topologised texture (bottom). (e) Original Texturing XYZ model (top) and original texture (bottom).

One motivation for improving DLMs to include areas such as the ears is to improve the performance of photogrammetry and 3D morphable model based inverse rendering methodologies. To include the ears would lead to better landmark alignment and improved results in both single image and multi view reconstruction.

4.3 Synthetic Data Pipeline

With increasingly powerful game engines and 3D modeling software, the procedural creation of realistic characters and environments is becoming more feasible over time. These improvements make synthetic data an increasingly attractive alternative to inconsistent human-labeled data. Generating images and video in this manner gives us an omniscient view of lighting, geometry, and material properties at every location in the scene and every pixel that is rendered to the screen [WBH⁺21]. To this end, we construct a synthetic data pipeline to generate a multi-use computer vision dataset.

To procedurally create the data we leverage Blender and the Human Generator Python API to render digital humans with adequate variance in appearance. The procedural rendering of characters (seen in Figure 4.3) is relatively straightforward; it is the annotation of the landmarks that requires a small amount of background.



Figure 4.3: Procedurally generated characters from our synthetic dataset.

Understanding the fundamental coordinate spaces and how transformations are applied to these spaces is essential for any 3D rendering process, including the realistic rendering of human skin. In computer graphics, different coordinate systems are used to manage how objects are represented and manipulated within a virtual environment. In this work, we apply these transforms to manipulate a 3D character, camera, and lights to generate a large number of synthetic faces with sufficient diversity. Moreover, for each vertex of each character, we must apply a coordinate space transform to find the screen and UV coordinates, which are used as 3D and UV landmarks (see Figure 4.4).

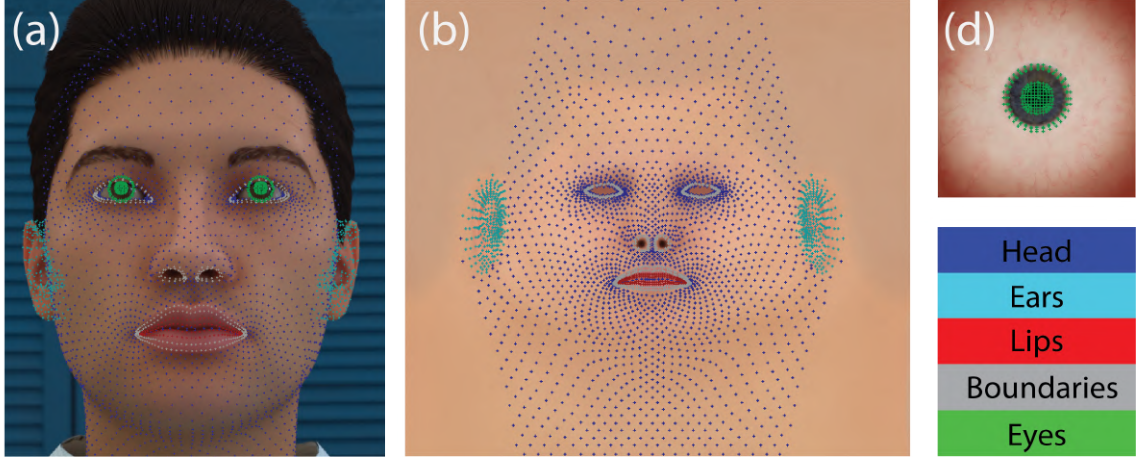


Figure 4.4: Annotations for one of the many procedurally generated characters, and the corresponding annotated diffuse albedo map and eye texture map. (a) is the annotated render, (b) is the head texture, (c) is the eye texture(s).

4.3.1 Affine Transformations

Affine Transformations are transformations that include a translational component, which necessitates the addition of an extra dimension than would be required for linear transforms such as rotation, sheering and scaling [Fol96]. In this work we utilize affine transformations for transforming coordinate spaces and in augmenting images and coordinates. Transforming object coordinates from local space to screen space represents a series of steps that can be represented using a series of coordinate transformations—local to world to view to screen—giving us consistently labeled vertices that can be used as dense landmarks. When dealing with generated image landmark pairs, we utilize a 2D affine transformation to augment the data, adding random variance to face shapes in our dataset. This helps to avoid overfitting due to the repetition of certain face shapes. The general form of a 3D transformation matrix $\mathbf{M}$ that combines translation, scaling, rotation, and shearing is

$$\mathbf{M} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Scaling factors along the x , y , and z axes are denoted as s_x , s_y , and s_z , respectively. The scaling transformation matrix $\mathbf{S}$ is

$$\mathbf{S} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotation around the x , y , and z axes by angles θ , ϕ , and ψ (in radians) are applied using

$$\begin{aligned} & \mathbf{R}_x(\theta), \mathbf{R}_y(\phi), \mathbf{R}_z(\psi) \\ & \equiv \\ & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) & 0 \\ 0 & \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} \cos(\phi) & 0 & \sin(\phi) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\phi) & 0 & \cos(\phi) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} \cos(\psi) & -\sin(\psi) & 0 & 0 \\ \sin(\psi) & \cos(\psi) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

The combined rotation matrix $\mathbf{R}$ is the product of the three rotation matrices

$$\mathbf{R} = \mathbf{R}_x(\theta) \cdot \mathbf{R}_y(\phi) \cdot \mathbf{R}_z(\psi)$$

Translation components along the x , y , and z axes are denoted as t_x , t_y , and t_z , respectively. The translation transformation matrix $\mathbf{T}$ is

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The complete affine transformation matrix $\mathbf{M}$ is obtained by multiplying the scaling, rotation, and translation matrices is [Fol96]:

$$\mathbf{M} = \mathbf{T} \cdot \mathbf{R} \cdot \mathbf{S}$$

4.3.2 Coordinate Space Conversion

Local Space is the starting point for coordinates extracted from a 3D object. Local space, or object space, refers to coordinates that are local to an object. Transformations in this space are applied directly to the object's vertices. These transformations include scaling, rotation, and translation, which modify the object's size, orientation, and position relative to its local origin. We transform local space coordinates ($\mathbf{p}_{local}$) to world space coordinates ($\mathbf{p}_{world}$) where $\mathbf{M}_{local}$ is the transformation matrix containing scaling, rotation, and translation parameters [Fol96].

$$\mathbf{p}_{local} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \quad (4.1)$$

$$\mathbf{p}_{world} = \mathbf{M}_{local} \cdot \mathbf{p}_{local} \quad (4.2)$$

World Space coordinates are used once an object's local coordinates are transformed relative to a global origin, which serves as a common reference point for all objects in the scene. This transformation is typically defined by a world matrix, which uniquely positions and orients each object in the global landscape of the virtual environment.

$$\mathbf{M}_{world} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$\mathbf{M}_{world}$ transforms vertices from local space to world space, $r_{i,j}$ and $t_{x,y,z}$ are the respective rotational and translational transform elements of the i 'th and j 'th rows and columns.

View space, or camera space, involves transforming world space coordinates into coordinates relative to the camera's perspective. This transformation is critical for determining how objects are viewed from a particular viewpoint and is facilitated by the view matrix. The view matrix adjusts all world space coordinates according to the camera's position and orientation, setting the stage for projection onto a 2D screen.

$$\mathbf{M}_{view} = \mathbf{M}_{camera}^{-1}$$

where $\mathbf{M}_{camera}$ is the camera's world matrix, and its inverse transforms world space to view space.

Clip Space coordinates remove or "clip" elements in the scene between the camera and the near field cutoff and elements beyond the far field cutoff. After coordinates have been transformed to the view space, they are further transformed to clip space using a projection matrix. This step involves converting 3D coordinates into a normalized form where they can be easily clipped and prepared for display on a 2D surface. The projection matrix defines how dimensions are projected and typically differs based on the camera's lens characteristics, like field of view and aspect ratio.

$$\mathbf{M}_{projection} = \begin{bmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

$\mathbf{M}_{projection}$ transforms view space to clip space. Where r , l , t , and b are the right, left, top and bottom of the near clip plane, n and f are the distances of the near and far clip planes respectively.

Finally, clip space coordinates are transformed into screen space, where they are mapped to specific pixels on the screen. This involves scaling to the screen’s resolution and adjusting based on specific rendering requirements. This final transformation ensures that the 3D world is accurately represented on a 2D display [Fol96].

$$x_{screen} = \frac{(x_{clip} + 1) \cdot width}{2}, \quad y_{screen} = \frac{(1 - y_{clip}) \cdot height}{2}$$

where (x_{clip}, y_{clip}) are the coordinates in clip space normalized to $[-1, 1]$. These steps are taken for each landmark in the dataset, described next.

4.3.3 Procedurally Generated Humans

To procedurally generate a large number of image-landmark pairs with sufficient variance, we use Blender and the Human Generator Python API. We begin by creating a random digital human, selecting hairstyles that minimally occlude the ears, and assigning random facial expressions. For each render, a camera is positioned at a random vertex on a hemisphere in front of the human, aimed at the head’s centroid, with the focal length determined by a uniform random variable. Lighting and background variability are introduced by randomly selecting and applying an HDRI map to the scene. This setup ensures varied camera perspectives, lighting conditions, and backgrounds for each image.

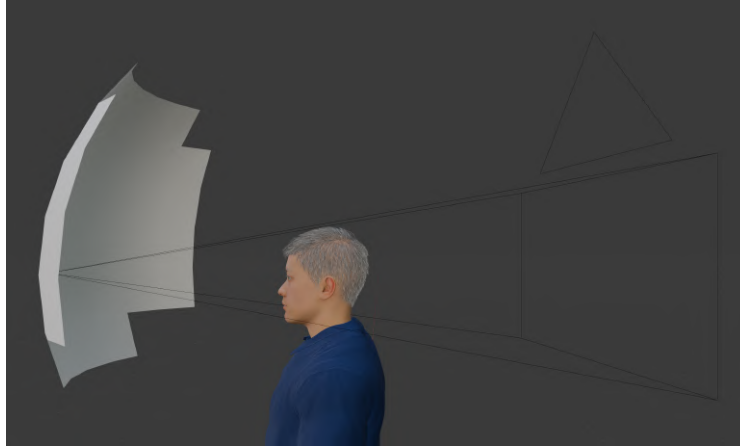


Figure 4.5: Cameras are spawned at random vertices of a hemisphere set to the centroid location of our synthetic landmarks for each character.

Due to the large amount of computation required to render thousands of diverse characters, we save more information than is needed for training a DLM. We also save diffuse,

glossy, depth, and ambient occlusion maps, in addition to the z and UV coordinates for each landmark. Our goal here is to provide a valuable dataset for graphics and computer vision research beyond the scope of this work.

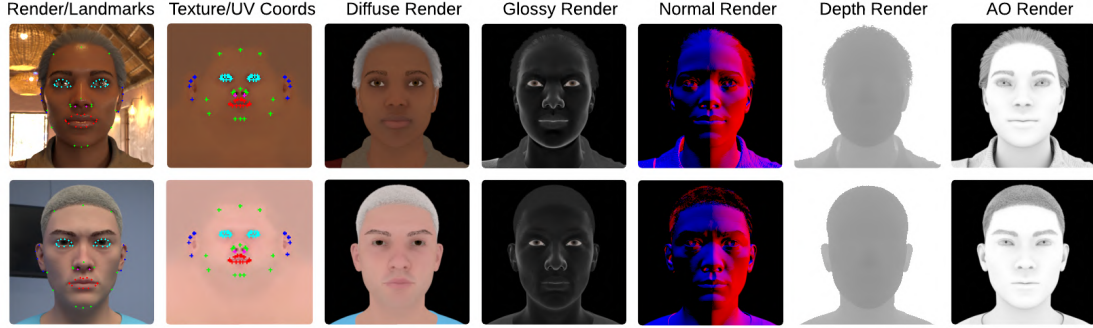


Figure 4.6: In addition to generating the render/landmarks for each character, we save the UV coordinates, z values, and additional render passes to facilitate a useful open source dataset for various computer vision tasks.

4.4 Data Augmentation Using SAEs

Due to the computational resources needed to render thousands of realistic characters, steps need to be taken to get the most out of the data. To improve the variability of skin tones in our dataset, we apply uniform modifications to the diffuse textures used to generate our characters.

- Increase/Decrease C_m and C_h for each texture.
- Decrease C_m for the lightest available skin texture.
- Increase C_m for the darkest available skin texture.

This gives us three times as many textures and allows us to include outliers with abnormal yet plausible chromophore concentrations. These textures still need to be rendered onto a 3D object, which is the computational bottleneck of our approach. Next we find a way to get more from less using augmentation of the rendered models.

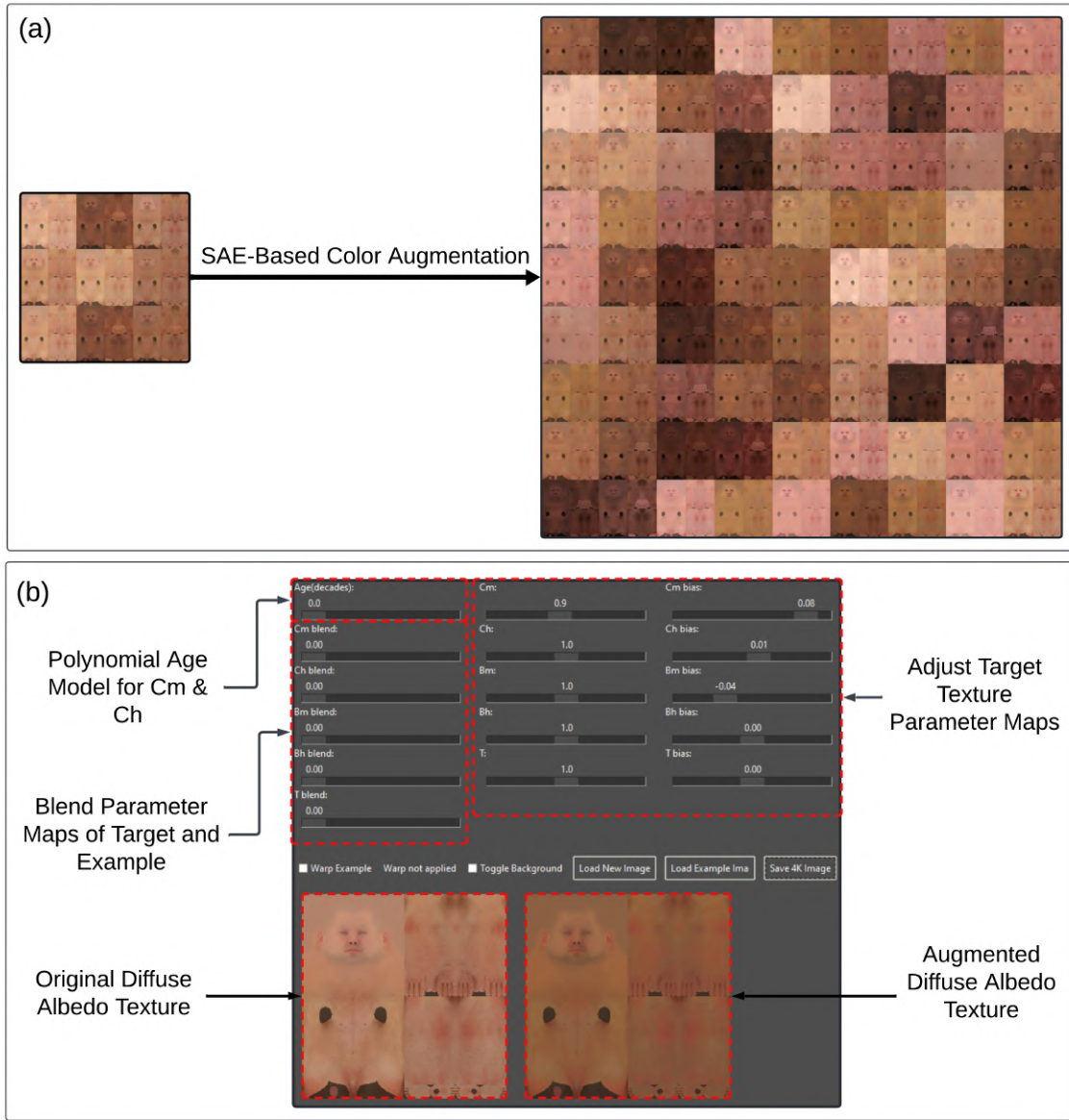


Figure 4.7: SAE-based augmentations used to create a larger and more comprehensive set of textures in our synthetic data pipeline. (a) Left is the original texture dataset, right is the augmented textures with a wider range and more variance in melanin and hemoglobin. (b) The offline texture editing GUI we made for visual/interactive adjustments to skin chromophores and thickness.

We expand our dataset by transforming the image and landmark pairs in order to get

a larger dataset with little extra computing power. First, we flip the image and landmarks over the y-axis, giving us a mirrored data point of the original. Next, we use a pre-trained image segmentation model to segment the hair, clothing, eyes, and skin [YWP⁺18]. We then run the image through the SAE and randomly increase and/or decrease C_m and C_h by a uniform random variable, using the created segmentation masks to only affect the skin pixels.



Figure 4.8: Example augmentations applied to the rendered images and landmarks using pre-trained segmentation model [YWP⁺18]. Where the top row is the original, and the bottom row (mirrored) has changes to skin, and hair using the SAE in addition to changes in clothing color.

4.5 Training a Dense Landmark Model

We introduce a DLM with the goal of texturing digital humans using 3D diffuse light cage data and high quality scan based data. Once a landmark is accurately predicted, we can apply that information to the target character’s texture, allowing transfer of latent biophysical properties. This goal is one of creating many diverse and realistic characters from a limited number of real life examples.

To learn a relationship between 3D renders, images, and textures of human faces and screen-space x,y coordinates, we employ a MobileNetV2 based ViT with additional regression layers [SHZ⁺18]. The additional layers learn to extract coordinates from the feature embeddings generated by the pre-trained ViT encoder [WBH⁺21].

For each image, the ViT encoder generates a set of feature embeddings. These embeddings are then passed through dense layers that serve as the regressor, where the loss L_{DLM} is the L1-norm between true and predicted landmarks is calculated.

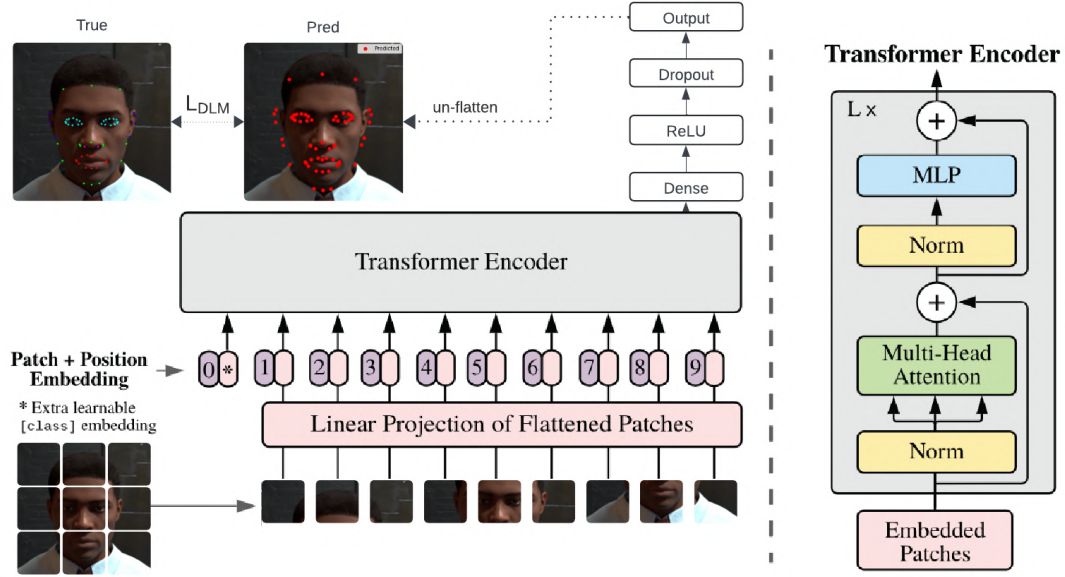


Figure 4.9: Schematic of our DLM with MobileNetV2 backbone, showing the patch embedding, encoding and regression for a synthetic data point. Image adapted from “An Image is Worth 16x16 Words” [DBK⁺20, VSP⁺23].

$$L_{DLM} = \sum_{i=1}^n (|x_{\text{true},i} - x_{\text{pred},i}| + |y_{\text{true},i} - y_{\text{pred},i}|) \quad (4.3)$$

Model Architecture:

- Feature Extraction:** The pre-trained MobileNetV2 model is used to extract features from the input images. The output of the MobileNetV2 encoder, which consists of convolutional layers, is processed to produce feature maps.
- Global Average Pooling (GAP):** The feature maps are then subjected to Global Average Pooling (GAP), reducing each 1280x1x1 feature map to a 1280-dimensional vector. This step ensures that the spatial dimensions are condensed into a single feature vector for each image.
- Regression Network:** The 1280-dimensional feature vector is passed through a series of dense (fully connected) layers. These layers are responsible for learning the mapping from the feature embeddings to the landmark coordinates. The architecture of the regressor is as follows:

- A fully connected layer of 512 neurons and ReLU activation.
- Dropout with a rate of 0.1 to prevent overfitting.
- A final, fully connected layer that outputs $2 \times \text{num_landmarks}$ values, representing the x and y coordinates for each landmark.

4.6 Implementation

Data Sources: We utilize a variety of data sources to best validate our approach, these include 3D light cage data from the 3DRF dataset [USC24], and high resolution textures from Texturing XYZ[Tex24].

Software: The digital humans are generated using Blender and the Human Generator Python API. Training of the DLM is done using PyTorch GPU with CUDA 11.8. Manual texture re-topology is done using Faceform Wrap [Fac24].

Code/Dataset: To access the dataset, please visit the Google Drive link that can be found in the Github repository at: <https://github.com/jj-tech-io/Deep-Albedo/blob/master>.

Hardware: The following results were achieved on a workstation equipped with a 13th Gen Intel(R) Core(TM) i9-13900K CPU at 3.00 GHz and 64 GB of RAM and GeForce RTX 4090 with 24 GB GDDR6X VRAM.

4.7 Results and Discussion

Due to the complex and spatially varying nature of skin structure and color, we employ an example-based pipeline to modify the skin color of digital humans. First, we present our synthetic dataset, which was created to train a Dense Landmark Model (DLM). Next, we evaluate the performance of our trained DLM and discuss its current limitations. Due to these limitations, we utilize the Mediapipe DLM [Goo23] in our texturing pipeline for facial areas. Finally, we demonstrate both DLM and manual point correspondence use of our spatially coherent texturing pipeline. This approach enables us to transfer latent biophysical parameters between diffuse 3D light cage images and different UV topologies, allowing for biophysically plausible modifications to skin color for applications such as aging and emotional expression.

4.7.1 Synthetic Dataset

We generated datasets with 1024x1024, 512x512, and 256x256 image sizes to enable use with a variety of CV model architectures. The majority of vertices of the head are included

to allow users of the dataset to select the most relevant landmarks for a given use case, annotations are shown on the rendered humans in Figure 4.10.



Figure 4.10: Landmark annotations on procedurally generated humans. Object vertices are converted to screen space and used as synthetic landmarks. Color-coded annotations: green for general face structure, red for lips, pink for nostrils, light blue for ears, and blue for eye contours. The dataset includes additional vertices and extra render passes beyond those used in this work.

One of the key variables for this procedural data is the selection of camera perspectives that align with the chosen CV model and the desired task being learned. More camera perspectives increase the difficulty of training exponentially, so it is key to only use views we expect to encounter with inference. To this end, we focus on views one would expect while using a laptop or smartphone camera, with perspective chosen as in Figure 4.5.

4.7.2 DLM

Due to the computational resources required for training a DLM with a non-trivial number of landmarks, much of the work surrounds pre-processing, augmentation, and selection of the most trainable landmarks. We gradually titrated the number of vertices from 2500 down to 107, and 61 landmark models, and the image size from 1024x1024 down to 224x224. The 107 landmark model shown in Figure 4.12 is trained on our 61 landmarks in addition to the facial perimeter, eyes, nostrils, and lips of the Mediapipe landmarks. This gives the option of running a second inference pass to get higher density in the regions of the face, such as cheeks and eyes, crucial for facial re-targeting, image reconstruction, and, in our case, texturing. We have access to the UV coordinates of the 61 synthetic landmarks, which

allows us to transform them to texture space for transfer of latent biophysical properties from light cage data onto our digital human. Below is the loss per epoch with a train test split of 0.95 and 0.5 respectively.

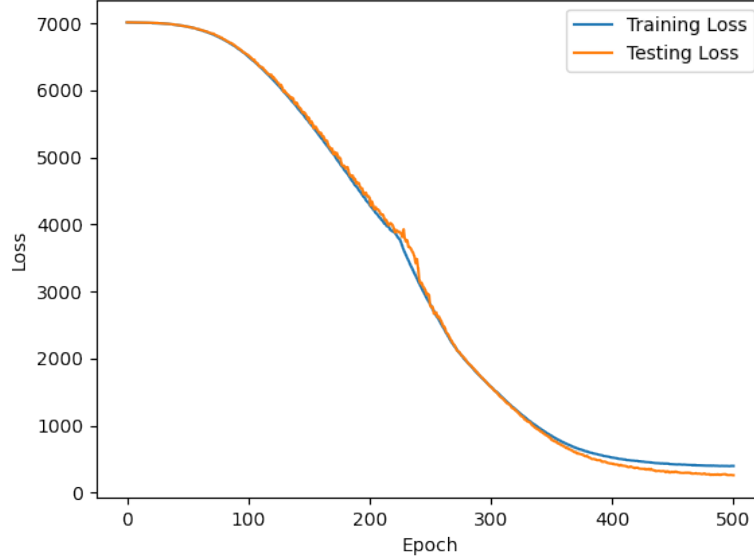


Figure 4.11: ViT based DLM training and validation loss by epoch.

In order to best train for inference on diffuse light cage data, we include both the renders and the diffuse render pass in the training and validation sets. To avoid overfitting, each image/landmarks pair is subject to a series of non-uniform (asymmetry in x and y) random affine transformations. We constrain these random transformations to maintain a bounding box of 20 to 70 pixels between the facial perimeter (Mediapipe face oval) and the edges of the image. We apply the same bounding box constraint to our diffuse light cage images.

We show promising results on the validation set, with some limitations when generalizing to “in-the-wild” images. In Figure 4.12 the lips and nose landmarks don’t align. This is likely a result of the limitations of face shape in the Human Generator API, in addition to the limited dataset size of 1500 before augmentation and 3000 after augmentation.

4.7.3 Spatially Coherent Texturing

We utilize our skin modification and texturing pipeline to demonstrate changes in chromophores associated with age and emotion. To motivate our work on DLMs we show the subtle difference in quality between manual and DLM based re-topology. Finally, we demonstrate augmentation of lit “in-the-wild” images.

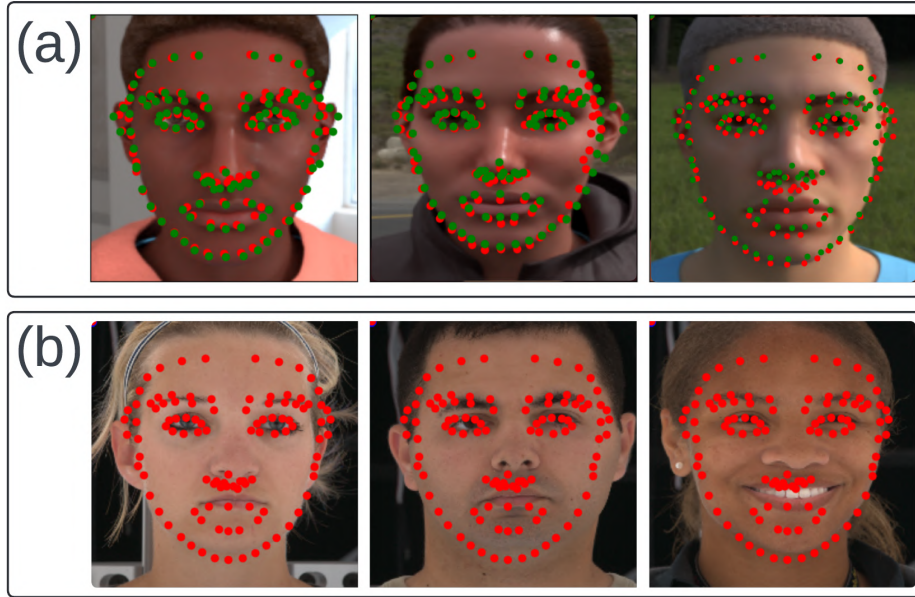


Figure 4.12: DLM inference results for 107 landmark inference, where (a) shows the synthetic data, and (b) shows the diffuse light cage images from the 3DRF dataset [USC24]. The green markings are the true value of selected screen space vertices and the red is predicted.

Face Only Landmarks Based Texturing: Using only face landmarks, we can effectively modify the digital human’s diffuse albedo texture through transfer of latent biophysical properties. Here we use Mediapipe landmarks, which are very accurate on 3D images and satisfactory for most UV topologies. The problem with this approach is that transferred properties will be discontinuous in a perimeter around the face, with the ears, upper forehead and neck limited to the original texture and any changes applied to the parameter maps writ large.

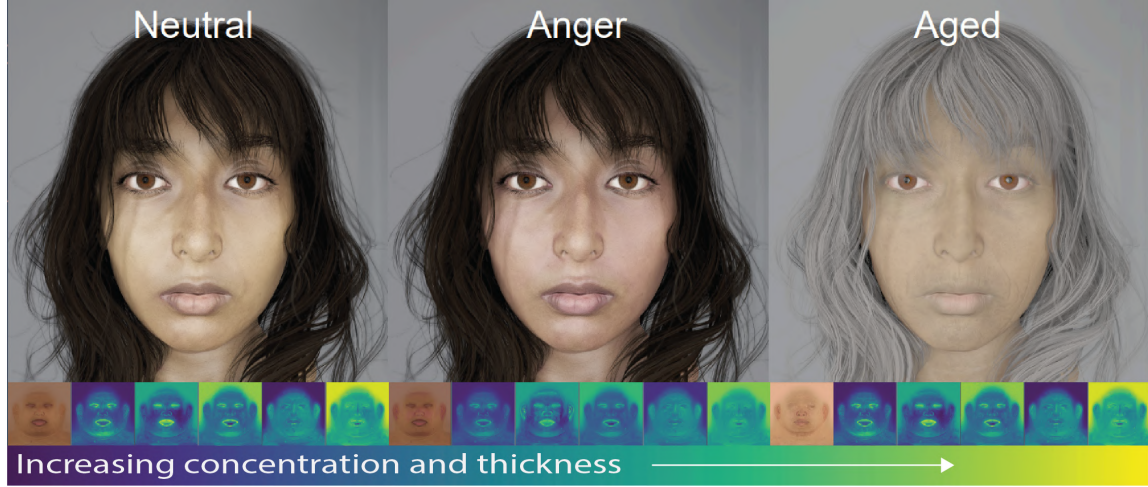


Figure 4.13: Digital human rendered in Maya with latent biophysical parameters transferred from example images and textures using dense landmarks. Anger and age properties are transferred from the latent space of FaceScape images and a Texturing XYZ example texture, respectively [JCL⁺23].

Using the face-only approach (as seen in Figure 4.13) works for characters where the sides of the face and the ears are occluded. For more detailed and non-occluded characters, manual re-topology and/or expanded landmarks are better suited.

Manual Point Correspondence Texturing: To encompass more of the head, including the ears, eyes, and lips, we manually align the target and example textures. For emotion, we see blood perfusion increases in the ears and cheeks. The points are used to transfer properties to textures for two Metahuman characters in Figure 4.14. For the age example, we notice the blemishes, sun spots, polynomial changes in C_m and C_h , and reduced T . The ideal DLM would be one that facilitates these texture-wide point correspondences while maintaining the accuracy of the Mediapipe landmarks.

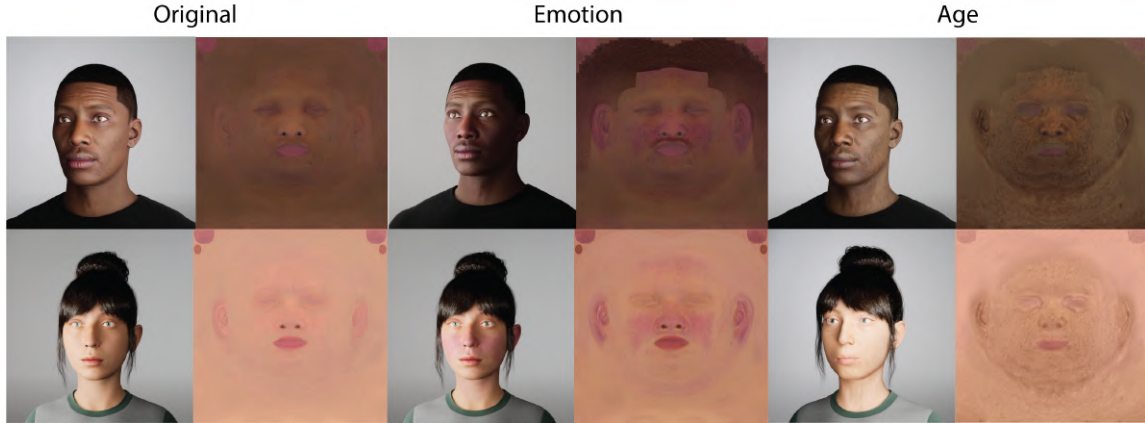


Figure 4.14: Metahumans rendered in Unreal Engine with our biophysically based texturing pipeline.

4.8 Conclusion and Future Work

We utilized our skin color to biophysical parameter SAE to augment a synthetic data pipeline for improved skin color variance. We then used the generated dataset to train a DLM for more comprehensive point correspondence between 3D images and UV textures of different topologies. Finally, we demonstrated the high-level spatially coherent skin color modification pipeline to produce biophysically plausible effects on digital humans. Further computational resources and more advanced architectures are required to enable the training of a DLM that can capture the full complexity of a greater range of vertices. With limited computational resources, the trained DLM is a proof of concept that requires larger computational resources to fully explore its utility in generalizing to 3D “in-the-wild” images.

Our Contributions

- A novel integration of SAEs and DLMs to enable spatially coherent editing of diffuse skin color based on biophysical parameters.
- A synthetic data pipeline for generating diverse human face datasets, incorporating biophysically based skin color augmentation using SAEs.
- Coordinate space-agnostic dense landmarks that facilitate mapping between 3D space, screen space, and various UV topologies, enabling targeted editing of skin color across different representations.
- Training of a DLM that includes often-overlooked facial features, such as ears, trained on our synthetic dataset.

Chapter 5

Conclusion

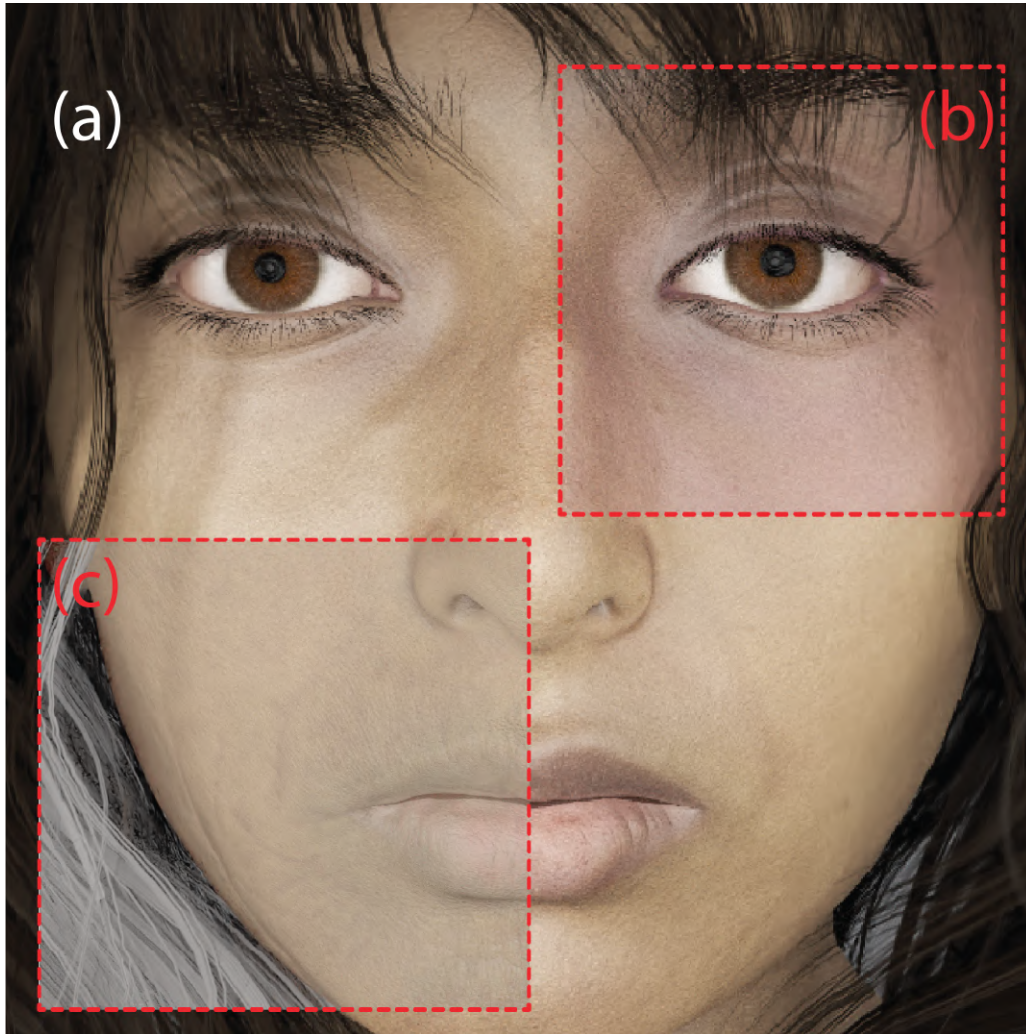


Figure 5.1: Maya render used in our work “Deep Albedo [JCL⁺23],” where (a) is the original diffuse albedo texture, (b) shows increased blood perfusion, and (c) shows aged diffuse albedo texture.

Humans have evolved to recognize each other. Our faces tell a story, with subtle changes in color and expression often saying more than the words being spoken. From the time we are born, we track the faces of our parents and then of our peers as we navigate complex social interactions. This makes digital humans a crucial element when trying to convey a feeling through a movie, game, or immersive experience. However, it comes with many challenges.

Human faces are highly detailed and ever-changing. To render such a complex object to a screen, in depth modeling of light behavior is needed. Skin poses a unique challenge, as light hitting skin is scattered and absorbed by chromophores in multiple layers of tissue. These chromophores such as melanin and hemoglobin are spatially varying and dynamic. So, for every skin pixel in an image, there is a corresponding concentration of the various chromophores that can explain it, and these values are a moving target.

If this sounds computationally expensive, it is. Movies and VFX houses spend millions on simulating light transport with human characters and human skin, often a large source of the workload. Often taking weeks to render a single scene. Real time graphics used in games and extended reality applications don't have the luxury of time, so the quality of the rendered characters is often much less photo-realistic.

This work bridges the gap between real-time and offline rendering quality. Using AI to learn a bidirectional relationship between skin structure and color. Allowing the real time modification of skin appearance based on its biophysical structure. Imagine a game character that blushes in real time as pixels are encoded, modulated, and decoded to increase hemoglobin and oxygen saturation. Rather than spending a day simulating light transport, we now have these results in fractions of a second.

This is not just great news for gamers or meta-verse enthusiasts, it also facilitates the augmentation of computer vision training data. We can now take real images in a training set and expand the dataset to represent all possible skin tones, medical conditions, ages, and ethnicity's. Modification of training data in this way ensures that apps powered by AI work well regardless of skin color. Moreover, the fast decomposition of skin color into its causal structure is an exciting prospect for biomedical applications.

5.1 Limitations

This work leverages simulated data, which is only as good as our models and approximations. We acknowledge the myriad approximations and simplifications in the optical models used as the best practical approximation for a real-time compatible model of skin color. Furthermore, the use of the RGB color space fails to capture the full complexity of the electromagnetic spectrum due to metamers. In the final chapters, we were limited computationally, thus our synthetic dataset may lack the size required for training more comprehensive dense landmark models (DLMs) or similar computer vision models.

The overall theme of simulation, data-driven, and synthetic data approaches to com-

puter graphics is exciting but carries a heavy responsibility in capturing a non-biased perspective. We did our best to portray the widest, most biologically accurate range of colors, shapes, and expressions in this work. We feel the most ethical approach to this work is to open source all code and generated data to facilitate the highest number of people having a say in the evolution of the dataset and what it contains.

5.2 Future Work

This work opens up several avenues for future research and development, which can be broadly categorized into four main areas: data studies, machine learning, synthetic data, and radiance fields.

Data Studies: The collection of robust hyperspectral (HSI) and light cage data in one unified dataset could help to train and validate models for RGB to HSI Vision Transformers (ViTs), reducing the burden of metamers. Capturing spatial and spectral correlation between pixels of an image, facilitating an RGB to HSI step with subsequent HSI-based supervised autoencoder (SAE) deployment, would significantly enhance the accuracy and realism of rendered skin.

Machine Learning: With more computational resources, one could generate many pairs of diffuse, specular, and combined render passes for the development of inverse lighting models. This would allow the effective use of the diffuse skin color SAE on lit “in the wild” images, an exciting prospect for biomedical diagnostics and digital avatar personalization. Similarly, generating depth map renders could be a very effective tool for training models to infer the depth component of “in the wild” images, useful in photogrammetry applications.

Synthetic Data: Exploring synthetic data generation techniques further, we could create more diverse and representative datasets, enhancing the training of machine learning models. This includes expanding the range of biophysical parameters and exploring different lighting conditions to simulate a wider variety of real-world scenarios.

Radiance Fields: Utilizing neural radiance-fields or Gaussian splat-maps to reconstruct 3D scenes and 3D meshes from images and video, we could then segment and texture the objects using neural material networks. Such networks would represent parameterized materials for things like skin, cloth, or hair.

In conclusion, our work sets the stage for significant advancements in the field of digital human rendering and texturing, with implications for gaming, virtual reality, biomedical imaging, and beyond. By continuing to refine our models and expand our datasets, we can achieve even greater realism and inclusivity in digital representations of human skin.

Bibliography

- [AHX22] Carlos Aliaga, Christophe Hery, and Mengqi Xia. Estimation of spectral biophysical skin properties from captured rgb albedo, 2022.
- [AK90] James Arvo and David Kirk. Particle transport and image synthesis. *Computer graphics (New York, N.Y.)*, 24(4):63–66, 1990.
- [ARVT⁺22] Arian Arjomandi Rad, Robert Vardanyan, Santhosh G. Thavarajasingam, Alina Zubarevich, Jef Van den Eynde, Michel Pompeu B O Sá, Konstantin Zhigalov, Peyman Sardiari Nia, Arjang Ruhparwar, and Alexander Weymann. Extended, virtual and augmented reality in thoracic surgery: a systematic review. *Interactive cardiovascular and thoracic surgery*, 34(2):201–211, 2022.
- [ATHPLM⁺23] Victor H. Aristizabal-Tique, Marcela Henao-Pérez, Diana C. López-Medina, Renato Zambrano-Cruz, and Gloria Díaz-Londoño. Facial thermal and blood perfusion patterns of human emotions: Proof-of-concept. *Journal of thermal biology*, 112:103464–103464, 2023.
- [BABANM22] Paris Buttfield-Addison, Mars Buttfield-Addison, Tim Nugent, and Jon Manning. *Practical Simulations for Machine Learning: Using Synthetic Data for AI*. O’Reilly, 2022.
- [Ber04] Jeffrey D. Bernhard. Clinical dermatology. 3rd edition. *Journal of the American Academy of Dermatology*, 50(4):655–656, 2004.
- [BGT11] Alexey N Bashkatov, Elina A Genina, and Valery V Tuchin. Optical properties of skin, subcutaneous, and muscle tissues: a review. *Journal of Innovative Optical Health Sciences*, 4(01):9–38, 2011.
- [BKZ⁺23] Haoran Bai, Di Kang, Haoxian Zhang, Jinshan Pan, and Linchao Bao. Ffhq-uv: Normalized facial uv-texture dataset for 3d face reconstruction. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 362–371. IEEE, 2023.
- [BL03] George Borshukov and J. P. Lewis. Realistic human face rendering for ”the matrix reloaded”. In *ACM SIGGRAPH 2003 Sketches & Applications*, pages 1–1, New York, NY, USA, 2003. ACM.

- [CG23] Shuangshuang Chen and Wei Guo. Auto-encoders in deep learning—a review with new perspectives. *Mathematics (Basel)*, 11(8):1777, 2023.
- [Chr15] Per H. Christensen. An approximate reflectance profile for efficient subsurface scattering. In *ACM SIGGRAPH 2015 Talks*, SIGGRAPH '15, New York, NY, USA, 2015. Association for Computing Machinery.
- [CNLE11] Cyril Crassin, Fabrice Neyret, Sylvain Lefebvre, and Elmar Eisemann. Interactive indirect illumination using voxel cone tracing. In *Computer Graphics Forum*, volume 30, pages 1921–1930. Wiley Online Library, 2011.
- [DBK⁺20] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. Technical report, Cornell University Library, arXiv.org, 2020.
- [DJ06] Craig Donner and Henrik Wann Jensen. A spectral bssrdf for shading human skin. *Rendering techniques*, 2006:409–418, 2006.
- [dRL11] Christophe de Rousiers and Sébastien Lagarde. Deferred rendering for current and future rendering pipelines. In *SIGGRAPH Courses*. ACM, 2011.
- [DS03] Carsten Dachsbacher and Marc Stamminger. Translucent shadow maps. In *Proceedings of the 14th Eurographics Workshop on Rendering*, pages 197–201. The Eurographics Association, 2003.
- [Epi21] Epic Games. Metahuman creator. Software, 2021. Available at <https://www.unrealengine.com/en-US/metahuman>.
- [Fac24] Faceform wrap software. <https://faceform.com/>, 2024. Accessed: 2024-06-26.
- [Fit88] Thomas B. Fitzpatrick. The validity and practicality of sun-reactive skin types i through vi. *Archives of dermatology (1960)*, 124(6):869–871, 1988.
- [FL13] Mark D. Fairchild and Wiley Online Library. *Color appearance models*. John Wiley and Sons, Inc, Chichester, West Sussex, third edition, 2013.
- [Fol96] James D. Foley. *Computer graphics: principles and practice*. Addison-Wesley, Boston, 2nd in c. edition, 1996.
- [FR98] Adrian Ford and Alan Roberts. Colour space conversions. *Westminster University, London*, 1998:1–31, 1998.

- [GBC16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. The MIT Press, Cambridge, Massachusetts, 2016.
- [GGD⁺20] Y. Gitlina, G. C. Guarnera, D. S. Dhillon, J. Hansen, A. Lattas, D. Pai, and A. Ghosh. Practical measurement and reconstruction of spectral skin reflectance. *Computer graphics forum*, 39(4):75–89, 2020.
- [Gia19] Carolyn Giardina. The rise of the all-digital actor, 2019.
- [Goo23] Google. Mediapipe face landmarker. https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker, 2023. Accessed: [Insert access date here].
- [HG41] L. C. Henyey and J. L. Greenstein. Diffuse radiation in the galaxy. *The Astrophysical journal*, 93:70, 1941.
- [HHAM18] Eric Haines, Naty Hoffman, and Tomas Akenine-Möller. *Real-Time Rendering, Fourth Edition*. CRC Press, Milton, 4;4th; edition, 2018.
- [IGAJG15] Jose A. Iglesias-Guitian, Carlos Aliaga, Adrian Jarabo, and Diego Gutierrez. A biophysically-based model of the optical properties of skin aging. *Computer graphics forum*, 34(2):45–55, 2015.
- [JCL⁺23] Joel Johnson, Kenneth Chau, Wei Sen Loi, Abraham Beauferris, Swati Kanwal, and Yingqian Gu. Deep albedo: A spatially aware autoencoder approach to interactive human skin rendering. In *SIGGRAPH Asia 2023 Posters*, SA '23, New York, NY, USA, 2023. Association for Computing Machinery.
- [JM91] Steven L Jacques and Daniel J McAuliffe. The melanosome: threshold temperature for explosive vaporization and internal absorption coefficient during pulsed laser irradiation. *Photochemistry and photobiology*, 53(6):769–775, 1991.
- [JMLH01] Henrik W. Jensen, Stephen R. Marschner, Marc Levoy, and Pat Hanrahan. A practical model for subsurface light transport. In *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques*, pages 511–518, New York, NY, USA, 2001. ACM.
- [JW95] Steven L Jacques and Lihong Wang. Monte carlo modeling of light transport in tissues. *Optical-thermal response of laser-irradiated tissue*, pages 73–100, 1995.
- [JZJ⁺15] Jorge Jimenez, Károly Zsolnai, Adrian Jarabo, Christian Freude, Thomas Auzinger, Xian-Chun Wu, Javier von der Pahlen, Michael Wimmer, and

- Diego Gutierrez. Separable subsurface scattering. *Computer Graphics Forum*, 2015.
- [Kaj86] James T. Kajiya. The rendering equation. *Computer graphics (New York, N. Y.)*, 20(4):143–150, 1986.
- [KB04] Aravind Krishnaswamy and Gladimir VG Baranoski. A biophysically-based spectral model of light interaction with human skin. In *Computer graphics forum*, volume 23, pages 331–340. Wiley Online Library, 2004.
- [KK18] Goanghun Kim and Hyeong-Seok Ko. A practical approach to physically-based reproduction of diffusive cosmetics. *Computer graphics forum*, 37(7):223–232, 2018.
- [LBB⁺17] Tianye Li, Timo Bolkart, Michael J. Black, Hao Li, and Javier Romero. Learning a model of facial shape and expression from 4D scans. *ACM Transactions on Graphics, (Proc. SIGGRAPH Asia)*, 36(6):194:1–194:17, 2017.
- [LLZ⁺18] Jiangang Liu, Hong Luo, Paul P. Zheng, Si J. Wu, and Kang Lee. Transdermal optical imaging revealed different spatiotemporal patterns of facial cardiovascular activities. *Scientific reports*, 8(1):10588–10, 2018.
- [MHP⁺07] Wan-Chun Ma, Tim Hawkins, Pieter Peers, Charles-Felix Chabert, Malte Weiss, and Paul Debevec. Rapid acquisition of specular and diffuse normal maps from polarized spherical gradient illumination. In *Eurographics Symposium on Rendering*, 2007.
- [MKS⁺19] Jörg Marotz, Axel Kulcke, Frank Siemers, Diogo Cruz, Ahmed Aljowder, Dominik Promny, Georg Daeschlein, and Thomas Wild. Extended perfusion parameter estimation from hyperspectral imaging data for bedside diagnostic in medicine. *Molecules (Basel, Switzerland)*, 24(22):4164, 2019.
- [MMC21] Samuel A. Minaker, Ryan H. Mason, and David R. Chow. Optimizing color performance of the ngenuity 3-dimensional visualization system. *Ophthalmology science (Online)*, 1(3):100054–100054, 2021.
- [MPS21] Araceli Morales, Gemma Piella, and Federico M. Sukno. Survey on 3d face reconstruction from uncalibrated images. *Computer science review*, 40:100400, 2021.
- [MVG⁺13] Nicholas MacKinnon, Fartash Vasefi, Eugene Gussakovsky, Gregory Bearman, Robert Chave, and Daniel L. Farkas. In vivo skin chromophore mapping using a multimode imaging dermoscope (skinspectm). In Daniel L.

- Farkas, Dan V. Nicolau, and Robert C. Leif, editors, *Proceedings of SPIE - The International Society for Optical Engineering*, volume 8587, pages 85870U–85870U–13. SPIE, 2013.
- [NBH⁺08] James J Nordlund, Raymond E Boissy, Vincent J Hearing, Richard A King, William S Oetting, and Jean-Paul Ortonne. *The pigmentary system: physiology and pathophysiology*. John Wiley & Sons, 2008.
- [NMM⁺19] Augusto LP Nunes, Anderson Maciel, Gary W Meyer, Nigel W John, Gladimir VG Baranoski, and Marcelo Walter. Appearance modelling of living human tissues. *Computer Graphics Forum*, 38:43–65, 2019.
- [ORL06] Noboru Ohta, Alan R. Robertson, and Wiley Online Library. *Colorimetry: fundamentals and applications*. J. Wiley, Hoboken, NJ, USA;Chichester, West Sussex, England;, 1. aufl.;1; edition, 2005;2006;.
- [Pal99] Stephen E. Palmer. *Vision science: photons to phenomenology*. MIT Press, Cambridge, MA, 1999.
- [PJH16] Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation*. Morgan Kaufmann, 3 edition, 2016.
- [PJH17] Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation, Third Edition*. Elsevier Science and Technology Books, Inc, Place of publication not identified, third edition, 2017.
- [RGK⁺09] Tobias Ritschel, Thorsten Grosch, Min H. Kim, Jan Kautz, Carsten Dachsbacher, and Hans-Peter Seidel. Approximating dynamic global illumination in image space. In *Proceedings of the 2009 symposium on Interactive 3D graphics and games*, pages 75–82. ACM, 2009.
- [RKOAJ08] Erik Reinhard, Erum A. Khan, Ahmet Oguz Akyuz, and Garrett Johnson. *Color Imaging: Fundamentals and Applications*. A K Peters/CRC Press, Milton, 1 edition, 2008.
- [SG31] T. Smith and J. Guild. The c.i.e. colorimetric standards and their use. *Transactions of the Optical Society*, 33(3):73–134, 1931.
- [SGDM11] Giota Stratou, Abhijeet Ghosh, Paul Debevec, and Louis-Philippe Morency. Effect of illumination on automatic expression recognition: A novel 3d relightable facial database. In *IEEE International Conference on Automatic Face and Gesture Recognition (FG)*, Santa Barbara, USA, March 2011.

- [SHZ⁺18] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4510–4520. IEEE, 2018.
- [SJT95] I. S. Saidi, S. L. Jacques, and F. K. Tittel. Mie and rayleigh modeling of visible-light scattering in neonatal skin. *Applied optics (2004)*, 34(31):7410–7418, 1995.
- [SKS02] Peter-Pike Sloan, Jan Kautz, and John Snyder. Precomputed radiance transfer for real-time rendering in dynamic, low-frequency lighting environments. *ACM Transactions on Graphics*, 21:527–536, 2002.
- [SLI⁺23] Eric Saindon, Joe Letteri, Nicholas Illingworth, Sarah Fitton, Richard Baneham, and Marco Revelant. Wētā fx presents avatar: The way of water. In *ACM SIGGRAPH 2023 Talks*, pages 1–1, New York, NY, USA, 2023. ACM.
- [SWD05] Gaurav Sharma, Wencheng Wu, and Edul N. Dalal. The ciede2000 color-difference formula: Implementation notes, supplementary test data, and mathematical observations. *Color research and application*, 30(1):21–30, 2005.
- [Tex24] Texturing XYZ textures. <https://texturing.xyz/>, 2024. Accessed: 2024-06-26.
- [Toe22] Rob Toews. Synthetic data is about to transform artificial intelligence. *Forbes*, Jun 2022.
- [USC24] USC Institute for Creative Technologies. 3drf light cage face scan dataset. <https://vgl.ict.usc.edu/Data/3DRFE/>, 2024. Accessed: 2024-06-26.
- [VSP⁺23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.
- [WBH⁺21] Erroll Wood, Tadas Baltrusaitis, Charlie Hewitt, Sebastian Dziadzio, Thomas J. Cashman, and Jamie Shotton. Fake it till you make it: face analysis in the wild using synthetic data alone. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3661–3671. IEEE, 2021.
- [WSS13] Chris Wyman, Peter-Pike Sloan, and Peter Shirley. Simple analytic approximations to the cie xyz color matching functions. *Journal of Computer*

Graphics Techniques, July 2013. Available at: [Paper PDF Link], [Supplementary Code Link].

- [YWP⁺18] Changqian Yu, Jingbo Wang, Chao Peng, Changxin Gao, Gang Yu, and Nong Sang. Bisenet: Bilateral segmentation network for real-time semantic segmentation. In *Computer Vision – ECCV 2018*, pages 334–349, Cham, 2018. Springer International Publishing.
- [YZW⁺20] Haotian Yang, Hao Zhu, Yanru Wang, Mingkai Huang, Qiu Shen, Ruigang Yang, and Xun Cao. Facescape: a large-scale high quality 3d face dataset and detailed riggable 3d face prediction. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

Appendices

Appendix A

Figures

Simulation Results

Below is a visualization of the LUT data using color maps for $T = 2.5mm$ and varying Bm and Bh values. This is a good demonstration of the asymmetry of the impact each variable has on the resulting color.

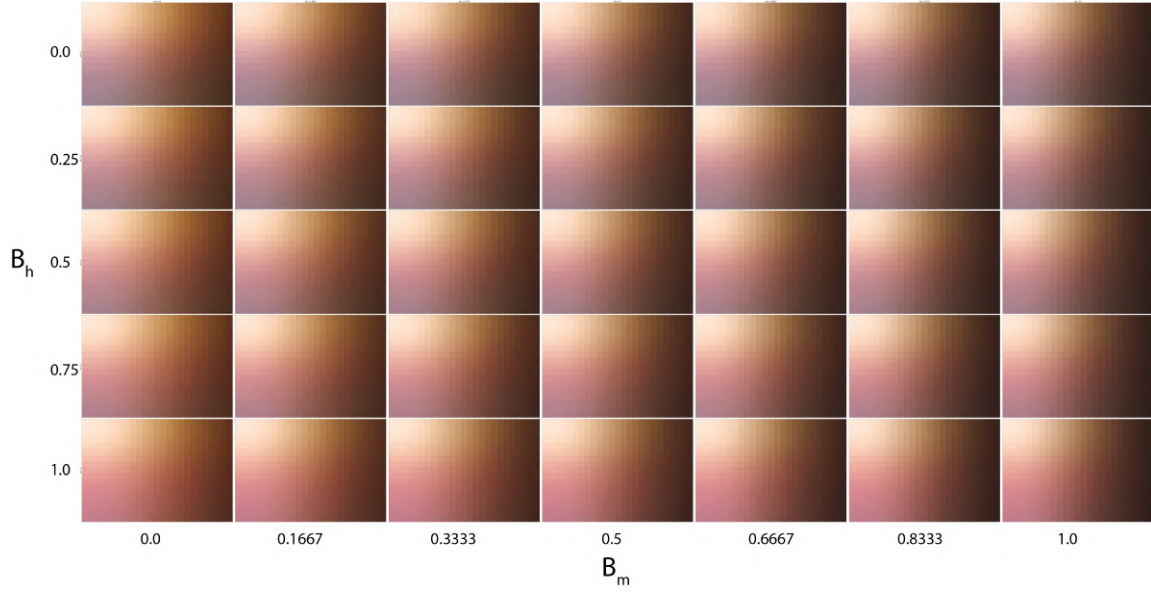


Figure 2: Cm vs Ch color maps for fixed $T = 2.5mm$ and varying Bm and Bh

Cylindrical Reflectance Bins

The bins used to store escaped photon weights are in cylindrical coordinates. Where the outermost radial bin also acts as an overflow, as in previous works [JW95]. This boundary is arbitrary for our work, as we collect all weights regardless of radius as diffuse reflectance. For multi-angle of incidence, the radius becomes a key variable. One use for such a plot of

escaped photon weights is in the creation of subsurface diffusion profiles to mimic scattering in real time, a common practice in game engines.

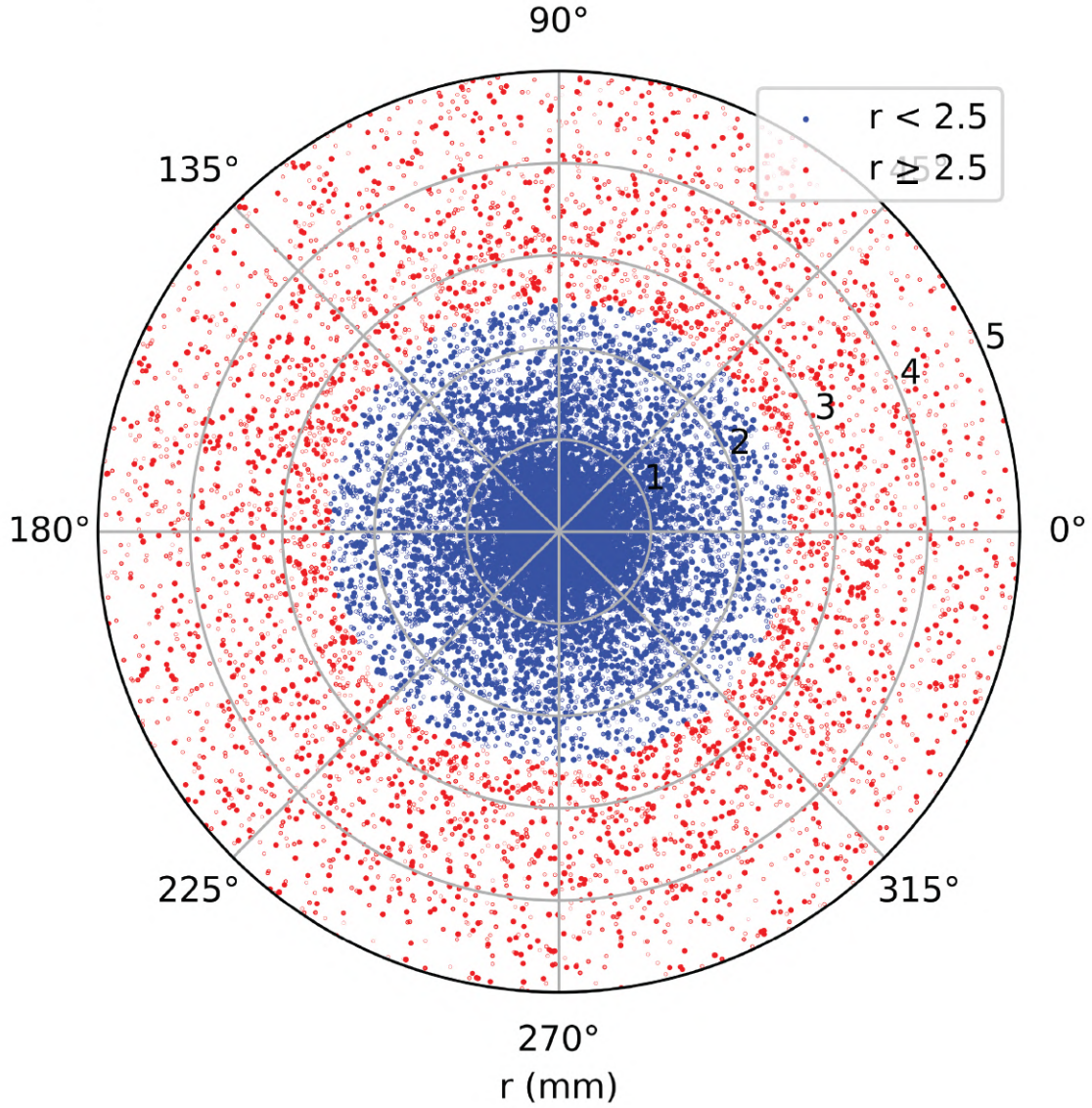


Figure 3: Two-dimensional scatter plot of escaped photon weights in polar coordinates. With the red weights, representing “overflow” weights, are placed in the outermost reflectance bins [JW95].

SAE Results

Results of the SAE on diffuse light cage images from the 3DRF dataset [USC24, SGDM11, MHP⁺07].

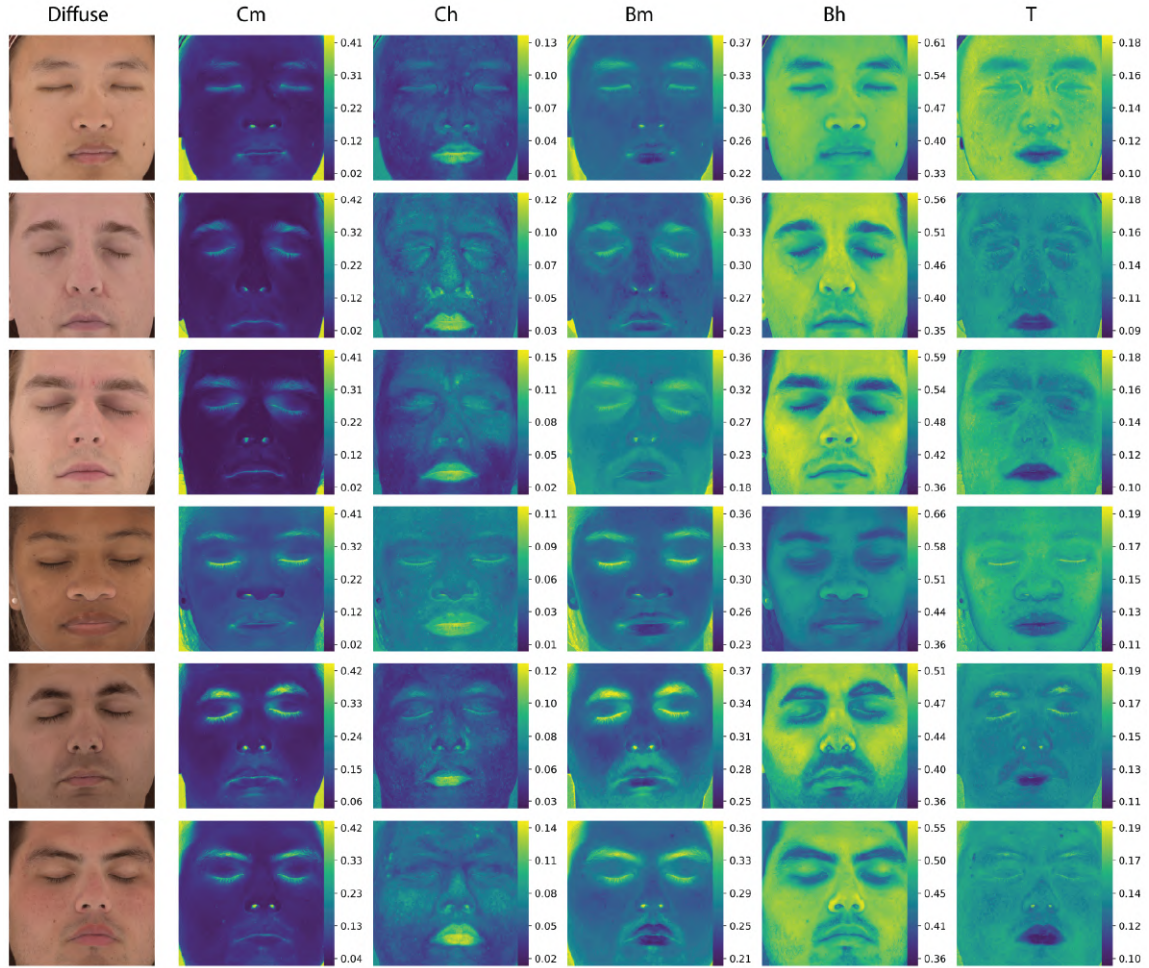


Figure 4: Diffuse light cage images of human faces [USC24, SGDM11, MHP⁺07], decomposed into biophysical parameters using the SAE.